

VOLUME 13 / RUST BACKEND MASTERY

完全コード — 演習・解答・用語集・索引

RustNote の全ファイルを順に読み、自分の PC で同じ結果を出して確かめる最終巻

第13巻: 完全コード — 演習・解答・用語集・索引

この巻で身に付けること

第12巻までで、RustNote を設計し（第9巻）、作り（第10巻）、確かめ（第11巻）、公開する（第12巻）までの流れを学びました。どの巻も、説明に必要な部分だけを抜き出して見せています。だから、「全部をつなげると、結局どのファイルに何が書いてあるのか」は、まだぼんやりしているかもしれません。

この巻は、教材のフォルダに入っている**完成した RustNote のソース**（`Rust/rustnote/`）を、**全部のファイルについて**、上から順に読んでいく巻です。ただ貼り付けて眺めるのではなく、ファイルごとに次の3つを確かめます。

- そのファイルは、**何を守るために**あるのか（どんな事故を防いでいるのか）
- **誰が読む**のか（cargo・rustup・rustc・ブラウザ・Docker・Fly.io など）
- 書き間違えたり消したりすると、**何が起きる**のか（実際のエラーで確かめる）

巻の最後には、全巻をまたぐ総合演習と解答、全巻の用語集、索引を置きます。

到達目標

この巻を終えると、次のことができるようになります。

- RustNote の 42 個のファイルについて、それぞれの役割と、読む道具（cargo・rustc・Docker など）を説明できる
- 教材と同じ版の道具と依存クレートで、RustNote を自分の PC でゼロからビルドし、テスト・整形・Clippy の確かめを通せる
- 1つのファイルを書き換えたときに、どのコマンドの、どんなエラーで気づけるかを予想し、実際に確かめられる
- 総合演習で、新しい機能を足すときの置き場所と、壊していないことの確かめ方を、自分で決められる

学習地図

10 個の単元で、RustNote のファイルを「土台から上へ」の順に読みます。最初は版を決める設定ファイル、次に DB の表、起動、型、保存、ログイン、画面、見た目、最後にテストと公開です。

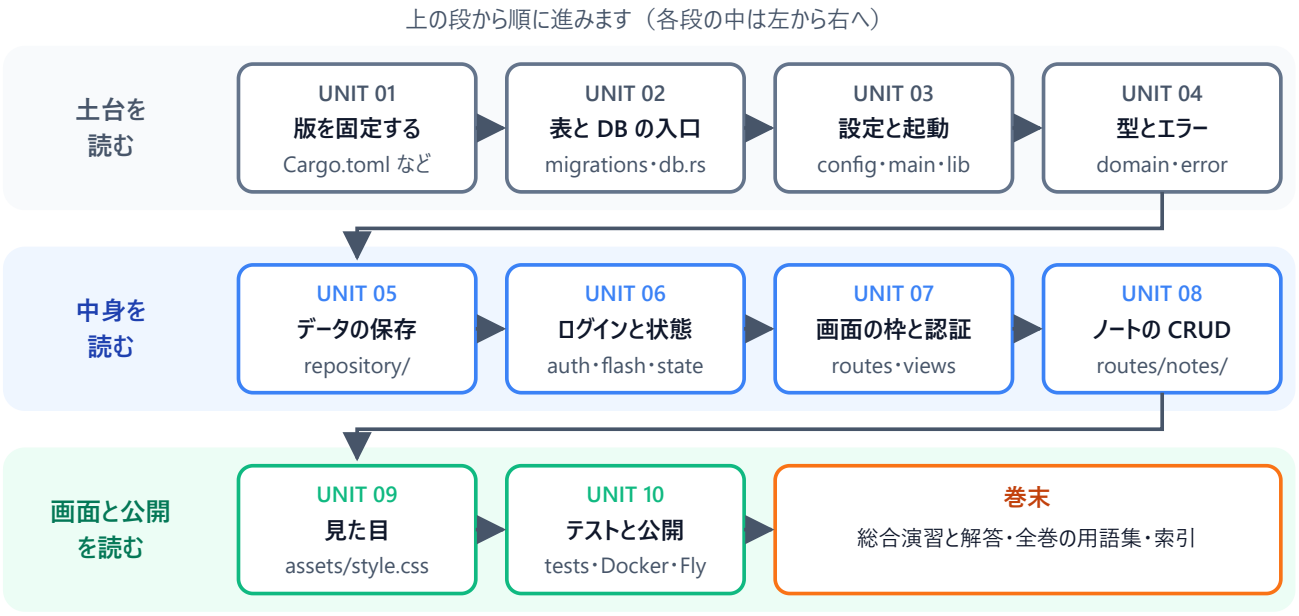


図13-1: 第13巻の学習地図。土台の設定ファイルから読み始め、型・保存・ログイン・画面を経て、テストと公開で終わる。巻末に総合演習・用語集・索引を置く

単元	テーマ	読むファイル
UNIT 01	版を固定する設定ファイル	`Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml`・`rustfmt.toml`・`.gitignore`・`.env.example`
UNIT 02	表を作る SQL と DB の入口	`migrations/0001〜0003`・`src/db.rs`
UNIT 03	設定と起動の順番	`src/config.rs`・`src/main.rs`・`src/lib.rs`
UNIT 04	ドメインの型とエラーの分類	`src/domain.rs`・`src/error.rs`
UNIT 05	リポジトリ	`src/repository.rs`・`src/repository/` の3つ
UNIT 06	ログイン・お知らせ・共有の状態（準備中）	`src/auth.rs`・`src/flash.rs`・`src/state.rs`
UNIT 07	画面の枠・部品・登録とログインの画面（準備中）	`src/routes.rs`・`src/views.rs`・`src/routes/` の signup・login・logout・health
UNIT 08	ノートの CRUD（準備中）	`src/routes/notes.rs`・`src/routes/notes/` の4つ
UNIT 09	見た目（準備中）	`assets/style.css`
UNIT 10	テスト・Docker・Fly.io・CI（準備中）	`tests/` の3つ・`Dockerfile`・`.dockerignore`・`fly.toml`・`ci.yml`・`README.md`

> **お知らせ:** この版には UNIT 01〜05 を収録しています。UNIT 06〜10 と、巻末の総合演習・解答・全巻の用語集・索引は準備中で、次の更新から順に追加します。

RustNote の全ファイル

RustNote は、次の 42 個のファイルでできています（`git ls-files` で数えた、Git で管理しているファイルです）。行数は `Cargo.lock` を除くと約 4,600 行です。

ファイル	行数	単元	ひとことで言うと
`Cargo.toml`	35	01	使うクレートとその版の一覧
`Cargo.lock`	3,753	01	実際に選ばれた全クレートの版と、中身の指紋（自動で作られる）
`rust-toolchain.toml`	5	01	このフォルダで使う Rust の版（1.99.1）
`rustfmt.toml`	2	01	`cargo fmt` の設定
`.gitignore`	15	01	Git に入れないファイル（ビルドの成果物・DB・`.env`）
`.env.example`	15	01	環境変数の見本
`migrations/0001_users.sql` ほか2つ	9・18・12	02	表を作る SQL
`src/db.rs`	185	02	DB への接続とマイグレーション
`src/config.rs`・`src/main.rs`・`src/lib.rs`	182・54・27	03	設定の読み込み・起動の入口・部品の一覧
`src/domain.rs`・`src/error.rs`	518・152	04	検査済みの型と、エラーの分類
`src/repository.rs` と `src/repository/` の3つ	39・243・85・112	05	ノート・ユーザー・セッションの読み書き
`src/auth.rs`・`src/flash.rs`・`src/state.rs`	179・117・42	06	ログイン・一度だけのお知らせ・共有の状態
`src/routes.rs`・`src/views.rs`	142・249	07	ルーターとレイアウト・画面の部品
`src/routes/` の signup・login・logout・health	162・89・22・34	07	登録・ログイン・ログアウト・ヘルスチェック
`src/routes/notes.rs` と `src/routes/notes/` の4つ	145・80・113・29・27	08	ノートの一覧・作成・詳細・編集・完了の切り替え・削除
`assets/style.css`	290	09	画面のスタイル
`tests/common/mod.rs`・`tests/http.rs`・`tests/repository.rs`	159・398・248	10	テストの道具と、41 件のテストのうち 22 件
`Dockerfile`・`.dockerignore`・`fly.toml`・`ci.yml`・`README.md`	50・7・38・48・192	10	コンテナ・公開・CI の設定と説明書

- 残りの 19 件のテストは、`src/` の中のファイルの最後にある `#[cfg(test)]` のユニットテストです。それぞれのファイルの単元で読みます。

- 名前が `.` で始まるファイル（`.gitignore` など）は、Windows の PowerShell やエクスプローラーではそのまま表示されます。macOS や Linux の `ls` では隠れるので、`ls -a` で表示します。

前提知識（先に読んでおく巻）

この巻は、全巻のまとめです。どの単元も、元になった巻を「第9巻 UNIT 04」のように示します。

- **第1巻（環境構築）**: Rust・VS Code・Git が使えること。ARM64 版の Windows では、LLVM（clang）も入れておきます（第9巻の巻頭にも手順があります）。
- **第2巻〜第5巻**: Rust の文法、所有権、型と trait とエラー、Cargo と features（第4巻 UNIT 07）、async。
- **第6巻〜第8巻**: Topcoat、Turso・libSQL と SQL、ログインとセッション。
- **第9巻〜第12巻**: RustNote の設計・実装・品質の確かめ・公開。この巻は、その完成形を読む巻です。

> **注意**: 第12巻は、書き直しの途中です。この巻の本文は、教材のフォルダの `Rust/rustnote/` のコードと、この巻で実際に動かした結果だけをもとに書いているので、ほかの巻が未完成でも読み進められます。

この巻の進め方

完成したコードを読むときは、つい「読んで分かったつもり」になりがちです。各単元では、次のサイクルで進めてください。

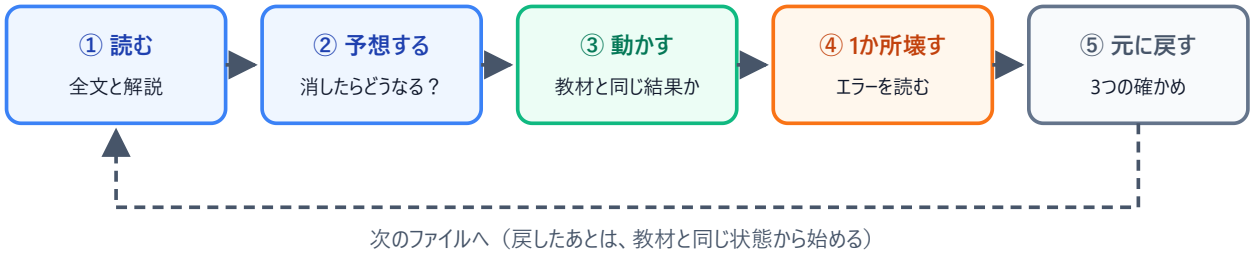


図13-2: この巻の進め方。読んで、予想して、動かして、1か所だけ壊してエラーを読み、元に戻してから次のファイルへ進む

1. **読む**: ファイルの全文と解説を読みます。
2. **予想する**: 「この行を消したら、どのコマンドが、どんなエラーを出すか」を、先に予想します。
3. **動かす**: 教材のコマンドをそのまま実行し、結果が教材と同じか比べます。
4. **1か所だけ壊す**: 「よくあるエラーと読み方」の節のとおりに、1か所だけ書き換えて、エラーを自分の目で見ます。
5. **元に戻す**: 書き換えたファイルを元に戻し、次の3つのコマンドが全部通ることを確かめてから、次のファイルへ進みます。

```
# 見た目が Rust の標準の形か（直すときは --check を外す）
cargo fmt --check
# よくある書き間違いを、警告ではなくエラーとして止める。--locked は Cargo.lock を書き換えさせない
cargo clippy --all-targets --locked -- -D warnings
# テストを全部実行する
cargo test --locked
```

この3つは、RustNote の `ci.yml`（GitHub Actions の設定の例）で、コードを送るたびに自動で実行されるのと同じコマンドです（UNIT 10（準備中））。

> **ヒント:** 元に戻すのがいちばん確実なのは、教材のフォルダから、そのファイルだけをもう一度コピーすることです。練習用のフォルダを Git で管理しておく（`git init` して最初に全部をコミットしておく）と、`git diff` で変えた所を確かめ、`git checkout -- ファイル名` で元に戻せます（UNIT 01 の問3）。

使うツールと版

この巻のコードは、すべて次の環境で実際にビルド・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.99 以上**（本書シリーズの基準。Topcoat 0.11 が 1.99 以上を求めます）。動作確認は rustc 1.99.1 / cargo 1.99.1（`aarch64-pc-windows-msvc`）
edition	2024
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では中身を変えずに読む
直接使うクレート	topcoat 0.11.0、libsql 0.10.0、tokio 1.54.1、argon2 0.6.0、async-trait 0.1.92、serde 1.0.229、thiserror 2.0.21、anyhow 1.0.104、dotenvy 0.15.7、tempfile 3.27.0（テスト用）
topcoat-cli	0.11.0（UNIT 09・10 で使います。UNIT 01 では使いません）

練習用のフォルダを用意する

第9巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。この巻では、わざとファイルを書き換えてエラーを見るので、お手本そのものには触らないことが特に大切です。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる
rustup show active-toolchain
```

```
1.99.1-aarch64-pc-windows-msvc (overridden by '...\rustnote\rust-toolchain.toml')
```

- `"教材のフォルダ\Rust\rustnote"` の部分は、自分の PC で教材を置いた場所に読み替えてください。
- 第9巻で使ったコピーがあれば、それとは別の、新しいコピーを作ってください。第9巻のコピーには、確かめ用のテストを足してあるからです。

UNIT 01: 版を固定する設定ファイル

この単元のゴール

- `Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml`・`rustfmt.toml`・`.gitignore`・`.env.example` の6つのファイルについて、それぞれ誰が読み、何を決めているかを説明できる
- `"=1.53.3"` と `"1.53.3"` の版の書き方の違いと、`Cargo.lock` と `--locked` が「誰の PC でも同じ物を作る」ために何をしているかを説明できる
- features（クレートの機能のスイッチ）は、依存のつながり全体で**足し算**されることを、実際の出力で確かめられる
- 版が古い・機能の名前の打ち間違い・`Cargo.lock` の食い違いなどのエラーを読み、どのファイルを直せばよいかを判断できる

たとえ話でつかむ

料理教室で、全員に同じケーキを焼いてもらう場面を考えます。先生は、次のような紙を配ります。

- **材料表**には「小麦粉 A 社の 1.53.3 番」「卵 M サイズ」のように、銘柄と規格まで書いてあります。「小麦粉 適量」では、人によって違うケーキになるからです。
- **買い物のレシート**には、実際に買った品物が全部、ロット番号つきで並んでいます。材料表に書いていない、小麦粉の袋の中に入っていた乾燥剤まで載っています。次に買うときも、同じロットを選べます。
- **オーブンの型番**も決めておきます。型番が古いオーブンでは、焼けない温度があるからです。
- **盛り付けの決まり**があれば、誰が作っても同じ見た目になります。
- **持ち帰らない物の札**を、生ごみや、家の鍵に付けておきます。
- **鍵の置き場所のメモ**には「玄関の鍵は、ここに置く」とだけ書きます。鍵そのものは、紙に貼りません。

RustNote の6つの設定ファイルは、この紙と同じ役目をしています。

料理教室	RustNote のファイル	読むもの
材料表（銘柄と規格）	`Cargo.toml`（使うクレートと版）	cargo
買い物のレシート（ロット番号つき）	`Cargo.lock`（選ばれた全クレートの版と指紋）	cargo
オーブンの型番	`rust-toolchain.toml`（Rust の版）	rustup
盛り付けの決まり	`rustfmt.toml`（整形の設定）	rustfmt（`cargo fmt`）
持ち帰らない物の札	`.gitignore`（Git に入れないファイル）	Git
鍵の置き場所のメモ	`.env.example`（環境変数の見本。秘密の値は書かない）	人（コピーして `.env` にする）

> **用語:** 版の固定（はんのこてい：使う道具や部品の版を、1つに決めて書いておくこと。英語では pin（ピン留め）とも言う）

> **用語:** 依存クレート（いぞんクレート：自分のプログラムが使う、ほかの人が作ったクレート。そのクレートが使うクレートも、間接的な依存になる）

図で理解する

6つのファイルを、読む道具ごとに並べると、次のようになります。

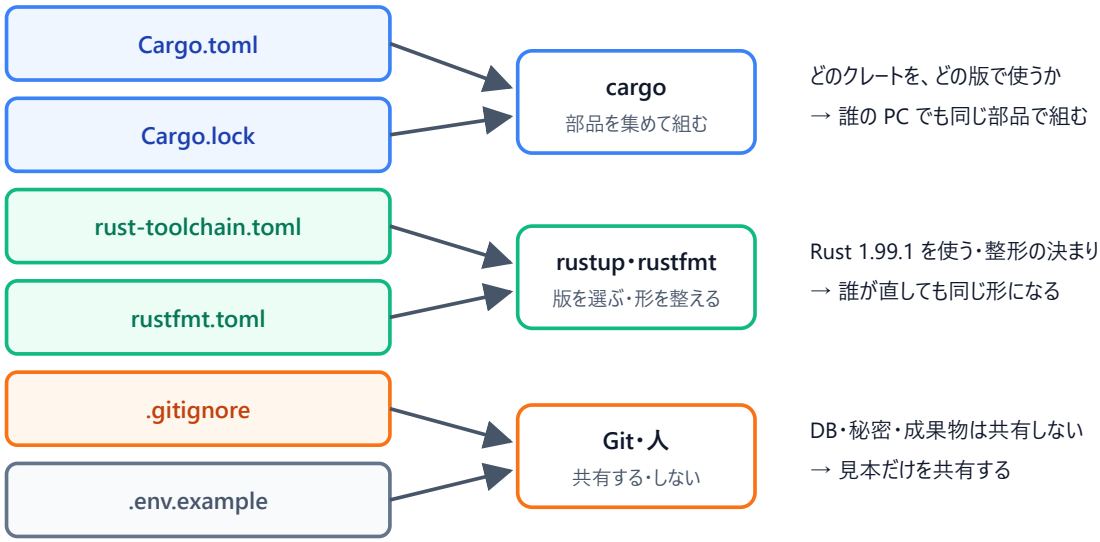


図13-3: 6つの設定ファイルと、それを読む道具。cargo が部品をそろえ、rustup と rustfmt が版と形をそろえ、Git と人が「共有しない物」を分ける

- **青の2つ**（`Cargo.toml`・`Cargo.lock`）は、cargo が読みます。使う部品と、その版を決めます。

- **緑の2つ**（`rust-toolchain.toml`・`rustfmt.toml`）は、rustup と rustfmt が読みます。Rust そのものの版と、コードの見た目を決めます。
- **橙と灰の2つ**（`.gitignore`・`.env.example`）は、Git と人のためのファイルです。共有してはいけない物（DB のファイル・秘密の値）を、共有する物から分けます。

`Cargo.toml` に書いたクレートは 10 個だけですが、それぞれが別のクレートを使っているので、実際に必要になる部品はずっと多くなります。



Cargo.toml は人が書く「材料表」、Cargo.lock は cargo が書く「レシート」

図13-4: 直接使う 10 個のクレートから、cargo が版を選んで 361 個の一覧（Cargo.lock）を作り、この PC（Windows ARM64）でのビルドには、そのうち 289 個を使う

- 361 個は、`Cargo.lock` の `[[package]]` の数です（RustNote 自身を含みます）。289 個は、この PC で `cargo build` したときに表示された `Compiling` の行の数です。この PC では使わない、ほかの OS 用のクレート（Linux や macOS でだけ使う物など）と、テストでだけ使うクレートは、`cargo build` ではコンパイルされません。
- 数はこれから先の版で変わりますが、「直接書いたのは少しでも、実際の部品はその何十倍にもなる」ことは変わりません。だから、版を人の記憶に頼らず、ファイルで固定しておくのです。

まず動かす最小コード

この単元の「コード」は、設定ファイルそのものです。まず全文を読み、次に、その設定で RustNote がビルドでき、確かめの3つのコマンドが通ることを確かめます。

例1-1: Cargo.toml の全文

> **▼ このファイルがやること（先に日本語で）**：パッケージの名前と版、使う Rust の最低の版（1.99）、使うクレート（9個）とテストでだけ使うクレート（1個）を、版を `=` で1つに固定して書きます。tokio と serde には、使う機能（features）も書きます。

```toml
[package]
name = "rustnote"
version = "0.1.0"
edition = "2024"
# Topcoat 0.11 は Rust 1.99 以上が必要（動作確認は 1.99.1）
rust-version = "1.99"
description = "RustNote: Topcoat + libSQL で作るログイン付きメモアプリ（教材のサンプル）"
publish = false
default-run = "rustnote"

# 教材と同じ結果になるよう、直接使うクレートの版は `=` で固定している。
# 間接的に使われるクレートの版は Cargo.lock で固定される。
[dependencies]
# アプリの入口（main）でエラーに文脈を付けて表示する
anyhow = "=1.0.104"
# パスワードのハッシュ化（Argon2id）
argon2 = "=0.6.0"
# trait に async fn を書き、dyn（Arc<dyn NoteRepository + Send + Sync>）で使えるようにする
async-trait = "=0.1.92"
# .env ファイルを環境変数として読み込む（開発用）
dotenvy = "=0.15.7"
# Turso / libSQL のクライアント（ローカルファイルとリモートの両方）
libsql = "=0.10.0"
# フォームや JSON を構造体に変換する
serde = { version = "=1.0.229", features = ["derive"] }
# エラー型を短く書く
thiserror = "=2.0.21"
# 非同期ランタイム
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros"] }
# Web フレームワーク
topcoat = "=0.11.0"

[dev-dependencies]
# テストごとに使い捨ての一時フォルダ（DB ファイル置き場）を作る
tempfile = "=3.27.0"
```

▼ コードを1つずつ分解して解説

解説1: [package] は、このパッケージの名札

- `name = "rustnote"` は、パッケージの名前です。ライブラリを外から使うときの `rustnote::domain` の `rustnote` は、この名前です（第9巻 UNIT 04）。
- `edition = "2024"` は、Rust の書き方の世代です。同じ Rust 1.99.1 でも、edition が違うと、使える書き方や警告が少し変わります。

- `rust-version = "1.99"` は、「このパッケージをビルドするには、Rust 1.99 以上が必要」という宣言です。Topcoat 0.11 が 1.99 以上を求めているので、それに合わせています。これより古い Rust でビルドしようとすると、コンパイルを始める前に止まります（エラー1）。
- `publish = false` は、`cargo publish` で crates.io（クレートの公開場所）に**うっかり公開しない**ための印です。RustNote は教材のアプリなので、公開しません。
- `default-run = "rustnote"` は、`cargo run` で動かす実行ファイルを決めます。RustNote の実行ファイルは `src/main.rs` の1つだけなので、無くても同じように動きます。あとで `src/bin/` に道具の実行ファイルを足したときも、`cargo run` が RustNote 本体を選ぶように、先に書いてあります。

解説2: "=1.54.1" は「この版だけ」

- `anyhow = "=1.0.104"` のように、版の前に `=` を付けると、**その版だけ**を使います。`=` を付けずに `"1.0.104"` と書くと、「1.0.104 以上、2.0.0 未満」の意味になり、cargo はその範囲で新しい版を選んでよくなります（例1-3で確かめます）。
- RustNote は、教材の出力と同じ結果になるように、直接使うクレートを全部 `=` で固定しています。`Cargo.toml` の上のコメント「教材と同じ結果になるよう、直接使うクレートの版は `=` で固定している」が、その理由です。
- 一方で、間接的に使われるクレートの版は、`Cargo.toml` には書きません。それを固定するのが `Cargo.lock` です（解説4）。

解説3: features は、クレートの機能のスイッチ

- `tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros"] }` は、tokio の機能のうち、複数のスレッドで動くランタイム（`rt-multi-thread`）と、`#[tokio::main]` や `#[tokio::test]` を書くためのマクロ（`macros`）を使う、という意味です（第4巻 UNIT 07、第5巻 UNIT 02）。
- `serde = { version = "=1.0.229", features = ["derive"] }` の `derive` は、`#[derive(Deserialize)]` と書いてフォームの値を構造体に変える機能です。
- 機能を細かく選べるのは、使わない機能のぶんだけ、コンパイルの時間と、できあがるプログラムの大きさを減らせるからです。ただし、features には「依存のつながり全体で足し算される」という性質があります（例1-4）。

解説4: [dev-dependencies] は、テストのときだけ

- `tempfile` は、テストごとに使い捨ての一時フォルダを作るクレートです。`[dev-dependencies]` に書いたクレートは、`tests/`・`examples/`・`#[cfg(test)]` のテストからだけ使えます。本番の実行ファイルには入りません（問1で確かめます）。

例1-2: rust-toolchain.toml と rustfmt.toml の全文

> **▼ このファイルがやること（先に日本語で）：** `rust-toolchain.toml` は、このフォルダの中で `cargo` や `rustc` を実行したときに使う Rust の版を 1.99.1 に決め、rustfmt と clippy も一緒に入れるよう指定します。`rustfmt.toml` は、`cargo fmt` が edition 2024 の決まりで整形するよう指定します。

```
# このフォルダの中で cargo を実行すると、Rust 1.99.1 が使われる
#  （既定のツールチェーンは変更しない）
[toolchain]
channel = "1.99.1"
components = ["rustfmt", "clippy"]
```

```
# rustfmt（cargo fmt / topcoat fmt --rustfmt）の設定
edition = "2024"
```

`Cargo.toml` の `rust-version = "1.99"` は「1.99 以上なら何でもよい」、`rust-toolchain.toml` の `channel = "1.99.1"` は「このフォルダでは 1.99.1 を使う」です。最低の版と、実際に使う版を、別々のファイルで決めています。

例1-3: 決めた版でビルドし、3つの確かめを通す

> **▼ このコマンドがやること（先に日本語で）**：コピーしたフォルダで、`Cargo.lock` を書き換えないようにして（`--locked`）、RustNote をビルドします。続いて、整形の確かめ・Clippy・テストの3つを、`ci.yml` と同じ形で実行します。

```
# このフォルダで使われる Rust の版を確かめる
rustup show active-toolchain
# Cargo.lock のとおりの版でビルドする
cargo build --locked
# 確かめの3つ（ci.yml と同じ）
cargo fmt --check
cargo clippy --all-targets --locked -- -D warnings
cargo test --locked
```

▼ 実行結果（`cargo build --locked` は `Compiling` の行が 289 行続くので、最後の3行だけを載せています）

```
1.99.1-aarch64-pc-windows-msvc (overridden by '...\rustnote\rust-toolchain.toml')
```

```
   Compiling libsql v0.10.0
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 54.58s
```

`cargo fmt --check` は、整形が必要な所が無ければ、何も表示しません。`cargo clippy` の最後の3行と、`cargo test` の区切りと結果の行です。

```
    Checking libsql v0.10.0
    Checking rustnote v0.1.0 (...\rustnote)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 32.68s
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 13.22s
     Running unittests src\lib.rs (target\debug\deps\rustnote-c39f4ee352b39312.exe)
test result: ok. 19 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished
in 0.00s
     Running unittests src\main.rs (target\debug\deps\rustnote-009228cf007e52ec.exe)
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
     Running tests\http.rs (target\debug\deps\http-afc265ac134dcb62.exe)
test result: ok. 14 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished
in 2.32s
     Running tests\repository.rs (target\debug\deps\repository-74d08b56e783b6bc.exe)
test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.24s
   Doc-tests rustnote
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

▼ コードを1つずつ分解して解説

解説1: overridden by は「フォルダの設定で上書きした」

- `rustup show active-toolchain` の `(overridden by '...\rustnote\rust-toolchain.toml')` は、「PC の既定の版ではなく、このファイルで決めた版を使っています」という意味です。この PC の既定の版は 1.97.1 ですが、RustNote のフォルダでは 1.99.1 が使われています。
- だから、PC ごとに既定の版が違っても、RustNote のフォルダの中では、全員が同じ 1.99.1 でビルドします。

解説2: --locked は「Cargo.lock を書き換えるな」

- `cargo build` は、`Cargo.toml` と `Cargo.lock` が食い違っていると、`Cargo.lock` を自動で書き直してからビルドします。`--locked` を付けると、書き直す必要があるときは、ビルドせずにエラーで止まります（エラー3）。
- 「レシートと違う物を、黙って買ってこない」ための指定です。RustNote の `Dockerfile`（`cargo build --release --locked`）と `ci.yml` も、`--locked` を付けています。

解説3: 3つの確かめは、全部「何も言われない」が合格

- `cargo fmt --check` は、何も表示しないのが合格です。`cargo clippy ... -- -D warnings` は、警告が1つでもあればエラーにします。`Finished` で終われば合格です。`cargo test --locked` は、全部の `test result:` が `ok.` なら合格です。
- RustNote のテストは、ユニットテスト 19 件、`tests/http.rs` の 14 件、`tests/repository.rs` の 8 件、合わせて 41 件です。

もう一歩: 実務に近い形

例1-4: 直接の依存を一覧にする

> **▼ このコマンドがやること（先に日本語で）:** `cargo tree` は、依存のつながりを木の形で表示します。`--depth 1` で、RustNote が直接使うクレートだけに絞ります。

```
cargo tree --locked --depth 1
```

▼ 実行結果

```
rustnote v0.1.0 (...\rustnote)
├── anyhow v1.0.104
├── argon2 v0.6.0
├── async-trait v0.1.92 (proc-macro)
├── dotenvy v0.15.7
├── libsql v0.10.0
├── serde v1.0.229
├── thiserror v2.0.21
├── tokio v1.54.1
└── topcoat v0.11.0
[dev-dependencies]
└── tempfile v3.27.0
```

- `Cargo.toml` に書いた 10 個が、`Cargo.lock` で選ばれた版で並んでいます。`[dev-dependencies]` の下が、テストでだけ使う `tempfile` です。
- `async-trait` の `(proc-macro)` は、「コンパイルのときにコードを書き足すマクロのクレート」という印です。`#[async_trait]` の印を付けた trait を、dyn で使える形に書き換えます（第9巻 UNIT 04）。

例1-5: "=1.53.3" と "1.53.3" の違いを確かめる

> **▼ このコマンドがやること（先に日本語で）:** `cargo update -p tokio --dry-run` は、「tokio を新しくしたら、どの版になるか」を、`Cargo.lock` を書き換えずに表示します（`--dry-run` は「試しに動かすだけ」）。ただし、今の RustNote の tokio（1.54.1）は、いちばん新しい版なので、このままでは違いが見えません。そこで、練習用のコピーで、tokio を1つ前の 1.53.3 に固定し直してから試します。確かめたら、`Cargo.toml` と `Cargo.lock` を教材のものに戻します。

```
# Cargo.toml の tokio の行を version = "=1.53.3" に変えてから、Cargo.lock も 1.53.3 にする
cargo update -p tokio --precise 1.53.3
```

```
    Updating crates.io index
 Downgrading tokio v1.54.1 -> v1.53.3
note: pass `--verbose` to see 1 unchanged dependencies behind latest
```

```
# =1.53.3 のまま
cargo update -p tokio --dry-run
```

```
    Updating crates.io index
     Locking 0 packages to highest Rust 1.99 compatible versions
note: pass `--verbose` to see 2 unchanged dependencies behind latest
warning: not updating lockfile due to dry run
```

`Cargo.toml` の tokio の行を、`version = "=1.53.3"` から `version = "1.53.3"`（`=` を消しただけ）に変えて、同じコマンドを実行します。

```
    Updating crates.io index
     Locking 1 package to highest Rust 1.99 compatible version
    Updating tokio v1.53.3 -> v1.54.1
note: pass `--verbose` to see 1 unchanged dependencies behind latest
warning: not updating lockfile due to dry run
```

- `=` があるときは、`Locking 0 packages`（変える版は 0 個）でした。`=1.53.3` の範囲には、1.53.3 しか無いからです。
- `=` を消すと、`Updating tokio v1.53.3 -> v1.54.1` と表示されました。`"1.53.3"` は「1.53.3 以上、2.0.0 未満」の意味なので、この範囲でいちばん新しい 1.54.1 を選べるのです。
- `to highest Rust 1.99 compatible version` は、「Rust 1.99 で使える版の中で、いちばん新しい版」という意味です。cargo は、`Cargo.toml` の `rust-version` を見て、それより新しい Rust を求めるクレートの版を、選ばないようにしています。

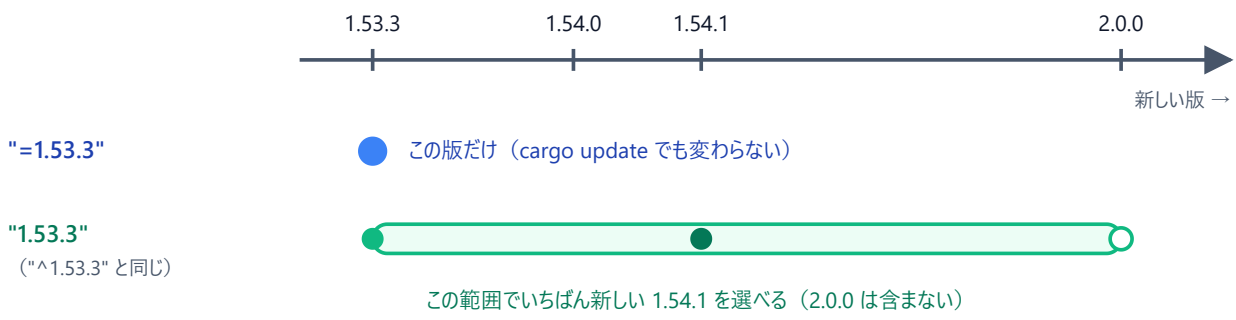


図13-5: 版の書き方と、選べる版の範囲。= を付けるとその版だけ、付けないと「上の桁が同じ範囲」の新しい版まで選べる

- `2 unchanged dependencies behind latest` は、「もっと新しい版が出ているのに、今の決まりでは選べないクレートが 2 個ある」という意味です。`=` を消すと 1 個に減りました。tokio が、新しい版を選べるようになったからです。`=` で固定すると、新しい版の改善も、自分で版を上げるまで入りません。固定は「同じ結果」の代わりに、「版を上げる作業を自分で決める」ことでもあります。

> **注意:** `--dry-run` を付け忘れると、`Cargo.lock` が本当に書き換わります。この例を試すときは、必ず `--dry-run` を付け、終わったら `Cargo.toml` の `=` を元に戻してください。

例1-6: features は、依存のつながり全体で足し算される

> **▼ このコマンドがやること（先に日本語で）：** `cargo tree -p tokio --depth 0 -f "{p} [{f}]"` は、tokio だけを表示し、`{f}` の所に、**実際に有効になっている機能**を並べます。

```
cargo tree --locked -p tokio --depth 0 -e normal -f "{p} [{f}]"
```

▼ 実行結果

```
tokio v1.54.1 [bytes,default,fs,full,io-std,io-
util,libc,macros,mio,net,parking_lot,process,rt,rt-multi-thread,signal,signal-hook-
registry,socket2,sync,time,tokio-macros,windows-sys]
```

- RustNote の `Cargo.toml` に書いた機能は `rt-multi-thread` と `macros` の2つだけなのに、`fs`・`net`・`signal`・`full` など、たくさんの機能が有効になっています。
- これは、RustNote が使うほかのクレートも、tokio の機能を求めているからです。`cargo tree -e features -i tokio` で調べると、libsql が使う `libsql_replication` が tokio の `full`（ほぼ全部の機能）を、`topcoat` が `macros`・`net`・`signal` を有効にしていることが分かります（それぞれのクレートの `Cargo.toml` にも、そう書いてあります）。cargo は、1つのビルドの中の tokio を1つにまとめ、**誰かが求めた機能を全部足したもの**でコンパイルします。
- そのため、試しに RustNote の `Cargo.toml` から `"macros"` を消しても、ビルドは通ってしまいます（`cargo build --locked` は `Finished` で終わりました）。ほかのクレートが `macros` を有効にしてくれているからです。
- それでも、RustNote の `Cargo.toml` には `macros` を書いておきます。libsql や topcoat が次の版で tokio の使い方を変えたら、RustNote の `#[tokio::main]` は突然コンパイルできなくなるからです。**自分が使う機能は、自分の `Cargo.toml` に書く**のが決まりです。だれも `macros` を有効にしていないと何が起きるかは、エラー5で確かめます。

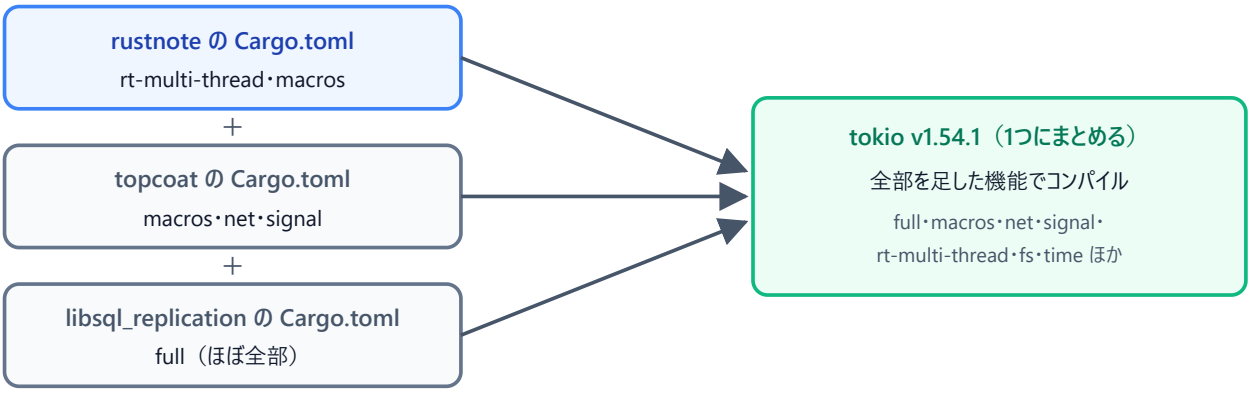


図13-6: features の足し算。1つのビルドの中の tokio は1つにまとめられ、誰かが求めた機能を全部足したものでコンパイルされる。だから自分が書き忘れた機能も、ほかのクレートのおかげで使えてしまうことがある

features とは別の話ですが、1つのビルドの中に、版の違う同じクレートが同時に入ることもあります。`Cargo.lock` の RustNote の項目を見てみましょう（抜き出しています）。

```
[[package]]
name = "rustnote"
version = "0.1.0"
dependencies = [
 "anyhow",
 "argon2",
 "async-trait",
 "dotenvy",
 "libsql",
 "serde",
 "tempfile",
 "thiserror 2.0.21",
 "tokio",
 "topcoat",
]
```

- `"thiserror 2.0.21"` だけ、名前の後ろに版が付いています。`Cargo.lock` に thiserror が 1.0.69 と 2.0.21 の2つあるので、どちらかを区別するためです。

```
cargo tree --locked -i thiserror@1.0.69 --edges normal
```

```
thiserror v1.0.69
├── libsql v0.10.0
│   └── rustnote v0.1.0 (...\rustnote)
└── libsql_replication v0.10.0
    └── libsql v0.10.0 (*)
```

- `-i`（invert：逆向き）は、「このクレートを使っているのは誰か」をたどります。thiserror 1.0.69 は libsql が使い、RustNote 自身は 2.0.21 を使っていました。1.x と 2.x のように、上の桁が違う版は「互換性のない別物」とみなされるので、1つのビルドの中に両方が入ってもかまわないのです。

例1-7: .gitignore と .env.example の全文

> **▼ このファイルがやること（先に日本語で）：** `.gitignore` は、Git に入れないファイルの名前の決まりを書きます。`.env.example` は、RustNote が読む環境変数の名前と、開発用の値の見本を書きます。秘密の値（Turso のトークン）は、空のままにしてあります。

```
# ビルドの成果物（cargo build / topcoat asset bundle が作る）
/target/

# ローカルの SQLite / libSQL データベース（WAL などの付属ファイルも含む）
*.db
*.db-shm
*.db-wal
*.db-journal

# 秘密の値を書いた環境変数ファイル（.env.example だけを共有する）
.env

# OS やエディタが作るファイル
.DS_Store
Thumbs.db
```

```
# RustNote の環境変数の見本。
# このファイルを .env という名前でコピーし、値を書き換えて使う（.env は Git に入れない）。

# サーバーが待ち受けるアドレスとポート（topcoat::start が読む）
# 開発: 127.0.0.1:3000 / Fly.io: 0.0.0.0:8080
HOST=127.0.0.1
PORT=3000

# データベース
# 開発: ローカルファイルのパス（無ければ自動で作られる）
DATABASE_URL=rustnote.db
# 本番（Turso）の例:
# DATABASE_URL=libsql://rustnote-<あなたの組織名>.turso.io
# DATABASE_AUTH_TOKEN=<turso db tokens create で作ったトークン>
DATABASE_AUTH_TOKEN=
```

- `.gitignore` の `*.db` は「名前が `.db` で終わるファイル全部」です。開発で作った DB には、自分で試しに登録したメールアドレスなどが入るので、Git に入れません。`*.db-shm`・`*.db-wal`・`*.db-journal` は、SQLite が DB の横に作る作業用のファイルです。
- `.env` は Git に入れず、`.env.example` だけを共有します。`.env.example` を `.env` という名前でコピーし、自分の値を書き込んで使います。RustNote の `main.rs` は、起動するときに `dotenvy::dotenv().ok()` で `.env` を読みます（`.env` が無ければ何もしません。UNIT 03）。
- `Cargo.lock` は `.gitignore` に**書いていません**。アプリ（実行ファイルを作るパッケージ）では、`Cargo.lock` を Git に入れて共有するのが、Cargo の公式の考え方です。共有しないと、PC ごとに違う版の部品でビルドすることになるからです。

よくあるエラーと読み方

この節のエラーは、どれも練習用のコピーの設定ファイルを1か所だけ書き換えて出したものです。確かめ終わったら、そのファイルを必ず元に戻してください。

エラー1: Rust の版が古い（rust-version）

`rust-toolchain.toml` が無いフォルダや、`cargo +stable build` のように版を指定して、Rust 1.97.1 でビルドしようとした場合です（この PC の既定の版 `stable` は 1.97.1 です）。

```
cargo +stable check --locked
```

（`requires rustc 1.99` の行が 35 行続くので、途中を省いています）

```
error: rustc 1.97.1 is not supported by the following packages:
  rustnote@0.1.0 requires rustc 1.99
  rustnote@0.1.0 requires rustc 1.99
  topcoat@0.11.0 requires rustc 1.99
  topcoat-asset@0.11.0 requires rustc 1.99
  …（途中を省略）…
  topcoat-view-grammar@0.11.0 requires rustc 1.99
  topcoat-view-macro@0.11.0 requires rustc 1.99
```

- **何が起きたか:**「rustc 1.97.1 は、次のパッケージに対応していません」というエラーです。`rustnote@0.1.0 requires rustc 1.99` は、RustNote の `Cargo.toml` の `rust-version = "1.99"`、`topcoat@0.11.0 requires rustc 1.99` は、Topcoat 側の同じ宣言です。同じ名前が何回か出ていますが、読むのは、パッケージの名前と `requires rustc 1.99` の部分だけで十分です。
- このエラーは、**コンパイルを始める前に**出ます。古い Rust で途中まで長いコンパイルをして、分かりにくいエラーで止まるより、ずっと親切です。`rust-version` を書いておく理由がこれです。

- **直し方**: RustNote のフォルダでは、`+stable` を付けずに `cargo check --locked` とします。`rust-toolchain.toml` があるので 1.99.1 が使われます。1.99.1 がまだ入っていない PC では、`rustup toolchain install 1.99.1 --profile minimal --component rustfmt,clippy` で入れます（既定の版は変わりません）。

> **注意**: `cargo +1.97.1 check` のように、PC に入っていない版の名前を指定すると、この PC の rustup は、その版を**自動でダウンロードして入れました**（`warn: the missing active toolchain ... has been auto-installed` と表示されます）。版を試すときは、`rustup toolchain list` で入っている版を確かめてから指定しましょう。入れてしまった版は、`rustup toolchain uninstall 版の名前` で消せます。

エラー2: 機能の名前を打ち間違えた

`Cargo.toml` の tokio の `"macros"` を、`"macro"`（`s` が無い）と打ち間違えた場合です。

```
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macro"] }
```

```
cargo build --locked
```

```
error: failed to select a version for `tokio`.
    ... required by package `rustnote v0.1.0 (...\rustnote)`
versions that meet the requirements `=1.54.1` (locked to 1.54.1) are: 1.54.1

package `rustnote` depends on `tokio` with feature `macro` but `tokio` does not have
that feature.
help: there is a feature `macros` with a similar name


failed to select a version for `tokio` which could resolve this conflict
```

- **何が起きたか**:「tokio の版を選べませんでした」というエラーです。その下の `package rustnote depends on tokio with feature macro but tokio does not have that feature.`（RustNote は tokio の `macro` という機能を求めていますが、tokio にその機能はありません）が本当の理由です。
- `versions that meet the requirements =1.54.1 (locked to 1.54.1) are: 1.54.1` は、「条件に合う版は 1.54.1 だけで、その版には `macro` という機能が無いので、選べる版がもう無い」という意味です。
- **直し方**: `help: there is a feature macros with a similar name`（似た名前の `macros` という機能があります）のとおり、`"macros"` に直します。機能の名前は、クレートの docs.rs のページの「Feature flags」で確かめられます。

エラー3: --locked なのに、Cargo.lock を書き換える必要がある

`Cargo.toml` の dotenvy の版を、`"=0.15.7"` から `"=0.15.6"` に変えて、`--locked` でビルドした場合です。

```
cargo build --locked
```

```
    Updating crates.io index
error: cannot update the lock file ...\rustnote\Cargo.lock because --locked was passed
to prevent this
help: to generate the lock file without accessing the network, remove the --locked flag
and use --offline instead.
```

- **何が起きたか:** 「`--locked` が指定されたので、Cargo.lock を書き換えられません」というエラーです。`Cargo.toml` は 0.15.6 を求めているのに、`Cargo.lock` には 0.15.7 が書いてあるので、ビルドするには `Cargo.lock` を書き直す必要がありました。
- **これはエラーで正しい:** CI や Docker のビルドで、`Cargo.lock` と違う版が黙って使われるのを止めるために、わざと `--locked` を付けています。
- **直し方:** 版を本当に変えたいときは、`--locked` を付けずに `cargo build`（または `cargo update -p dotenvy`）を実行して `Cargo.lock` を更新し、`Cargo.toml` と `Cargo.lock` を**一緒に**コミットします。この練習では、`Cargo.toml` を `"=0.15.7"` に戻します。
- `help:` の `--offline`（ネットにつながずに作る）の案内は、ネットを使わずに `Cargo.lock` を作り直したいときの方法です。今回は版を戻せばよいので、使いません。

エラー4: Cargo.lock の指紋（checksum）が違う

`Cargo.lock` の dotenvy の `checksum` の、先頭の1文字だけを書き換えた場合です。

```
[[package]]
name = "dotenvy"
version = "0.15.7"
source = "registry+https://github.com/rust-lang/crates.io-index"
checksum = "0aaf95b3e5c8f23aa320147307562d361db0ae0d5124234 0f558153b4eb2439b"
```

```
cargo build --locked
```

```
error: checksum for `dotenvy v0.15.7` changed between lock files

this could be indicative of a few possible errors:

    * the lock file is corrupt
    * a replacement source in use (e.g., a mirror) returned a different checksum
    * the source itself may be corrupt in one way or another

unable to verify that `dotenvy v0.15.7` is the same as when the lockfile was generated
```

- **何が起きたか**: 「dotenvy 0.15.7 の checksum が、ロックファイルの間で変わりました」というエラーです。`checksum` は、crates.io から取ってきたクレートの中身から計算した**指紋**（SHA-256）です。本物の指紋は `1aaf95b3…` で始まるのに、`Cargo.lock` には `0aaf95b3…` と書いてあったので、cargo は「同じ物か確かめられない」と止まりました。
- その下の3つの点は、考えられる原因です。ロックファイルが壊れている、ミラー（代わりの取得先）が違う中身を返した、取得先そのものが壊れている、のどれかです。
- **これはエラーで正しい**: 途中で中身をすり替えられたクレートを、気づかずにビルドしてしまう事故を防いでいます。`Cargo.lock` を Git で共有すると、この指紋も全員で共有されます。
- **直し方**: `Cargo.lock` は手で書き換えるファイルではありません（先頭に `It is not intended for manual editing.` と書いてあります）。教材のフォルダから `Cargo.lock` をコピーし直します。

エラー5: 誰も macros を有効にしていない（E0433・E0752）

例1-6で、RustNote では `"macros"` を消してもビルドが通ることを見ました。では、ほかのクレートが助けてくれない場合はどうなるでしょう。tokio だけを使う小さなプロジェクト（`feat_demo`）で試します。

```
[package]
name = "feat_demo"
version = "0.1.0"
edition = "2024"

[dependencies]
# RustNote と同じ版の tokio を、macros なしで使う
tokio = { version = "=1.54.1", features = ["rt-multi-thread"] }
```

```
// #[tokio::main] は、tokio の macros の機能が有効なときだけ使える
#[tokio::main]
async fn main() {
    println!("tokio のランタイムの中で動いています");
}
```

```
cargo build
```

```
   Compiling tokio v1.54.1
   Compiling feat_demo v0.1.0 (...\feat_demo)
error[E0433]: cannot find `main` in `tokio`
 --> src\main.rs:2:10
  |
2 | #[tokio::main]
  |          ^^^^ could not find `main` in `tokio`

error[E0752]: `main` function is not allowed to be `async`
 --> src\main.rs:3:1
  |
3 | async fn main() {
  | ^^^^^^^^^^^^^^^ `main` function is not allowed to be `async`

Some errors have detailed explanations: E0433, E0752.
For more information about an error, try `rustc --explain E0433`.
error: could not compile `feat_demo` (bin "feat_demo") due to 2 previous errors
```

- **何が起きたか:** 1つ目の `error[E0433]: cannot find main in tokio` は、「tokio の中に `main` が見つかりません」です。`#[tokio::main]` は、`macros` の機能を有効にしたときだけ tokio の中に現れるので、機能が無いと「無い物を使った」エラーになります。
- 2つ目の `error[E0752]: main function is not allowed to be async` は、「`main` 関数を `async` にはできません」です。本当は `#[tokio::main]` が `async fn main` を普通の `fn main` に書き換えてくれるはずでしたが、1つ目のエラーでそれが働かなかったので、続けてこのエラーが出ました。**最初のエラーから直す**と、2つ目は一緒に消えます。
- **直し方:** `Cargo.toml` の features に `"macros"` を足します。

```toml
[package]
name = "feat_demo"
version = "0.1.0"
edition = "2024"

[dependencies]
# macros を足すと、#[tokio::main] が使える
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros"] }
```

```
cargo run -q
```

```
tokio のランタイムの中で動いています
```

RustNote ではここで使う

この単元のファイルは、RustNote のほかのファイルから、次のように使われています。

使う場所	書いてあること	つながるファイル
`Dockerfile`	`FROM rust:1.99.1-bookworm AS builder`	`rust-toolchain.toml` と同じ 1.99.1
`Dockerfile`	`RUN cargo install topcoat-cli --version 0.11.0 --locked`	`Cargo.toml` の topcoat と同じ 0.11.0
`Dockerfile`	`RUN cargo build --release --locked`	`Cargo.lock`（`--locked`）
`ci.yml`	`toolchain: 1.99.1` と `components: rustfmt, clippy`	`rust-toolchain.toml`
`ci.yml`	`cargo clippy --all-targets --locked -- -D warnings`・`cargo test --locked`	`Cargo.lock`
`.dockerignore`	`target/`・`*.db`・`.env` を送らない	`.gitignore` と同じ考え方
`src/main.rs`	`dotenvy::dotenv().ok();`	`.env.example` をコピーした `.env`

- Rust の版（1.99.1）は、`rust-toolchain.toml`・`Dockerfile`・`ci.yml` の3か所に書いてあります。版を上げるときは、この3か所を**一緒に**変えます。1か所だけ変えると、「自分の PC では通るのに、CI では通らない」という食い違いが起きます。
- topcoat の版（0.11.0）も、`Cargo.toml` のクレートと、`Dockerfile` の CLI の2か所です。クレートと CLI は同じ Topcoat の一部なので、版を上げるときは両方を一緒に上げます。

練習問題

問1: `tempfile` は `[dev-dependencies]` に書いてあります。もし `src/lib.rs` の中で `tempfile::tempdir()` を使う関数を書いたら、どうなるでしょうか。予想してから、練習用のコピーで確かめてください（確かめたら元に戻します）。また、なぜ `tempfile` を `[dependencies]` ではなく `[dev-dependencies]` に書くのかを答えてください。

問2: ARM64 版の Windows で RustNote をビルドするには、LLVM（clang）が必要でした（第9巻の巻頭）。これは、暗号のクレート `ring` を C のコンパイラでビルドするためです。では、RustNote の `Cargo.toml` には `ring` が無いのに、なぜ `ring` がビルドされるのでしょうか。`cargo tree` で、`ring` を使っているクレートをたどってください。

問3: 練習用のコピーで `git init` し、`.env.example` を `.env` にコピーし、`rustnote.db` という名前の空でないファイルを作ります。そのうえで、次の2つを確かめてください。

1. `git check-ignore -v` で、`.env`・`rustnote.db`・`rustnote.db-wal`・`target/debug/rustnote.exe`・`.env.example`・`Cargo.lock` のうち、どれが Git から外されるか。外されるものは、`.gitignore` の何行目の決まりによるか。
2. `git status --short` に、`.env` と `rustnote.db` が出てこないこと。

解答と解説

問1の解答:

予想: コンパイルエラーになる。`[dev-dependencies]` のクレートは、ライブラリの本体（`src/`）からは見えないからです。`src/lib.rs` の最後に、次の関数を足して確かめます。

```
/// 試し: dev-dependencies のクレートを src/ で使う
pub fn scratch_dir() -> tempfile::TempDir {
    tempfile::tempdir().expect("一時フォルダを作れません")
}
```

```
cargo build --locked
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0433]: cannot find module or crate `tempfile` in this scope
  --> src\lib.rs:30:5
   |
30 |     tempfile::tempdir().expect("一時フォルダを作れません")
   |     ^^^^^^^^ use of unresolved module or unlinked crate `tempfile`
   |
   = help: if you wanted to use a crate named `tempfile`, use `cargo add tempfile` to
add it to your `Cargo.toml`

error[E0433]: cannot find module or crate `tempfile` in this scope
  --> src\lib.rs:29:25
   |
29 | pub fn scratch_dir() -> tempfile::TempDir {
   |                         ^^^^^^^^ use of unresolved module or unlinked crate
`tempfile`
   |
   = help: if you wanted to use a crate named `tempfile`, use `cargo add tempfile` to
add it to your `Cargo.toml`

For more information about this error, try `rustc --explain E0433`.
error: could not compile `rustnote` (lib) due to 2 previous errors
```

- `cannot find module or crate tempfile in this scope`（この場所では `tempfile` というモジュールもクレートも見つかりません）と、使った2か所でエラーになりました。`help:` は「`cargo add tempfile` で `Cargo.toml` に足してください」と案内していますが、テストのための道具を本体の依存にするのは、正しい直し方ではありません。
- `[dev-dependencies]` に書くのは、本番の実行ファイルに**入れる必要が無い**からです。入れないぶん、本番のビルドが速くなり、できあがる実行ファイルも小さくなり、本番で動くコードが使うクレートの数（気をつけて見ておくべき範囲）も減ります。
- 確かめたら、足した関数を消して、`src/lib.rs` を元に戻します。

問2の解答:

```
cargo tree --locked -i ring --edges normal
```

```
ring v0.17.14
├── rustls v0.22.4
│   ├── hyper-rustls v0.25.0
│   │   └── libsql v0.10.0
│   │       └── rustnote v0.1.0 (...\rustnote)
│   └── tokio-rustls v0.25.0
│       └── hyper-rustls v0.25.0 (*)
└── rustls-webpki v0.102.8
    └── rustls v0.22.4 (*)
```

- 下から読むと、`rustnote → libsql → hyper-rustls → rustls → ring` とつながっています。libsql が Turso（ネットの向こうの DB）と暗号化して通信するために rustls（TLS という暗号化の仕組みを Rust で書いたクレート）を使い、rustls と、証明書を確かめる rustls-webpki が、暗号の計算に ring を使っているのです。
- `(*)` は、「この先は、上ですでに表示した」という印です。同じつながりを何度も書かないように省いています。
- RustNote の `Cargo.toml` に書いていないクレートでも、ビルドに必要な道具（ここでは clang）を求めることがあります。ビルドが止まったら、エラーに出てきたクレートの名前で `cargo tree -i` を調べると、「誰のために必要なのか」が分かります。

問3の解答:

```
git init
Copy-Item .env.example .env
Set-Content rustnote.db "x"
git check-ignore -v .env rustnote.db rustnote.db-wal target/debug/rustnote.exe
.env.example Cargo.lock
```

```
.gitignore:11:.env   .env
.gitignore:5:*.db    rustnote.db
.gitignore:7:*.db-wal    rustnote.db-wal
.gitignore:2:/target/    target/debug/rustnote.exe
```

```
git status --short
```

```
?? .dockerignore
?? .env.example
?? .gitignore
?? Cargo.lock
?? Cargo.toml
?? Dockerfile
?? README.md
?? assets/
?? ci.yml
?? fly.toml
?? migrations/
?? rust-toolchain.toml
?? rustfmt.toml
?? src/
?? tests/
```

1. 外されるのは `.env`（11 行目の `.env`）、`rustnote.db`（5 行目の `*.db`）、`rustnote.db-wal`（7 行目の `*.db-wal`）、`target/debug/rustnote.exe`（2 行目の `/target/`）の4つです。`-v`（verbose）を付けると、「どのファイルの何行目の、どの決まりに当たったか」が、`.gitignore:11:.env` の形で表示されます。`.env.example` と `Cargo.lock` は表示されません。外されない（Git で共有する）からです。
2. `git status --short` の `??`（まだ Git に入れていないファイル）の一覧に、`.env` と `rustnote.db` はありません。フォルダの中には確かにあるのに、Git からは見えていないのです。`.gitignore` の決まりの、いちばんの目的は、この「うっかり共有しない」ことです。

- `git check-ignore` は、ファイルが本当にあるかどうかに関係なく、名前だけで調べます。`target/debug/rustnote.exe` は、このフォルダにはまだありません（ビルドの成果物は別の場所に作っています）が、名前の決まりで判定されました。
- `/target/` の先頭の `/` は、「`.gitignore` のあるフォルダの直下の `target` だけ」という意味です。`src/target/` のような、下のほうのフォルダの `target` は外しません。

まとめ

- `Cargo.toml` は人が書く材料表、`Cargo.lock` は cargo が書くレシート（全クレートの版と指紋）。RustNote は直接使うクレートを `=` で固定し、`Cargo.lock` を Git で共有して、`--locked` で書き換えを止める。

- Rust の版は、`rust-version`（最低の版）と `rust-toolchain.toml`（このフォルダで使う版）で決め、`Dockerfile` と `ci.yml` にも同じ版を書く。features は依存のつながり全体で足し算されるので、自分が使う機能は自分の `Cargo.toml` に書く。
- 秘密の値と DB と成果物は `.gitignore` で共有から外し、環境変数は `.env.example` という見本だけを共有する。

UNIT 02: 表を作る SQL と DB の入口

この単元のゴール

- `migrations/` の3つの SQL ファイルが、どの表・索引・制約を作るかを説明できる
- `src/db.rs` の `migrate` が、まだ適用していない版だけを、1つずつトランザクションの中で適用し、`schema_migrations` に記録する流れを説明できる
- `Db::open_local` と `Db::connect` が入れている設定（WAL・外部キー・待ち時間）を、実際の値で確かめられる
- マイグレーションが途中で失敗したときに何が残るか、適用済みの SQL ファイルを書き換えたら何が起きるかを、実験で説明できる

たとえ話でつかむ

ある建物に、何年もかけて部屋を増やしていく場面を考えます。

- 工事には**番号**が付いています。「1番: 1階を建てる」「2番: 2階を足す」「3番: 倉庫を足す」。工事は、必ず番号の順に、1回ずつ行います。
- 建物には**工事の記録簿**があり、終わった工事の番号が書いてあります。新しい担当者が来ても、記録簿を見れば「3番まで終わっている。次は4番から」と分かります。
- 工事の途中で問題が起きたら、その工事で作りかけた物は**全部取り壊して**、工事の前の状態に戻します。半分だけできた部屋を残すと、次の工事ができなくなるからです。
- 終わった工事の**図面を書き換えても、建物は変わりません。**2番の図面に「窓を1つ増やす」と書き足しても、すでに建った2階の窓は増えません。窓を増やしたいなら、新しく「4番: 2階に窓を足す」という工事を足します。

RustNote の DB も、同じやり方で作ります。

建物の工事	RustNote
番号の付いた工事	**マイグレーション**（`migrations/0001_users.sql` など、番号順の SQL ファイル）
工事の記録簿	`schema_migrations` 表（適用済みの版の名前と時刻）
番号の順に、まだの工事だけをする	`db::migrate` が、未適用の版だけを古い順に適用する
途中で失敗したら、作りかけを取り壊す	1つの版を**トランザクション**の中で適用し、失敗したら `rollback`
終わった工事の図面を書き換えても変わらない	適用済みの SQL ファイルを書き換えても、今ある DB には反映されない（エラー2）

> **用語:** マイグレーション（migration：DB の表の作り（スキーマ）を変える手順を、番号順のファイルに分けて書いたもの。第7巻 UNIT 06）

> **用語:** トランザクション（transaction：いくつかの変更を「全部やる」か「全部やめる」かの1まとまりにする仕組み。第7巻 UNIT 05）

図で理解する

3つのマイグレーションと、それが作る物の対応です。

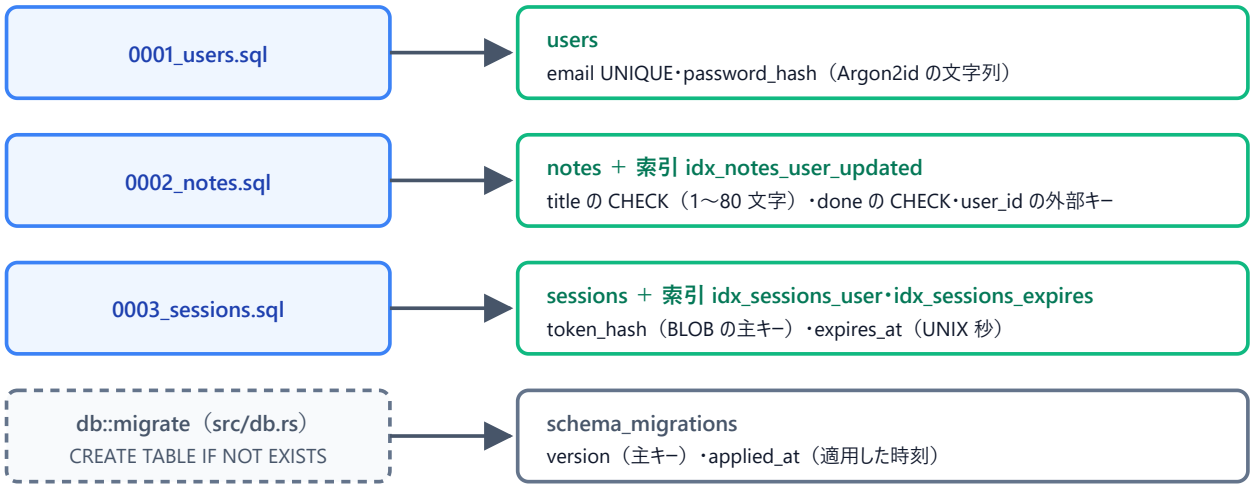


図13-7: マイグレーションのファイルと、それが作る表と索引。記録簿の schema_migrations だけは、ファイルではなく db::migrate のコードが作る

`db::migrate` は、次の順番で動きます。

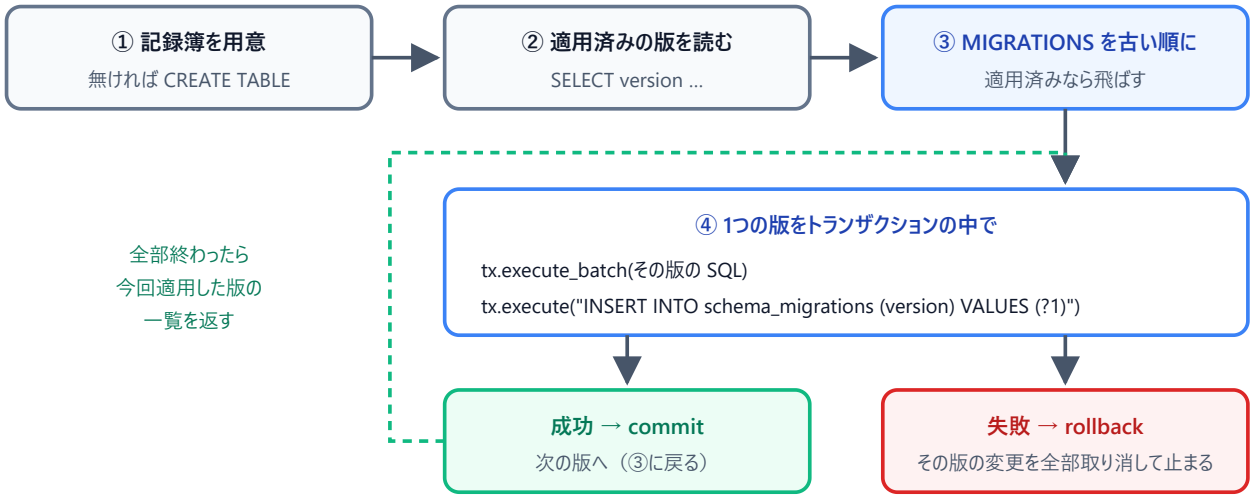


図13-8: db::migrate の流れ。適用済みの版は飛ばし、まだの版だけを1つずつトランザクションの中で適用する。SQL の実行と記録簿への書き込みは、同じトランザクションに入れる

- 大事なのは、④の中で、**SQL の実行と記録簿への書き込みを、同じトランザクションに入れている**ことです。もし SQL だけが成功して、記録が残らなかったら、次の起動で同じ SQL がもう一度流れ、「表がもうあります」のエラーになります。反対に、記録だけが残って SQL が失敗していたら、表が無いのに「適用済み」になります。同じトランザクションなら、両方が成功するか、両方とも無かったことになるかの、どちらかです。

まず動かす最小コード

例2-1: マイグレーションの SQL とコードの全文

> **▼ このファイルがやること（先に日本語で）：** `0001_users.sql` はユーザーの表、`0002_notes.sql` はノートの表と一覧を速くする索引、`0003_sessions.sql` はログイン中のセッションの表と2つの索引を作ります。どれも、決まり（`UNIQUE`・`CHECK`・外部キー）を表に書き込んでいます（第9巻 UNIT 03）。

```
-- 0001: ユーザーのテーブル
-- email は UNIQUE（同じメールアドレスで二重に登録できない）。
-- password_hash には Argon2id の PHC 文字列（$argon2id$v=19$...）を入れる。生のパスワードは保存しない。
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
```

```sql
-- 0002: ノートのテーブル
-- title は 1〜80 文字（length() は文字数を数える。Rust 側の chars().count() と同じ）。
-- done は 0（未完了）か 1（完了）。
-- version は楽観的ロック用の番号。更新のたびに 1 増える。
-- 日時は UTC の 'YYYY-MM-DD HH:MM:SS' 形式の文字列（datetime('now') の形）。
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 80),
    body TEXT NOT NULL DEFAULT '',
    done INTEGER NOT NULL DEFAULT 0 CHECK (done IN (0, 1)),
    version INTEGER NOT NULL DEFAULT 1,
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now'))
);

-- 「自分のノートを新しい順に」を速くするための索引
CREATE INDEX idx_notes_user_updated ON notes (user_id, updated_at DESC, id DESC);
```

```sql
-- 0003: ログイン中のセッションのテーブル
-- token_hash は Topcoat が発行したトークンの SHA-256（32 バイト）。生のトークンは保存しない。
-- expires_at は有効期限。1970-01-01 からの秒数（UNIX 時刻）で持ち、比較しやすくする。
CREATE TABLE sessions (
    token_hash BLOB PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    expires_at INTEGER NOT NULL,
    created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE INDEX idx_sessions_user ON sessions (user_id);
CREATE INDEX idx_sessions_expires ON sessions (expires_at);
```

▼ **このファイルがやること（先に日本語で）**： `src/db.rs` は、DB への入口の `Db` 型と、マイグレーションの `migrate` 関数を持ちます。`Db` は、設定に合わせてローカルのファイルか Turso を開き、接続のたびに必要な設定を入れます。`migrate` は、SQL ファイルを `include_str!` で実行ファイルに埋め込み、まだ適用していない版だけを適用します。

```rust
//! DB（Turso / libSQL）への接続と、マイグレーション（テーブルの作成・変更）。
//!
//! - 開発ではローカルファイル（例: rustnote.db）を使う
//! - 本番では Turso の URL（libsql://...）と認証トークンを環境変数で渡す
//!
//! 接続（Connection）は処理のたびに `Db::connect` で作る。
//! ローカルファイルなら開くだけ、リモートなら HTTP の準備だけなので軽い。

use std::path::Path;
use std::sync::Arc;

use libsql::{Builder, Connection, Database, Transaction};
use thiserror::Error;

use crate::config::Config;

/// DB への入口。中身は Arc なので、clone しても同じ DB を指す（安く複製できる）
#[derive(Clone)]
pub struct Db {
    database: Arc<Database>,
    is_local: bool,
}

impl Db {
    /// 設定に従って、ローカルファイルかリモート（Turso）の DB を開く
    pub async fn open(config: &Config) -> Result<Db, libsql::Error> {
        if config.is_remote() {
            // 秘密のトークンは、接続するこの場所でだけ取り出す
            let token = config
                .database_auth_token
                .as_ref()
                .map(|secret| secret.expose().to_string())
                .unwrap_or_default();
            let database = Builder::new_remote(config.database_url.clone(), token)
                .build()
                .await?;
            Ok(Db {
                database: Arc::new(database),
                is_local: false,
            })
        } else {
            Db::open_local(&config.database_url).await
        }
    }

    /// ローカルファイルの DB を開く（無ければ作られる）
    pub async fn open_local(path: impl AsRef<Path>) -> Result<Db, libsql::Error> {
        let database = Builder::new_local(path).build().await?;
        let db = Db {
```

```
            database: Arc::new(database),
            is_local: true,
        };
        // WAL モード: 読み込みと書き込みが同時に来ても待たされにくくなる（ファイルに記録される）
        let conn = db.connect().await?;
        conn.execute_batch("PRAGMA journal_mode = WAL;").await?;
        Ok(db)
    }

    /// 新しい接続を作る
    pub async fn connect(&self) -> Result<Connection, libsql::Error> {
        let conn = self.database.connect()?;
        if self.is_local {
            // SQLite は接続ごとに設定が必要:
            // foreign_keys … REFERENCES（外部キー）を守らせる
            // busy_timeout … ほかの接続が書き込み中なら最大 5 秒待つ
            conn.execute_batch("PRAGMA foreign_keys = ON; PRAGMA busy_timeout = 5000;")
                .await?;
        }
        Ok(conn)
    }

    /// DB に問い合わせができるか確かめる（GET /health で使う）
    pub async fn ping(&self) -> Result<(), libsql::Error> {
        let conn = self.connect().await?;
        let mut rows = conn.query("SELECT 1", ()).await?;
        rows.next().await?;
        Ok(())
    }
}

// ---------------------------------------------------------------------------
// マイグレーション
// ---------------------------------------------------------------------------

/// 1つのマイグレーション（版の名前と SQL）
pub struct Migration {
    pub version: &'static str,
    pub sql: &'static str,
}

/// アプリが持つマイグレーションの一覧（古い順）。
/// SQL ファイルは include_str! でコンパイル時に実行ファイルへ埋め込む。
pub const MIGRATIONS: &[Migration] = &[
    Migration {
        version: "0001_users",
        sql: include_str!("../migrations/0001_users.sql"),
    },
    Migration {
        version: "0002_notes",
```

```
        sql: include_str!("../migrations/0002_notes.sql"),
    },
    Migration {
        version: "0003_sessions",
        sql: include_str!("../migrations/0003_sessions.sql"),
    },
];

#[derive(Debug, Error)]
pub enum MigrateError {
    #[error("schema_migrations を準備できません")]
    Prepare(#[source] libsql::Error),
    #[error("マイグレーション {version} に失敗しました")]
    Apply {
        version: &'static str,
        #[source]
        source: libsql::Error,
    },
}

/// まだ適用していないマイグレーションだけを、1つずつトランザクションの中で適用する。
/// 戻り値は、今回新しく適用した版の名前の一覧。
pub async fn migrate(db: &Db) -> Result<Vec<&'static str>, MigrateError> {
    let conn = db.connect().await.map_err(MigrateError::Prepare)?;

    // 適用済みの版を記録する表（無ければ作る）
    conn.execute(
        "CREATE TABLE IF NOT EXISTS schema_migrations (
            version TEXT PRIMARY KEY,
            applied_at TEXT NOT NULL DEFAULT (datetime('now'))
        )",
        (),
    )
    .await
    .map_err(MigrateError::Prepare)?;

    let applied = applied_versions(&conn)
        .await
        .map_err(MigrateError::Prepare)?;

    let mut newly_applied = Vec::new();
    for migration in MIGRATIONS {
        if applied.iter().any(|v| v == migration.version) {
            continue;
        }
        let apply_error = |source| MigrateError::Apply {
            version: migration.version,
            source,
        };
        let tx = conn.transaction().await.map_err(apply_error)?;
```

```
        match apply(&tx, migration).await {
            // 全部成功したときだけ確定する
            Ok(()) => tx.commit().await.map_err(apply_error)?,
            // 途中で失敗したら、その版の変更をすべて取り消す
            Err(source) => {
                tx.rollback().await.map_err(apply_error)?;
                return Err(apply_error(source));
            }
        }
        newly_applied.push(migration.version);
    }
    Ok(newly_applied)
}

/// schema_migrations に記録された版の名前を読む
async fn applied_versions(conn: &Connection) -> Result<Vec<String>, libsql::Error> {
    let mut rows = conn
        .query("SELECT version FROM schema_migrations ORDER BY version", ())
        .await?;
    let mut versions = Vec::new();
    while let Some(row) = rows.next().await? {
        versions.push(row.get::<String>(0)?);
    }
    Ok(versions)
}

/// 1つのマイグレーションの SQL を実行し、適用済みとして記録する
async fn apply(tx: &Transaction, migration: &Migration) -> Result<(), libsql::Error> {
    tx.execute_batch(migration.sql).await?;
    tx.execute(
        "INSERT INTO schema_migrations (version) VALUES (?1)",
        [migration.version],
    )
    .await?;
    Ok(())
}
```

▼ コードを1つずつ分解して解説

解説1: SQL のファイルは、実行ファイルの中に入る

- `MIGRATIONS` の `include_str!("../migrations/0001_users.sql")` は、コンパイルのときに、SQL ファイルの中身を文字列として実行ファイルに埋め込みます。公開した実行ファイルの隣に `migrations/` フォルダが無くても、マイグレーションを流せます（第9巻 UNIT 03 のエラー2）。
- `MIGRATIONS` の並び順が、適用の順番です。ファイル名の番号（0001・0002・0003）は、人が見て分かるように付けているもので、順番を決めているのはこの配列です。

解説2: Db は、Arc で包んだ入口

- `Db` は、libsql の `Database` を `Arc` で包んでいます。`clone()` しても、中の DB は1つのままで、参照の数が増えるだけです（第3巻）。RustNote の `AppState` は、この `Db` を、3つのリポジトリに1つずつ `clone()` して渡しています。
- 接続（`Connection`）は、処理のたびに `connect()` で作ります。先頭のコメントのとおり、ローカルのファイルなら開くだけ、Turso なら HTTP の準備だけなので軽い、という判断です。

解説3: 失敗の理由を2種類に分ける

- `MigrateError` は、「記録簿を準備できなかった」（`Prepare`）と、「この版の適用に失敗した」（`Apply`、失敗した版の名前つき）の2種類です。`#[source]` を付けたので、もとの libsql のエラーも、原因としてつながったまま残ります（第4巻 UNIT 05）。エラー1で、実際の表示を見ます。

例2-2: 同じ DB に、migrate を2回かける

> **▼ このコードがやること（先に日本語で）**：引数で渡した場所に DB ファイルを開き（無ければ作られる）、`db::migrate` を2回続けて呼んで、それぞれが返した「今回適用した版」を表示します。そのあと、記録簿の中身、できた表と索引、`notes` の列の名前を表示します。

```rust
//! RustNote の db::migrate を同じ DB に2回かけて、何が作られ、2回目に何が起きるかを確かめる。
//! DB ファイルの場所を引数で渡す（無ければ作られる）。
//! 実行: cargo run -q --example u02_migrate -- try.db

use libsql::Connection;
use rustnote::db::{self, Db};

/// 1列目の文字列を、行の数だけ集める
async fn strings(conn: &Connection, sql: &str) -> anyhow::Result<Vec<String>> {
    let mut rows = conn.query(sql, ()).await?;
    let mut values = Vec::new();
    while let Some(row) = rows.next().await? {
        values.push(row.get::<String>(0)?);
    }
    Ok(values)
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let path = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "try.db".to_string());
    let db = Db::open_local(&path).await?;

    println!("1回目の migrate: {:?}", db::migrate(&db).await?);
    println!("2回目の migrate: {:?}", db::migrate(&db).await?);

    let conn = db.connect().await?;
    println!(
        "適用済みの版: {:?}",
        strings(
            &conn,
            "SELECT version FROM schema_migrations ORDER BY version"
        )
        .await?
    );
    println!(
        "表: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name"
        )
        .await?
    );
    println!(
        "索引: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'index' AND name LIKE 'idx_%'
```

```
ORDER BY name"
        )
        .await?
    );
    println!(
        "notes の列: {:?}",
        strings(&conn, "SELECT name FROM pragma_table_info('notes')").await?
    );
    Ok(())
}
```

```
cargo run -q --example u02_migrate -- try.db
```

▼ 実行結果

```
1回目の migrate: ["0001_users", "0002_notes", "0003_sessions"]
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at"]
```

- 1回目は3つの版がすべて適用され、2回目は `[]`（何も適用しない）でした。RustNote は起動のたびに `migrate` を呼びますが、2回目からは何もしません。何回呼んでも同じ結果になる（**冪等**な）作りです。
- 表には、マイグレーションの4つのほかに `sqlite_sequence` があります。`AUTOINCREMENT` を付けた列の「次に使う番号」を、SQLite が自動で記録する表です。自分で作る必要も、触る必要もありません。

> **用語:** 冪等（べきとう：同じ操作を何回くり返しても、1回したときと同じ結果になること）

もう一歩: 実務に近い形

例2-3: 接続ごとの設定を、libsql の素の接続と比べる

> **▼ このコードがやること（先に日本語で）:** 一時フォルダの DB を `Db::open_local` で開き、`Db::connect` の接続で、3つの設定（`journal_mode`・`foreign_keys`・`busy_timeout`）の値を表示します。同じファイルを、RustNote の `Db` を通さずに、libsql の素の接続でも開いて比べます。最後に、フォルダの中のファイルを表示します。

```rust
//! Db::connect が接続ごとに入れている設定（PRAGMA）を、libsql の素の接続と比べる。
//! 実行: cargo run -q --example u02_pragmas

use libsql::{Builder, Connection, Value};
use rustnote::db::Db;

/// PRAGMA の値を1つ読む（数のときも文字のときもあるので、両方を文字にする）
async fn pragma(conn: &Connection, name: &str) -> anyhow::Result<String> {
    let mut rows = conn.query(&format!("PRAGMA {name}"), ()).await?;
    let row = rows.next().await?.expect("PRAGMA は1行を返すはずです");
    Ok(match row.get_value(0)? {
        Value::Integer(number) => number.to_string(),
        Value::Text(text) => text,
        other => format!("{other:?}"),
    })
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let path = dir.path().join("u02.db");

    // RustNote の Db を通した接続
    let db = Db::open_local(&path).await?;
    let conn = db.connect().await?;
    println!("[Db::connect の接続]");
    for name in ["journal_mode", "foreign_keys", "busy_timeout"] {
        println!("    {name} = {}", pragma(&conn, name).await?);
    }

    // libsql の素の接続（RustNote の設定を通さない）
    let raw = Builder::new_local(&path).build().await?.connect()?;
    println!("[libsql の素の接続]");
    for name in ["journal_mode", "foreign_keys", "busy_timeout"] {
        println!("    {name} = {}", pragma(&raw, name).await?);
    }

    // WAL モードにすると、DB の横に作業用のファイルができる
    let mut files: Vec<String> = std::fs::read_dir(dir.path())?
        .filter_map(|entry| entry.ok())
        .map(|entry| entry.file_name().to_string_lossy().to_string())
        .collect();
    files.sort();
    println!("フォルダの中のファイル: {files:?}");
    Ok(())
}
```

```
cargo run -q --example u02_pragmas
```

▼ 実行結果

```
[Db::connect の接続]
    journal_mode = wal
    foreign_keys = 1
    busy_timeout = 5000
[libsql の素の接続]
    journal_mode = wal
    foreign_keys = 1
    busy_timeout = 0
フォルダの中のファイル: ["u02.db", "u02.db-shm", "u02.db-wal"]
```

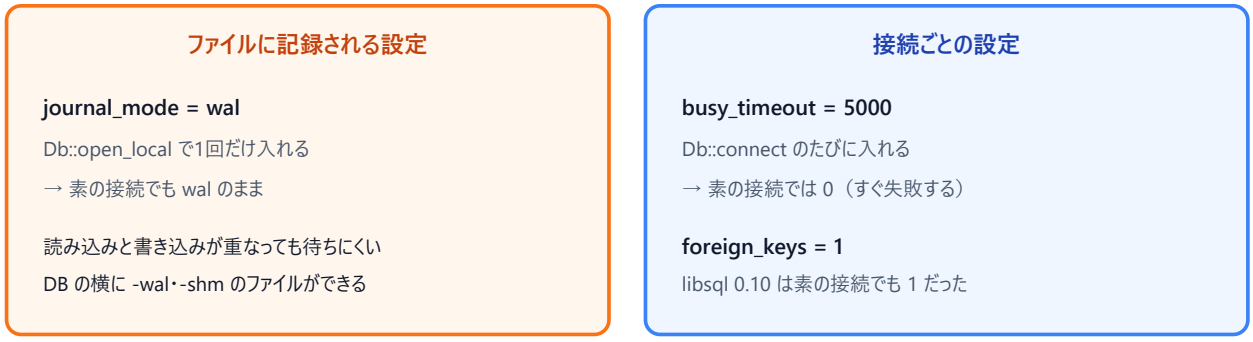


図13-9: DB の設定の2種類。WAL はファイルに記録されるので1回でよく、待ち時間と外部キーは接続ごとに入れる。RustNote は、外部キーも念のため毎回入れている

- **WAL（journal_mode = wal）**：ファイルに記録される設定です。`Db::open_local` で1回入れたので、素の接続でも `wal` のままでした。WAL にすると、DB のファイル（`u02.db`）の横に、`u02.db-wal`（まだ本体に書き込んでいない変更）と `u02.db-shm`（その目次）ができます。RustNote の `.gitignore` が `*.db-wal`・`*.db-shm` を外しているのは、このためです（UNIT 01）。
- **busy_timeout:** 接続ごとの設定です。`Db::connect` の接続は 5000（ミリ秒）、素の接続は 0 でした。0 のままだと、ほかの接続が書き込み中のとき、待たずにすぐ「DB がふさがっている」エラーになります。RustNote は、最大 5 秒は待つようにしています。
- **foreign_keys:** SQLite では、外部キーの確かめは「接続ごとに ON にしないと働かない」決まりです。ところが、この libsql 0.10 では、素の接続でも最初から `1`（ON）でした。RustNote の `Db::connect` は、それでも毎回 `PRAGMA foreign_keys = ON` を入れています。libsql の既定が変わっても外部キーが働くように、そして、コードを読んだ人が「外部キーを守らせている」と分かるように書いてある、と読めます。

よくあるエラーと読み方

この節の実験では、練習用のコピーの `migrations/` のファイルを書き換えます。確かめ終わったら、教材のフォルダから、そのファイルをコピーし直して元に戻してください。

エラー1: マイグレーションが途中で失敗した

`0003_sessions.sql` の最後の行の列の名前を、`expires_at` から `expire_at`（`s` が無い）に打ち間違えて、新しい DB に `migrate` をかけた場合です。

```
CREATE INDEX idx_sessions_expires ON sessions (expire_at);
```

```
cargo run -q --example u02_migrate -- broken.db
```

```
Error: マイグレーション 0003_sessions に失敗しました

Caused by:
    SQLite failure: `no such column: expire_at`
```

- **何が起きたか:** `Error:` の行は、`MigrateError::Apply` の文で、どの版で失敗したか（`0003_sessions`）を教えています。`Caused by:` の下が、もとの libsql のエラーで、「`expire_at` という列がありません」です。
- **何が残ったか:** 失敗したあとの DB の中を、`migrate` をかけずに表示するプログラム（`examples/u02_state.rs`）で調べます。

```rust
//! DB の今の状態（適用済みの版・表・索引）を、migrate をかけずに表示する。
//! 実行: cargo run -q --example u02_state -- try.db

use libsql::Connection;
use rustnote::db::Db;

/// 1列目の文字列を、行の数だけ集める（表が無ければ空にする）
async fn strings(conn: &Connection, sql: &str) -> Vec<String> {
    let Ok(mut rows) = conn.query(sql, ()).await else {
        return Vec::new();
    };
    let mut values = Vec::new();
    while let Ok(Some(row)) = rows.next().await {
        if let Ok(value) = row.get::<String>(0) {
            values.push(value);
        }
    }
    values
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let path = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "try.db".to_string());
    let db = Db::open_local(&path).await?;
    let conn = db.connect().await?;
    println!(
        "適用済みの版: {:?}",
        strings(
            &conn,
            "SELECT version FROM schema_migrations ORDER BY version"
        )
        .await
    );
    println!(
        "表: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name"
        )
        .await
    );
    println!(
        "索引: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'index' AND name LIKE 'idx_%'
ORDER BY name"
```

```
        )
        .await
    );
    Ok(())
}
```

```
cargo run -q --example u02_state -- broken.db
```

```
適用済みの版: ["0001_users", "0002_notes"]
表: ["notes", "schema_migrations", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated"]
```

- 0001 と 0002 は記録されていますが、0003 は記録されていません。そして、`sessions` の表も、その前の行で作ったはずの索引 `idx_sessions_user` も、**ありません**。0003 の SQL は、`sessions` の表と1つ目の索引を作ったところで失敗しましたが、トランザクションの `rollback` で、作りかけの物が全部取り消されたのです。
- **直し方**: SQL ファイルの打ち間違いを直してから、同じ DB にもう一度 `migrate` をかけます。0001 と 0002 は記録簿にあるので飛ばされ、0003 だけが適用されました。

```
1回目の migrate: ["0003_sessions"]
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at"]
```

エラー2（エラーが出ない間違い）: 適用済みの SQL ファイルを書き換えた

ノートに「ピン留め」の列 `pinned` を足したくなって、**適用済み**の `0002_notes.sql` の `CREATE TABLE notes` に、1行を書き足した場合です。

```
    body TEXT NOT NULL DEFAULT '',
    pinned INTEGER NOT NULL DEFAULT 0,
```

例2-2で作った、3つの版が適用済みの `try.db` に、`migrate` をかけます。

```
cargo run -q --example u02_migrate -- try.db
```

```
1回目の migrate: []
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at"]
```

- **何が起きたか:** 1回目も2回目も `[]` で、`notes` の列に `pinned` はありません。エラーも出ません。`migrate` は、版の**名前**（`0002_notes`）が記録簿にあるかどうかだけを見るので、中身が変わっても、もう一度は流さないのです。
- **なぜ困るのか:** 新しく作った DB（たとえば、別の人の PC や、テストの DB）では、書き換えた 0002 が流れるので、`pinned` があります。

```
cargo run -q --example u02_migrate -- fresh.db
```

```
1回目の migrate: ["0001_users", "0002_notes", "0003_sessions"]
notes の列: ["id", "user_id", "title", "body", "pinned", "done", "version",
"created_at", "updated_at"]
```

- 同じ RustNote なのに、**古い DB には `pinned` が無く、新しい DB にはある**という食い違いが、黙って生まれました。テストは新しい DB で動くので通り、本番の古い DB で初めて「列がありません」のエラーになります。
- **直し方:** 適用済みのファイルは、書き換えません。`0002_notes.sql` を元に戻し、変更は**新しい版**（`0004_…`）として足します（問1）。

RustNote ではここで使う

起動の入口の `src/main.rs` は、DB を開いて、マイグレーションをかけてから、待ち受けを始めます（抜き出しています）。

```
    let db = Db::open(&config)
        .await
        .with_context(|| format!("DB（{}）を開けませんでした", config.database_url))?;
    let applied = db::migrate(&db)
        .await
        .context("マイグレーションに失敗しました")?;
    for version in &applied {
        eprintln!("マイグレーションを適用しました: {version}");
    }
```

- `migrate` が返した「今回適用した版」を、1つずつ表示しています。第12巻 UNIT 01 で、新しい DB で起動したときに `マイグレーションを適用しました: 0001_users` などが3行出たのは、ここです。2回目からは、何も表示されません。
- マイグレーションに失敗したら、`?` で起動をやめます。表がそろっていない DB のまま、リクエストを受け付けることはありません。

テストの道具 `tests/common/mod.rs` も、同じ関数を使っています（抜き出しています）。

```
/// マイグレーション済みの空の DB を作る
pub async fn test_db() -> TestDb {
    let dir = tempfile::tempdir().expect("一時フォルダを作れません");
    let db = Db::open_local(dir.path().join("test.db"))
        .await
        .expect("DB を開けません");
    db::migrate(&db).await.expect("マイグレーションに失敗");
    TestDb { db, _dir: dir }
}
```

- テストごとに、一時フォルダに新しい DB を作り、本番と**同じ** `migrate` をかけます。テスト用に別の `CREATE TABLE` を書かないので、「テストの表と本番の表が違う」ことが起きません。

> **注意:** `0002_notes.sql` のコメントには「`length()` は文字数を数える。Rust 側の `chars().count()` と同じ」とあります。ふつうの文字ではそのとおりですが、第11巻 UNIT 04 の property test で、**NUL**（番号 0 の文字）だけは違うことが分かりました。SQLite の `length()` は NUL より前の文字しか数えないので、NUL で始まる題は、Rust の `NoteTitle::parse` を通っても、この `CHECK` で断られて 500 になります。教材のコードは、まだ直していません（改善点。直し方は、第11巻 UNIT 04 の差分と、巻末の総合演習で扱います）。

練習問題

問1: エラー2の「ピン留め」の列を、正しいやり方で足してください。`0002_notes.sql` は元に戻したうえで、新しい版 `migrations/0004_note_pinned.sql` を作り、`src/db.rs` の `MIGRATIONS` に足します（練習用のコピーの `src/` を書き換える練習です。確かめたら元に戻します）。3つの版が適用済みの `try.db` に `migrate` をかけたら、何が適用されるでしょうか。

問2: `0002_notes.sql` の索引 `idx_notes_user_updated` は、「自分のノートを新しい順に」を速くするためのものです。RustNote の一覧の SQL（`src/repository/notes.rs`）が、本当にこの索引を使っているかを、`EXPLAIN QUERY PLAN` で確かめてください。持ち主の条件が無い SQL とも比べます。

問3: `.env` の `DATABASE_URL` に、まだ無いフォルダの中のファイル（たとえば `data/rustnote.db`）を書いたら、RustNote はどうなるでしょうか。`Db::open_local` で確かめてください。

解答と解説

問1の解答:

新しいファイル `migrations/0004_note_pinned.sql` です。

```
-- 0004: ノートを一覧の上に固定する印（ピン留め）
-- 0（固定しない）か 1（固定する）。今あるノートは 0 になる。
ALTER TABLE notes ADD COLUMN pinned INTEGER NOT NULL DEFAULT 0 CHECK (pinned IN (0,
1));
```

`src/db.rs` の `MIGRATIONS` の最後に、1つ足します。

```
    Migration {
        version: "0004_note_pinned",
        sql: include_str!("../migrations/0004_note_pinned.sql"),
    },
```

```
cargo run -q --example u02_migrate -- try.db
```

```
1回目の migrate: ["0004_note_pinned"]
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions", "0004_note_pinned"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at", "pinned"]
```

- 1回目の `migrate` で、`0004_note_pinned` だけが適用されました。0001〜0003 は記録簿にあるので飛ばされます。`notes` の列の最後に `pinned` が加わりました。`ALTER TABLE ... ADD COLUMN` は、列を最後に足すからです。
- 今あるノートには、`DEFAULT 0` の値が入ります。`NOT NULL` の列を足すときは、`DEFAULT` が無いと、今ある行に入れる値が無いので失敗します。
- この状態で `cargo test` を実行すると、41 件のテストがすべて通りました。新しい DB では 0001〜0004 がすべて流れ、古い DB では 0004 だけが流れて、どちらも同じ形になります。確かめたら、`src/db.rs` を元に戻し、`0004_note_pinned.sql` を消します。

問2の解答:

`examples/u02_q2.rs` です。

```rust
//! 問2の解答: 一覧の SQL が、索引 idx_notes_user_updated を使っているかを EXPLAIN QUERY PLAN で確かめる。
//! 実行: cargo run -q --example u02_q2

use rustnote::db::{self, Db};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u02.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;

    // src/repository/notes.rs の一覧の SQL と同じ形（列は短くしている）
    let list_sql = r"SELECT id, title FROM notes
        WHERE user_id = ?1 AND (title LIKE ?2 ESCAPE '\' OR body LIKE ?2 ESCAPE '\')
        ORDER BY updated_at DESC, id DESC
        LIMIT ?3 OFFSET ?4";
    for (label, sql) in [
        ("RustNote の一覧", list_sql.to_string()),
        (
            "持ち主の条件が無い一覧",
            "SELECT id, title FROM notes ORDER BY updated_at DESC, id DESC LIMIT 20".to_string(),
        ),
    ] {
        println!("[{label}]");
        let mut rows = conn.query(&format!("EXPLAIN QUERY PLAN {sql}"), ()).await?;
        while let Some(row) = rows.next().await? {
            println!("    {}", row.get::<String>(3)?);
        }
    }
    Ok(())
}
```

```
cargo run -q --example u02_q2
```

```
[RustNote の一覧]
    SEARCH notes USING INDEX idx_notes_user_updated (user_id=?)
[持ち主の条件が無い一覧]
    SCAN notes
    USE TEMP B-TREE FOR ORDER BY
```

- RustNote の一覧の SQL は `SEARCH notes USING INDEX idx_notes_user_updated (user_id=?)` でした。索引を使って、持ち主（`user_id`）のノートだけを探しています。しかも、並べ替えの手順（`USE TEMP B-TREE FOR ORDER BY`）がありません。索引が、はじめから `updated_at DESC, id DESC` の順に並んでいるので、並べ替えが要らないのです。
- 持ち主の条件が無い SQL は、`SCAN notes`（全部の行を読む）と `USE TEMP B-TREE FOR ORDER BY`（読んだあとで並べ替える）でした。ノートが増えるほど遅くなります。索引の列の順番（`user_id` が先）は、「持ち主で絞ってから、新しい順に並べる」という使い方に合わせて決めてあるのです（第7巻 UNIT 04）。
- `r"..."` は、`\` をそのまま書ける Rust の文字列の書き方です。SQL の `ESCAPE '\'` を、`\\` と書かずに済みます。

問3の解答:

`examples/u02_q3.rs` です。

```
//! 問3の解答: まだ無いフォルダの中の DB ファイルを開こうとすると、どうなるか。
//! 実行: cargo run -q --example u02_q3

use rustnote::db::Db;

#[tokio::main]
async fn main() {
    let dir = tempfile::tempdir().expect("一時フォルダを作れません");
    // ファイルは無くてもよいが、フォルダ（data）は作っていない
    let path = dir.path().join("data").join("rustnote.db");
    match Db::open_local(&path).await {
        Ok(_) => println!("開けました"),
        Err(error) => println!("開けません: {error}"),
    }

    // フォルダを作ってから開くと、ファイルは自動で作られる
    std::fs::create_dir(dir.path().join("data")).expect("フォルダを作れません");
    match Db::open_local(&path).await {
        Ok(_) => println!(
            "フォルダを作ってから: 開けました（ファイルがある: {}）",
            path.exists()
        ),
        Err(error) => println!("フォルダを作ってから: 開けません: {error}"),
    }
}
```

```
cargo run -q --example u02_q3
```

```
開けません: Failed to connect to database: `Unable to open connection to local database
...\rustnote.db: 14`
フォルダを作ってから: 開けました（ファイルがある: true）
```

- フォルダ（`data`）が無いと、`Unable to open connection to local database ...\rustnote.db: 14` で開けませんでした。14 は SQLite のエラーの番号で、「ファイルを開けない」（`SQLITE_CANTOPEN`）という意味です。SQLite は、ファイルは自動で作りますが、**フォルダは作りません**。
- フォルダを作ってから開くと、ファイルが自動で作られました。
- RustNote では、このエラーは `main.rs` の `with_context` で「DB（data/rustnote.db）を開けませんでした」と包まれて表示され、起動が止まります。公開するときの `Dockerfile` が、`mkdir -p /data` でフォルダを先に作っているのは、このためです（UNIT 10（準備中））。

まとめ

- RustNote の表は、`migrations/` の番号順の SQL ファイルが作る。`db::migrate` は、未適用の版だけを古い順に、1つずつトランザクションの中で適用し、SQL の実行と記録簿への書き込みを同じトランザクションに入れる。失敗した版の作りかけは、`rollback` で残らない。
- `Db::open_local` は WAL（ファイルに記録される）を1回入れ、`Db::connect` は外部キーと待ち時間（接続ごと）を毎回入れる。libsql 0.10 は素の接続でも外部キーが ON だったが、RustNote は明示している。
- 適用済みの SQL ファイルは書き換えない。古い DB には反映されず、新しい DB とだけ形が変わってしまう。変更は、新しい版として足す。

UNIT 03: 設定と起動の順番

この単元のゴール

- `src/config.rs` が、環境変数から `Config` を作る決まり（既定値・Turso ならトークンが要る・空のトークンは「無い」と同じ）を、6つの組み合わせの実際の結果で説明できる
- `Secret` 型が、`{:?}` ではトークンを隠し、`{}` ではそもそも表示させない仕組みを説明できる
- `src/main.rs` の起動の順番（`.env` → 設定 → CSS の束 → DB を開く → マイグレーション → ルーター → 待ち受け）を言え、どの段で止まると何が残るかを、本物の実行ファイルで確かめられる
- `src/lib.rs` と `src/main.rs` を分けている理由と、`pub mod` を1つ消すと何が起きるかを説明できる

たとえ話でつかむ

お店の、朝の開店の準備を考えます。

- まず、**今日の決まり**（営業時間・レジの金額）を、店長の書き置き（`.env`）や本部からの連絡（環境変数）で確かめます。本部からの連絡と書き置きが食い違うときは、**本部の連絡が勝ち**ます。
- 次に、**看板と飾り**（CSS の束）がそろっているかを見ます。無ければ、倉庫を開ける前にやめます。

- それから、**倉庫**（DB）を開けて、棚の配置が今日の決まりどおりか（マイグレーション）を確かめ、足りない棚を組み立てます。
- 最後に、**シャッターを開けて**（待ち受け）、お客さんを入れます。
- 金庫の暗証番号（トークン）は、メモ（ログ）に書くときは `********` と伏せます。そもそも、店先の黒板（`{}` の表示）には書けないようにしてあります。
- 準備は、**手早く確かめられる物から**順にします。看板が無いと分かっているのに、倉庫の棚を組み立ててしまうと、片付けが増えるからです。

開店の準備	RustNote の起動（`src/main.rs`）
店長の書き置き	`.env`（`dotenvy::dotenv().ok()`。無くてもよい）
本部からの連絡	環境変数（`.env` より優先）
今日の決まり	`Config::from_env()`（`src/config.rs`）
看板と飾り	`AssetBundle::load()`（CSS の束）
倉庫を開けて棚を組む	`Db::open` と `db::migrate`（UNIT 02）
シャッターを開ける	`topcoat::start(router)`（`HOST`・`PORT` で待ち受け）
暗証番号を伏せる	`Secret` 型（`{:?}` で `Secret(********)`）

> **用語:** 環境変数（かんきょうへんすう：プログラムの外から渡す、名前と値の組。`DATABASE_URL=rustnote.db` のように書く。Fly.io では `fly secrets set` で設定する（第12巻））

> **用語:** `.env`（ドット・エンブ：開発のとき、環境変数をまとめて書いておくファイル。RustNote は起動のときに読むが、すでにある環境変数は上書きしない）

図で理解する

`src/main.rs` の起動の順番と、それぞれの段で止まったときに残る物です（例3-3の結果）。

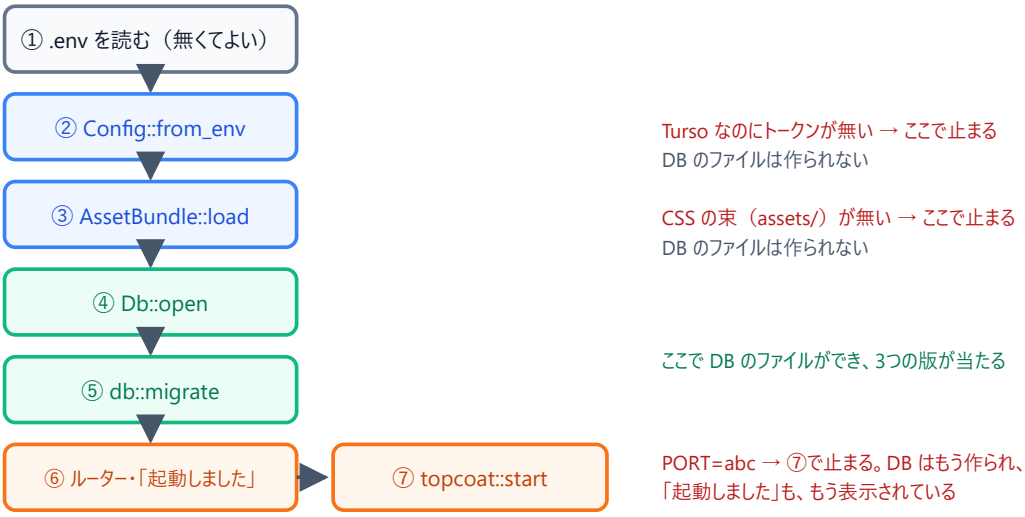


図13-10: 起動の順番。手早く確かめられる設定と CSS の束を先に見て、DB に触る前に止める。待ち受けのアドレス（HOST・PORT）だけは最後の topcoat::start で読むので、DB を作ってから止まる

まず動かす最小コード

例3-1: src/config.rs の全文

▼ **このファイルがやること（先に日本語で）**：環境変数の `DATABASE_URL` と `DATABASE_AUTH_TOKEN` を読んで、`Config` を作ります。`DATABASE_URL` が無ければ `rustnote.db`、Turso の URL（`libsql://`・`https://`・`wss://`）なのにトークンが無ければ、エラーにします。トークンは、読んだらすぐ `Secret` 型に包み、`{:?}` で表示しても中身が出ないようにします。待ち受けのアドレスを、起動の知らせのために組み立てる関数 `listen_address` もあります。

```rust
//! 設定（Config）: 環境変数から読み込む。
//!
//! | 環境変数 | 意味 | 既定値 |
//! |---|---|---|
//! | `DATABASE_URL` | ローカルの DB ファイルのパス、または Turso の URL（`libsql://...`） |
`rustnote.db` |
//! | `DATABASE_AUTH_TOKEN` | Turso の認証トークン（リモートのときだけ必要） | なし |
//! | `HOST` / `PORT` | 待ち受けるアドレス。`topcoat::start` が直接読む | `127.0.0.1` /
`3000` |
//!
//! トークンのような秘密の値は `Secret` 型に包む。`{:?}` で設定全体をログに出しても、
//! 中身は `Secret(********)` としか表示されない。

use std::env;
use std::fmt;

use thiserror::Error;

/// DATABASE_URL が無いときに使うローカルの DB ファイル
pub const DEFAULT_DATABASE_URL: &str = "rustnote.db";

// ---------------------------------------------------------------------------
// 秘密の値を包む型
// ---------------------------------------------------------------------------

/// 秘密の値（Turso のトークンなど）。
/// Debug は derive せずに手で書き、中身の代わりに `********` を表示する。
/// Display は実装しないので、`{}` で表示しようとするとコンパイルエラーになる。
#[derive(Clone, PartialEq, Eq)]
pub struct Secret(String);

impl Secret {
    pub fn new(value: String) -> Secret {
        Secret(value)
    }

    /// 中身が本当に必要な場所（DB に接続するとき）だけで呼ぶ。
    /// わざと目立つ名前にして、秘密を使っている場所をすぐ見つけられるようにしている
    pub fn expose(&self) -> &str {
        &self.0
    }
}

// {:?} で表示されても中身を出さない
impl fmt::Debug for Secret {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str("Secret(********)")
    }
}
```

```rust
// ------------------------------------------------------------------------
// 設定
// ------------------------------------------------------------------------

/// アプリの設定
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Config {
    /// ローカルファイルのパス（例: `rustnote.db`）か、Turso の URL（例: `libsql://...`）
    pub database_url: String,
    /// Turso の認証トークン（秘密）。ローカルファイルのときは `None`
    pub database_auth_token: Option<Secret>,
}

/// 設定を読めなかった理由
#[derive(Debug, Error, PartialEq, Eq)]
pub enum ConfigError {
    #[error("環境変数 {0} の値が UTF-8 の文字列として読めません")]
    NotUnicode(&'static str),
    #[error("DATABASE_URL が Turso（{0}）なのに DATABASE_AUTH_TOKEN が設定されていません")]
    MissingAuthToken(String),
}

impl Config {
    /// 環境変数から設定を作る
    pub fn from_env() -> Result<Config, ConfigError> {
        let database_url =
            read_var("DATABASE_URL")?.unwrap_or_else(||
DEFAULT_DATABASE_URL.to_string());
        // 空文字の DATABASE_AUTH_TOKEN は「設定されていない」と同じに扱う。
        // 読んだらすぐ Secret に包み、String のまま持ち歩かない
        let database_auth_token = read_var("DATABASE_AUTH_TOKEN")?
            .filter(|t| !t.is_empty())
            .map(Secret::new);
        Config::new(database_url, database_auth_token)
    }

    /// 値を直接指定して設定を作る（テストでも使う）
    pub fn new(
        database_url: String,
        database_auth_token: Option<Secret>,
    ) -> Result<Config, ConfigError> {
        let config = Config {
            database_url,
            database_auth_token,
        };
        if config.requires_auth_token() && config.database_auth_token.is_none() {
            return Err(ConfigError::MissingAuthToken(config.database_url));
        }
        Ok(config)
```

```
    }

    /// ネットワークの向こうの DB（Turso / sqld）につなぐなら true
    pub fn is_remote(&self) -> bool {
        ["libsql://", "https://", "http://", "wss://", "ws://"]
            .iter()
            .any(|prefix| self.database_url.starts_with(prefix))
    }

    /// Turso（インターネット上の DB）ならトークンが必須
    fn requires_auth_token(&self) -> bool {
        ["libsql://", "https://", "wss://"]
            .iter()
            .any(|prefix| self.database_url.starts_with(prefix))
    }
}

/// 環境変数を1つ読む。無ければ `Ok(None)`
fn read_var(name: &'static str) -> Result<Option<String>, ConfigError> {
    match env::var(name) {
        Ok(value) => Ok(Some(value)),
        Err(env::VarError::NotPresent) => Ok(None),
        Err(env::VarError::NotUnicode(_)) => Err(ConfigError::NotUnicode(name)),
    }
}

/// 起動時のログに出す「待ち受けアドレス」。
/// `topcoat::start` と同じ規則（HOST / PORT、既定 127.0.0.1:3000）で組み立てる。
pub fn listen_address() -> String {
    let host = env::var("HOST").unwrap_or_else(|_| "127.0.0.1".to_string());
    let port = env::var("PORT").unwrap_or_else(|_| "3000".to_string());
    format!("{host}:{port}")
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn local_file_needs_no_token() {
        let config = Config::new("rustnote.db".to_string(), None).unwrap();
        assert!(!config.is_remote());
    }

    #[test]
    fn turso_url_needs_a_token() {
        let result = Config::new("libsql://example.turso.io".to_string(), None);
        assert_eq!(
            result,
            Err(ConfigError::MissingAuthToken(
```

```
                "libsql://example.turso.io".to_string()
            ))
        );

        let config = Config::new(
            "libsql://example.turso.io".to_string(),
            Some(Secret::new("token".to_string())),
        )
        .unwrap();
        assert!(config.is_remote());
    }

    #[test]
    fn debug_output_hides_the_token() {
        let config = Config::new(
            "libsql://example.turso.io".to_string(),
            Some(Secret::new("very-secret-token".to_string())),
        )
        .unwrap();

        // 設定全体を {:?} で表示しても、トークンの中身は出ない
        let shown = format!("{config:?}");
        assert!(!shown.contains("very-secret-token"), "{shown}");
        assert!(shown.contains("database_auth_token: Some(Secret(********))"));

        // 中身が必要な場所では expose() で取り出す
        let token = config.database_auth_token.as_ref().map(Secret::expose);
        assert_eq!(token, Some("very-secret-token"));
    }

    #[test]
    fn local_sqld_over_http_needs_no_token() {
        let config = Config::new("http://127.0.0.1:8080".to_string(), None).unwrap();
        assert!(config.is_remote());
    }
}
```

▼ コードを1つずつ分解して解説

解説1: Secret は「隠す」と「出させない」の2つで守る

- `Debug` を `derive` せずに手で書き、`Secret(********)` とだけ表示します。だから、`Config` を `{:?}` でログに出しても（`Config` は `#[derive(Debug)]`）、トークンの中身は出ません。
- `Display` は**実装していません**。うっかり `{}` で表示しようとすると、コンパイルエラーになります（エラー2）。
- 中身が本当に要る所（`src/db.rs` の `Db::open` で Turso につなぐ所）だけで、`expose()` を呼びます。目立つ名前なので、`expose` で探せば、秘密を使っている場所がすぐ分かります。

解説2: from_env と new を分ける

- `from_env` は「環境変数を読む」だけで、決まりの確かめは `new` に任せています。テストでは、環境変数を書き換えずに `Config::new(…)` で決まりを確かめられます（ `#[cfg(test)]` の4つのテスト）。
- `is_remote`（ネットの向こうの DB か）と `requires_auth_token`（トークンが要るか）は、別の判定です。`http://` と `ws://` は「手元で動かす sqld（libSQL のサーバー）」のためのもので、リモートですが、トークンは要りません（例3-3の 6）。

解説3: 環境変数が読めないときと、無いとき

- `read_var` は、無ければ `Ok(None)`、UTF-8 として読めなければ `ConfigError::NotUnicode` を返します。「無い」と「おかしい」を分けているので、既定値を使ってよい場合と、止めるべき場合を、取り違えません。
- 空の文字列のトークン（ `DATABASE_AUTH_TOKEN=` ）は、`filter(|t| !t.is_empty())` で「無い」と同じにしています。`.env.example` をコピーしたままの `DATABASE_AUTH_TOKEN=` の行が、「空のトークンで接続しようとする」事故にならないようにするためです。

例3-2: src/main.rs と src/lib.rs の全文

> **▼ このファイルがやること（先に日本語で）：** `src/main.rs` は、起動の順番そのものです。`.env` を読み、設定を作り、CSS の束を読み、DB を開いてマイグレーションをかけ、ルーターを組み立てて待ち受けます。どこかで失敗したら、`anyhow` の `context` で「何をしようとして失敗したか」を付けて表示し、終わります。`src/lib.rs` は、アプリの中身を部品（モジュール）として公開するだけのファイルです。

```rust
//! RustNote の実行ファイルの入口。
//!
//! 起動の流れ:
//! 1. .env があれば環境変数として読み込む（開発用。本番では使わない）
//! 2. 設定（Config）を環境変数から作る
//! 3. アセット（CSS）の束を読み込む
//! 4. DB を開き、まだ適用していないマイグレーションを適用する
//! 5. ルーターを組み立て、HOST:PORT（既定 127.0.0.1:3000）で待ち受ける
//!
//! 起動時の失敗は anyhow で「何をしようとして失敗したか」を付けて表示し、終了する。

use anyhow::Context;
use rustnote::{
    config::{self, Config},
    db::{self, Db},
    routes,
    state::AppState,
};
use topcoat::asset::AssetBundle;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // .env が無くてもエラーにしない（本番では環境変数を直接設定する）
    dotenvy::dotenv().ok();

    let config = Config::from_env().context("設定を読み込めませんでした")?;

    // `topcoat asset bundle`（または `topcoat dev`）が実行ファイルの隣に作る assets/ を読
む。
    // 無ければ DB に触る前に止める
    let assets = AssetBundle::load().context(
        "アセットの束（assets/）がありません。先に `topcoat asset bundle` を実行するか、
`topcoat dev` で起動してください",
    )?;

    let db = Db::open(&config)
        .await
        .with_context(|| format!("DB（{}）を開けませんでした", config.database_url))?;
    let applied = db::migrate(&db)
        .await
        .context("マイグレーションに失敗しました")?;
    for version in &applied {
        eprintln!("マイグレーションを適用しました: {version}");
    }

    let router = routes::build_router(AppState::new(db), assets);

    eprintln!(
        "RustNote を起動しました: http://{}",
```

```
        config::listen_address()
    );
    topcoat::start(router)
        .await
        .context("サーバーを起動できませんでした（HOST / PORT を確認してください）")?;
    Ok(())
}
```

```
//! RustNote のライブラリ部分（アプリ本体）。
//!
//! 実行ファイル（src/main.rs）とテスト（tests/）の両方から使えるように、
//! アプリの中身はすべてライブラリとして書き、ここで部品（モジュール）を公開する。
//!
//! 部品の役割:
//! - config     … 環境変数から設定（Config）を読む
//! - domain     … 検証済みの型（UserId・NoteId・Email・NoteTitle など）
//! - error      … エラーの分類（RepoError・AppError）と HTTP ステータスへの対応
//! - db         … DB への接続とマイグレーション
//! - repository … DB の読み書き（NoteRepository などの trait と libSQL 版の実装）
//! - state      … アプリ全体で共有する状態（AppState）
//! - auth       … パスワードのハッシュ化・ログイン・ログイン中のユーザー
//! - flash      … 「保存しました」などの一度だけ出すメッセージ
//! - views      … 画面の部品（コンポーネント）とエラー画面
//! - routes     … URL ごとの処理（ページ・フォームの受け取り）とルーター

pub mod auth;
pub mod config;
pub mod db;
pub mod domain;
pub mod error;
pub mod flash;
pub mod repository;
pub mod routes;
pub mod state;
pub mod views;
```

解説4: 手早く確かめられる物から

- 設定（②）と CSS の束（③）は、ファイルや環境変数を読むだけなので、一瞬で確かめられます。DB を開く（④）とマイグレーション（⑤）は、ファイルを作ったり、ネットの向こうに接続したりします。だから、先に②③を確かめ、だめなら、DB に触る前に止めます（例3-3）。
- `main` が返す `anyhow::Result<()>` は、`Err` で終わると、`Error:` と、`context` で付けた文、`Caused by:` の下に元のエラーを表示して、終了の番号を 1 にします。

解説5: main.rs は薄く、中身は lib.rs に

- アプリの中身は、全部 `src/lib.rs` から公開する部品（`rustnote::config` など）に置き、`src/main.rs` は、それを順に呼ぶだけです。こうすると、`tests/` のテストや `examples/` の練習のプログラムも、同じ部品を `rustnote::…` で使えます（第9巻 UNIT 04）。
- `pub mod` の1行が、そのファイル（`src/flash.rs` など）を、このクレートの部品にする宣言です。1行消すと、そのファイルは無いのと同じになります（エラー1）。

もう一歩: 実務に近い形

例3-3: 環境変数の組み合わせと、本物の起動

まず、`Config::from_env` が、6つの組み合わせをどう読むかを、練習用のプログラムで確かめます。

> **▼ このコードがやること（先に日本語で）**: `DATABASE_URL` と `DATABASE_AUTH_TOKEN` を、6通りに書き換えては、`Config::from_env()` を呼んで、結果（`Ok` の中身か、`Err` の文）を表示します。edition 2024 では、プログラムの中で環境変数を書き換える `set_var`・`remove_var` は `unsafe` なので、`unsafe { … }` の中で呼びます（エラー3）。

```rust
//! 第13巻 UNIT 03: 環境変数の組み合わせごとに、Config::from_env がどう読むかを確かめる。
//! 実行: cargo run -q --example u03_config

use std::env;

use rustnote::config::Config;

/// 環境変数をそろえてから、Config::from_env を呼んで結果を表示する
fn try_with(label: &str, url: Option<&str>, token: Option<&str>) {
    // SAFETY: このプログラムは1つのスレッドだけで動き、ほかに環境変数を読む人がいない
    unsafe {
        match url {
            Some(value) => env::set_var("DATABASE_URL", value),
            None => env::remove_var("DATABASE_URL"),
        }
        match token {
            Some(value) => env::set_var("DATABASE_AUTH_TOKEN", value),
            None => env::remove_var("DATABASE_AUTH_TOKEN"),
        }
    }
    match Config::from_env() {
        Ok(config) => println!(
            "{label}\n  → Ok: {config:?} (remote: {}) ",
            config.is_remote()
        ),
        Err(error) => println!("{label}\n  → Err: {error}"),
    }
}

fn main() {
    try_with("1. 何も設定しない", None, None);
    try_with("2. ローカルのファイル", Some("data/rustnote.db"), None);
    try_with(
        "3. Turso の URL・トークン無し",
        Some("libsql://rustnote-demo.turso.io"),
        None,
    );
    try_with(
        "4. Turso の URL・トークンが空",
        Some("libsql://rustnote-demo.turso.io"),
        Some(""),
    );
    try_with(
        "5. Turso の URL・トークンあり",
        Some("libsql://rustnote-demo.turso.io"),
        Some("very-secret-token"),
    );
    try_with(
        "6. 手元の sqld（http）・トークン無し",
```

```
        Some("http://127.0.0.1:8080"),
        None,
    );
}
```

```
cargo run -q --example u03_config
```

```
1. 何も設定しない
  → Ok: Config { database_url: "rustnote.db", database_auth_token: None } (remote:
false)
2. ローカルのファイル
  → Ok: Config { database_url: "data/rustnote.db", database_auth_token: None } (remote:
false)
3. Turso の URL・トークン無し
  → Err: DATABASE_URL が Turso (libsql://rustnote-demo.turso.io) なのに
DATABASE_AUTH_TOKEN が設定されていません
4. Turso の URL・トークンが空
  → Err: DATABASE_URL が Turso (libsql://rustnote-demo.turso.io) なのに
DATABASE_AUTH_TOKEN が設定されていません
5. Turso の URL・トークンあり
  → Ok: Config { database_url: "libsql://rustnote-demo.turso.io", database_auth_token:
Some(Secret(********)) } (remote: true)
6. 手元の sqld (http) ・トークン無し
  → Ok: Config { database_url: "http://127.0.0.1:8080", database_auth_token: None }
(remote: true)
```

- 4（空のトークン）は、3（トークン無し）と同じエラーでした。5 は、トークンの中身が `Secret(********)` と隠れて表示されました。6 の `http://` は、リモート（`remote: true`）ですが、トークン無しで通りました。

次に、本物の実行ファイル（`cargo build` と `topcoat asset bundle` を済ませたもの）を、4つの設定で起動しました。それぞれのあとで、作業のフォルダの中のファイルも表示しました（`run_case` という小さなスクリプトで、標準エラー出力と、フォルダの中身を記録しました）。

▼ ふつうに起動（`PORT=3014`・`DATABASE_URL=ok.db`）

```
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://127.0.0.1:3014
（このあとのフォルダの中: ok.db ）
HTTP/1.1 200 OK
```

▼ Turso の URL で、トークンが無い

```
Error: 設定を読み込めませんでした

Caused by:
    DATABASE_URL が Turso（libsql://rustnote-demo.turso.io）なのに DATABASE_AUTH_TOKEN が設定されていません
（このあとのフォルダの中: ok.db ）
```

▼ CSS の束（assets/）が無いフォルダの実行ファイル

```
Error: アセットの束（assets/）がありません。先に `topcoat asset bundle` を実行するか、`topcoat dev` で起動してください

Caused by:
    no asset bundle at ...\u03exe\assets
（このあとのフォルダの中: ok.db port.db ）
```

▼ `PORT=abc`（数ではないポート）

```
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://127.0.0.1:abc
Error: サーバーを起動できませんでした（HOST / PORT を確認してください）

Caused by:
    PORT must be a valid port number: invalid digit found in string
（このあとのフォルダの中: ok.db port.db ）
```

- ふつうの起動では、3つの版が当たり、`/health` が `200 OK` でした。`ok.db` ができました。
- トークンが無いときと、CSS の束が無いときは、②・③で止まり、**DB のファイルは作られません**でした（フォルダの中は、前の `ok.db` だけ）。
- `PORT=abc` は、⑦の `topcoat::start` で初めて読まれるので、その前に、**`port.db` を作って3つの版を当て**、しかも「**RustNote を起動しました: http://127.0.0.1:abc**」と表示してから、`PORT must be a valid port number`（PORT は正しいポートの番号でなければならない）で止まりました。起動の知らせが、待ち受けを始める**前**に出ているので、ログだけを見ると、起動したように見えてしまいます（改善点。問3）。

よくあるエラーと読み方

エラー1: `src/lib.rs` から `pub mod flash;` を消した

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0432]: unresolved import `crate::flash`
  --> src\auth.rs:28:12
   |
28 | use crate::flash::{self, Flash};
   |            ^^^^^ could not find `flash` in the crate root
  …（同じエラーが続く）…
error: could not compile `rustnote` (lib) due to 11 previous errors
```

- **何が起きたか:** `src/flash.rs` は、ファイルとしてはあるのに、`pub mod flash;` が無いので、クレートの部品になりません。`use crate::flash::…` と書いた 11 か所が、「クレートの根元に `flash` が見つかりません」（E0432）になりました。
- **読み方:** 同じエラーがたくさん出ても、原因は1つのことがよくあります。`could not find flash in the crate root`（根元に無い）なので、根元（`src/lib.rs`）の `mod` の宣言を見ます。
- **直し方:** `pub mod flash;` を戻します。新しいファイルを足したときも、`src/lib.rs`（または親のモジュール）に `mod` の1行を足すのを忘れないようにします。

エラー2: `Secret` を `{}` で表示しようとした（E0277）

```rust
//! 第13巻 UNIT 03 エラー2: Secret を {} で表示しようとした。
//! 実行: cargo run -q --example e03_display

use rustnote::config::Secret;

fn main() {
    let token = Secret::new("very-secret-token".to_string());
    println!("トークン: {}", token);
}
```

```
error[E0277]: `Secret` doesn't implement `std::fmt::Display`
  |
8 |     println!("トークン: {}", token);
  |                          --   ^^^^^ `Secret` cannot be formatted with the default
formatter
  |                          |
  |                          required by this formatting parameter
  |
  = help: the trait `std::fmt::Display` is not implemented for `Secret`
  = note: in format strings you may be able to use `{:?}` (or `{:#?}` for pretty-print)
instead
```

- **何が起きたか:**「`Secret` は `Display` を実装していません」というエラーです。`Secret` は、わざと `Display` を実装していないので、`{}` では表示できません。
- **読み方:** `note:` の「`{:?}` を使えるかもしれません」のとおり、`{:?}` なら表示できますが、出るのは `Secret(********)` です。中身が要るなら、`expose()` を呼んだ所が「秘密を使う所」だと、読む人に分かるようにします。

エラー3: edition 2024 で、`set_var` を `unsafe` なしで呼んだ（E0133）

```
//! 第13巻 UNIT 03 エラー3: edition 2024 で、環境変数を unsafe なしで書き換えようとした。
//! 実行: cargo run -q --example e03_setvar

use rustnote::config::Config;

fn main() {
    std::env::set_var("DATABASE_URL", "data/rustnote.db");
    println!("{:?}", Config::from_env());
}
```

```
error[E0133]: call to unsafe function `set_var` is unsafe and requires unsafe block
  |
7 |     std::env::set_var("DATABASE_URL", "data/rustnote.db");
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ call to unsafe function
  |
  = note: consult the function's documentation for information on how to avoid
undefined behavior
```

- **何が起きたか:**「`set_var` は `unsafe` な関数なので、`unsafe` のブロックが要ります」というエラーです。edition 2024 から、環境変数を書き換える `std::env::set_var` と `remove_var` は `unsafe` になりました。ほかのスレッドが同じ時に環境変数を読むと、OS によっては、こわれた値を読むおそれがあるからです。

- **直し方**: 例3-3のように、ほかのスレッドが環境変数を読まないと分かっている所で、`unsafe { … }` に入れ、`// SAFETY:` のコメントで、なぜ安全かを書きます。アプリの中で環境変数を書き換えるのは、なるべく避けます（RustNote は、読むだけです）。

RustNote ではここで使う

- **本番の設定**: Fly.io では、`DATABASE_URL` と `DATABASE_AUTH_TOKEN` を `fly secrets set` で渡し、`.env` は使いません（第12巻）。`HOST=0.0.0.0`・`PORT=8080` は、`fly.toml` の `[env]` で渡します（UNIT 10（準備中））。
- **テスト**: `tests/common/mod.rs` は、`Config` を使わず、`Db::open_local` で一時フォルダの DB を直接開きます（UNIT 02）。設定の決まりは、`src/config.rs` の4つのユニットテストで確かめます。

```
running 4 tests
test config::tests::local_file_needs_no_token ... ok
test config::tests::local_sqld_over_http_needs_no_token ... ok
test config::tests::debug_output_hides_the_token ... ok
test config::tests::turso_url_needs_a_token ... ok
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 15 filtered out; finished
in 0.00s
```

- **改善点（この単元で見つかったこと）**：`PORT` が正しくないと、DB を作ってマイグレーションを当て、「起動しました」と表示してから止まります（例3-3）。問3で、直し方を考えます。

練習問題

問1: `.env` に `DATABASE_URL=…` と書き、同じ名前の環境変数も設定して起動すると、どちらが使われるでしょうか。

問2: `src/config.rs` のテスト `debug_output_hides_the_token` は、何を確かめているでしょうか。`Config` に、あとから秘密の項目（たとえばメールを送るサービスのパスワード）を足すときに、気をつけることは何でしょうか。

問3: 例3-3の `PORT=abc` では、DB のファイルを作ってから止まり、「起動しました」とも表示されました。どう直せばよいでしょうか。

解答と解説

問1の解答:

作業のフォルダの `.env` に `DATABASE_URL=libsql://from-dotenv.example` と書き、①環境変数にも `DATABASE_URL=libsql://from-env.example` を設定して起動、②環境変数を設定せずに起動、の2回を比べました（どちらもトークンが無いので、設定のエラーで止まり、どちらの値が使われたかが分かります）。

```
Error: 設定を読み込めませんでした
Caused by:
    DATABASE_URL が Turso（libsql://from-env.example）なのに DATABASE_AUTH_TOKEN が設定されていません
```

```
Error: 設定を読み込めませんでした
Caused by:
    DATABASE_URL が Turso（libsql://from-dotenv.example）なのに DATABASE_AUTH_TOKEN が設定されていません
```

- 環境変数があるときは、環境変数（`from-env`）が使われ、無いときだけ `.env`（`from-dotenv`）が使われました。`dotenvy::dotenv()` は、**すでにある環境変数を上書きしません。**
- だから、本番（Fly.io）で環境変数を設定していれば、間違って `.env` がまぎれ込んでいても、本番の値が勝ちます。逆に、手元で `.env` を書き換えたのに効かないときは、同じ名前の環境変数が残っていないかを確かめます。

問2の解答:

- `Config` 全体を `{:?}` で文字にしたとき、①トークンの中身（`very-secret-token`）が含まれないこと、② `database_auth_token: Some(Secret(********))` と表示されること、③ `expose()` で取り出すと、ちゃんと中身が返ること、の3つを確かめています（例3-1のテストの結果のとおり、通っています）。
- 秘密の項目を足すときは、`String` のままではなく、`Secret` に包みます。`Config` は `#[derive(Debug)]` なので、`String` の項目は `{:?}` でそのまま表示されてしまいます。足したら、同じ形のテスト（中身が `{:?}` に出ない）も足します（第8巻 UNIT 10・第11巻 UNIT 07 の「記録に秘密を書かない」）。

問3の解答:

- **直し方1（先に確かめる）**：`PORT` と `HOST` も、②の設定のところで読んで確かめます（`Config` に `port: u16` を足し、数でなければ `ConfigError` にする）。そうすれば、DB に触る前に止まります。
- **直し方2（先に待ち受けの場所を取る）**：`topcoat::start` は、中で「`HOST`・`PORT` を読む → `TcpListener::bind` → `serve`」をしています（topcoat 0.11.0 の `serve.rs`）。`main` の中で、DB を開く前に `TcpListener::bind` までしておき、マイグレーションのあとで `topcoat::serve(listener, router)` を呼べば、ポートが使えないこと（ほかのプログラムが使っている、など）も、DB に触る前に分かります。
- 「**起動しました」の知らせ**: どちらの直し方でも、知らせは、待ち受けの場所を取れた**あと**に出します。ログを読む人（や、Fly.io の見張り）が、「起動した」と思い込まないようにするためです。教材のコードは、まだ直していません（巻末の総合演習で扱います）。

まとめ

- `src/config.rs` は、`DATABASE_URL`（無ければ `rustnote.db`）と `DATABASE_AUTH_TOKEN`（空は無いのと同じ）を読み、Turso の URL なのにトークンが無ければエラーにする。トークンは `Secret` に包み、`{:?}` では `Secret(********)`、`{}` ではコンパイルエラーにして守る。`.env` は、すでにある環境変数を上書きしない。

- `src/main.rs` は、.env → 設定 → CSS の束 → DB を開く → マイグレーション → ルーター → 待ち受けの順に動く。トークン無しと束無しは DB に触る前に止まり、`PORT=abc` は DB を作って「起動しました」と表示してから止まった（改善点）。
- `src/lib.rs` は部品を公開するだけで、`main.rs` は薄い。`pub mod` を1行消すと、そのファイルを使う 11 か所が E0432 になる。edition 2024 では `set_var` は `unsafe`。

UNIT 04: ドメインの型とエラーの分類

この単元のゴール

- `src/domain.rs` の「検査済みの型」（`NoteTitle`・`Email`・`NoteId` など）が、中身を private にして、`parse` を通らずには作れないようにしている仕組みを説明できる
- タイトル・本文・メール・パスワード・ID の、それぞれの「数え方」と「そろえ方」（trim するか・小文字にするか・改行をそろえるか）を、いろいろな入力の実際の結果で言える
- `src/error.rs` の2つの層（DB の `RepoError` と、利用者への返事の `AppError`）と、`From` による変換、`status_code` と `public_message` の役目を説明できる
- 型が守れていない約束（`NoteQuery` の `pub` な項目、NUL の題）を、本物の実行で確かめられる

たとえ話でつかむ

宅配便の受付の窓口を考えます。

- 窓口では、荷物の**大きさと重さを測り**、合格した荷物にだけ「**検査済み**」のシールを貼ります。シールは**窓口の係しか貼れません**。だから、奥で荷物を運ぶ人は、シールを見るだけで、測り直しません。
- 送り状の「届け先」と「依頼主」は、どちらも住所ですが、**欄が分かれている**ので、取り違えて書くことはありません。
- 断るときは、「重さも大きさも、両方オーバーです」と、**理由を全部まとめて**伝えます。1つ伝えて直してもらい、また次を伝えるのでは、お客さんが何度も並び直すことになります。
- 奥でトラックが故障しても、窓口では「時間をおいて、もう一度お越しください」とだけ伝え、**詳しい事情は業務日誌**に書きます。
- 「その荷物はありません」と「それは他人の荷物です」は、**同じ返事**にします。他人の荷物があること自体を、知らせないためです。

宅配の窓口	RustNote（ `src/domain.rs` ・ `src/error.rs` ）
大きさと重さを測る	`NoteTitle::parse` など（検査に通れば `Ok` 、だめなら `Err` ）
検査済みのシール	`NoteTitle` などの型（中身は private）
シールは窓口の係しか貼れない	中身が private なので、`domain` の外では `NoteTitle(…)` と書けない（エラー1）
届け先と依頼主の欄	`UserId` と `NoteId` （中身は同じ `i64` でも、別の型。エラー2）
理由を全部まとめて伝える	`NewNote::parse` （タイトルと本文の問題を両方集める）
窓口での説明と、業務日誌	`public_message()` （500 では中身を隠す）と、ログ
「ありません」と「他人のです」を同じに	他人のノートも `AppError::NotFound` （404）

> **用語:** 検査済みの型（けんさずみのかた：決まり（文字数など）の検査に合格した値だけが入る型。中身を private にして、検査をする関数（ `parse` ）を通らないと作れないようにする）

> **用語:** `thiserror` （ジス・エラー： `#[derive(Error)]` と `#[error("…")]` で、エラーの型と、その文（ `Display` ）を短く書けるようにするクレート）

図で理解する

外から来た値が検査済みの型に変わる流れ（上）と、失敗が利用者への返事に変わる流れ（下）です。

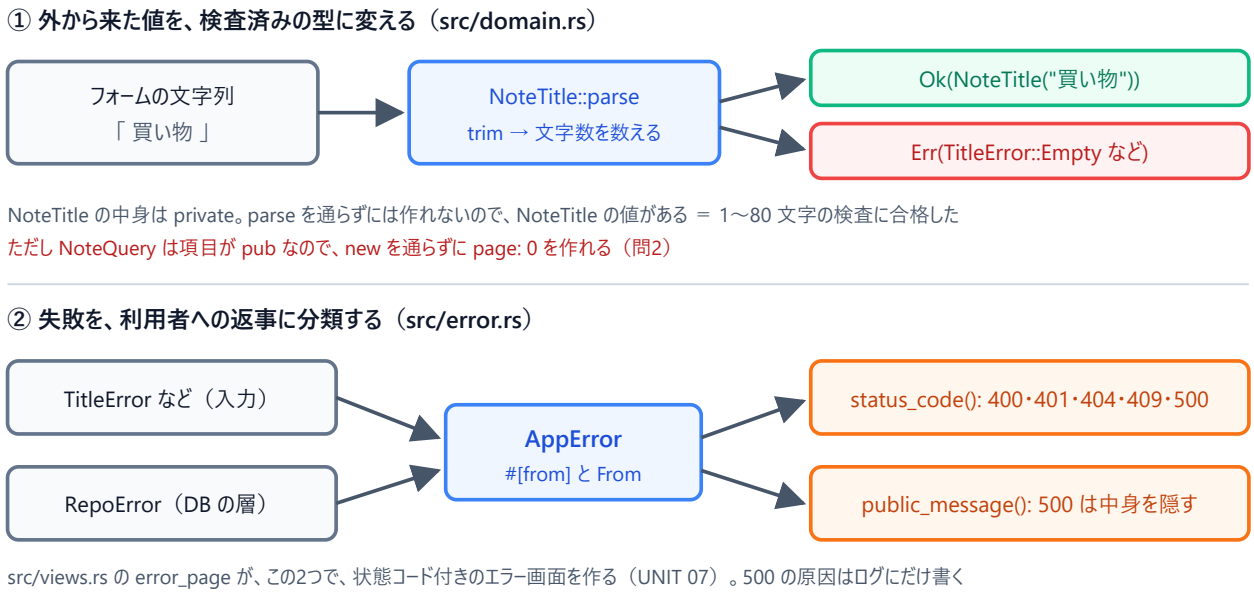


図13-11: 検査済みの型とエラーの分類。外から来た値は parse で型に変え、失敗は AppError にまとめてから、状態コードと画面の文に変える

まず動かす最小コード

例4-1: src/domain.rs の全文

> **▼ このファイルがやること（先に日本語で）**: RustNote が扱う「もの」の型を決めます。外から来た文字列（フォームの入力・URL の `{id}`）は、ここの `parse` を通して、検査済みの型（`NoteTitle`・`NoteBody`・`Email`・`Password`・`NoteId`）に変えます。ユーザーとノート（`User`・`Note`）、作成と編集の入力のまとまり（`NewNote`）、一覧の検索の条件とページ送り（`NoteQuery`・`NotePage`）も、ここにあります。最後に、13 のユニットテストのうち 11 があります。

```rust
//! ドメイン（アプリが扱う「もの」）の型。
//!
//! 外から来た文字列（フォーム・URL・DB の値）は、ここの `parse` / `new` を通して
//! 検査済みの型に変える。中身は private なので、検査を通らずに作る方法はない。
//! 「`NoteTitle` 型の値がある」＝「1〜80 文字の検査に合格した」が必ず成り立つ。

use std::fmt;
use std::str::FromStr;

use thiserror::Error;

// ---------------------------------------------------------------------------
// アプリ全体の定数
// ---------------------------------------------------------------------------

/// タイトルの最大文字数（chars().count() で数える。SQLite の length() と同じ）
pub const MAX_TITLE_CHARS: usize = 80;
/// 本文の最大文字数
pub const MAX_BODY_CHARS: usize = 10_000;
/// パスワードの最小文字数
pub const MIN_PASSWORD_CHARS: usize = 8;
/// パスワードの最大文字数
pub const MAX_PASSWORD_CHARS: usize = 128;
/// 一覧の1ページに表示するノートの数
pub const PER_PAGE: u32 = 20;
/// 検索キーワードの最大文字数（長すぎる入力で DB に負担をかけないため）
pub const MAX_KEYWORD_CHARS: usize = 100;

// ---------------------------------------------------------------------------
// ID（UserId / NoteId）: 中身は同じ i64 でも、取り違えるとコンパイルエラーになる
// ---------------------------------------------------------------------------

/// ID が不正だった理由
#[derive(Debug, Clone, Copy, PartialEq, Eq, Error)]
pub enum IdError {
    #[error("ID は数字で指定してください")]
    NotANumber,
    #[error("ID は 1 以上の数です")]
    NotPositive,
}

/// ユーザーの ID（users.id）
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct UserId(i64);

impl UserId {
    /// 1 以上の数だけを受け付ける
    pub fn new(value: i64) -> Result<UserId, IdError> {
        if value <= 0 {
```

```
            return Err(IdError::NotPositive);
        }
        Ok(UserId(value))
    }

    /// 中身の数を取り出す（DB に渡すときに使う）
    pub fn get(self) -> i64 {
        self.0
    }
}

impl fmt::Display for UserId {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.0)
    }
}

/// ノートの ID（notes.id）。URL の `/notes/{id}` から作る
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct NoteId(i64);

impl NoteId {
    /// 1 以上の数だけを受け付ける
    pub fn new(value: i64) -> Result<NoteId, IdError> {
        if value <= 0 {
            return Err(IdError::NotPositive);
        }
        Ok(NoteId(value))
    }

    /// 中身の数を取り出す（DB に渡すときに使う）
    pub fn get(self) -> i64 {
        self.0
    }
}

// "42".parse::<NoteId>() と書けるようにする。
// Topcoat の module_param! もこの FromStr を使って URL の {id} を NoteId に変える。
impl FromStr for NoteId {
    type Err = IdError;

    fn from_str(s: &str) -> Result<Self, Self::Err> {
        let value: i64 = s.parse().map_err(|_| IdError::NotANumber)?;
        NoteId::new(value)
    }
}

// href! で URL を組み立てるときに使う（/notes/42 の 42 の部分）
impl fmt::Display for NoteId {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
```

```
        write!(f, "{}", self.0)
    }
}

// ---------------------------------------------------------------------------
// メールアドレス
// ---------------------------------------------------------------------------

/// メールアドレスが不正だった理由
#[derive(Debug, Clone, Copy, PartialEq, Eq, Error)]
pub enum EmailError {
    #[error("メールアドレスに @ がありません")]
    MissingAt,
    #[error("メールアドレスの形式が正しくありません")]
    Malformed,
    #[error("メールアドレスが長すぎます")]
    TooLong,
}

/// 検査済みのメールアドレス（前後の空白を除き、小文字にそろえてある）
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct Email(String);

impl Email {
    /// メールアドレスの最大の長さ（バイト数）
    pub const MAX_LEN: usize = 254;

    pub fn parse(input: &str) -> Result<Email, EmailError> {
        // 同じ人を同じ値にするため、前後の空白を除いて小文字にそろえる
        let normalized = input.trim().to_lowercase();
        if normalized.len() > Self::MAX_LEN {
            return Err(EmailError::TooLong);
        }
        let Some((local, domain)) = normalized.split_once('@') else {
            return Err(EmailError::MissingAt);
        };
        let bad = local.is_empty()
            || domain.is_empty()
            || domain.contains('@')
            || !domain.contains('.')
            || domain.starts_with('.')
            || domain.ends_with('.')
            || normalized.contains(char::is_whitespace);
        if bad {
            return Err(EmailError::Malformed);
        }
        Ok(Email(normalized))
    }

    pub fn as_str(&self) -> &str {
```

```
        &self.0
    }
}

impl fmt::Display for Email {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str(&self.0)
    }
}

// ---------------------------------------------------------------------------
// パスワード（ハッシュ化する前の、利用者が入力したもの）
// ---------------------------------------------------------------------------

/// パスワードが不正だった理由
#[derive(Debug, Clone, Copy, PartialEq, Eq, Error)]
pub enum PasswordError {
    #[error("パスワードは{min}文字以上にしてください")]
    TooShort { min: usize },
    #[error("パスワードは{max}文字以内にしてください")]
    TooLong { max: usize },
}

/// 検査済みのパスワード。
/// Debug で中身が表示されないよう、Debug は手で実装している（ログへの漏れを防ぐ）。
#[derive(Clone, PartialEq, Eq)]
pub struct Password(String);

impl Password {
    pub fn parse(input: &str) -> Result<Password, PasswordError> {
        // パスワードは空白も含めて利用者が決めたものなので、trim しない
        let len = input.chars().count();
        if len < MIN_PASSWORD_CHARS {
            return Err(PasswordError::TooShort {
                min: MIN_PASSWORD_CHARS,
            });
        }
        if len > MAX_PASSWORD_CHARS {
            return Err(PasswordError::TooLong {
                max: MAX_PASSWORD_CHARS,
            });
        }
        Ok(Password(input.to_string()))
    }

    pub fn as_str(&self) -> &str {
        &self.0
    }
}
```

```rust
impl fmt::Debug for Password {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str("Password(***)")
    }
}

// ----------------------------------------------------------------------
// ノートのタイトルと本文
// ----------------------------------------------------------------------

/// タイトルが不正だった理由
#[derive(Debug, Clone, Copy, PartialEq, Eq, Error)]
pub enum TitleError {
    #[error("タイトルを入力してください")]
    Empty,
    #[error("タイトルは{max}文字以内にしてください（現在{len}文字）")]
    TooLong { len: usize, max: usize },
}

/// 検査済みのタイトル（前後の空白を除いて 1〜80 文字）
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct NoteTitle(String);

impl NoteTitle {
    pub const MAX_CHARS: usize = MAX_TITLE_CHARS;

    /// 文字列を検査し、合格したときだけ NoteTitle を返す
    pub fn parse(input: &str) -> Result<NoteTitle, TitleError> {
        let trimmed = input.trim();
        // 文字数は chars().count() で数える（len() はバイト数なので日本語で狂う）
        let len = trimmed.chars().count();
        if len == 0 {
            return Err(TitleError::Empty);
        }
        if len > Self::MAX_CHARS {
            return Err(TitleError::TooLong {
                len,
                max: Self::MAX_CHARS,
            });
        }
        Ok(NoteTitle(trimmed.to_string()))
    }

    pub fn as_str(&self) -> &str {
        &self.0
    }
}

impl fmt::Display for NoteTitle {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
```

```
        f.write_str(&self.0)
    }
}

/// 本文が不正だった理由
#[derive(Debug, Clone, Copy, PartialEq, Eq, Error)]
pub enum BodyError {
    #[error("本文は{max}文字以内にしてください（現在{len}文字）")]
    TooLong { len: usize, max: usize },
}

/// 検査済みの本文（空でもよい。最大 10,000 文字）
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct NoteBody(String);

impl NoteBody {
    pub const MAX_CHARS: usize = MAX_BODY_CHARS;

    pub fn parse(input: &str) -> Result<NoteBody, BodyError> {
        // ブラウザのフォームは改行を \r\n で送ってくるので \n にそろえる
        let normalized = input.replace("\r\n", "\n");
        let len = normalized.chars().count();
        if len > Self::MAX_CHARS {
            return Err(BodyError::TooLong {
                len,
                max: Self::MAX_CHARS,
            });
        }
        Ok(NoteBody(normalized))
    }

    pub fn as_str(&self) -> &str {
        &self.0
    }
}

// ---------------------------------------------------------------------------
// ユーザーとノート
// ---------------------------------------------------------------------------

/// ログインできるユーザー（パスワードのハッシュは含めない）
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct User {
    pub id: UserId,
    pub email: Email,
}

/// DB から読んだノート
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Note {
```

```rust
    pub id: NoteId,
    /// 持ち主（notes.user_id）
    pub owner: UserId,
    pub title: String,
    pub body: String,
    pub done: bool,
    /// 楽観的ロック用の番号。更新のたびに 1 増える
    pub version: i64,
    /// 作成日時（UTC、'YYYY-MM-DD HH:MM:SS'）
    pub created_at: String,
    /// 最終更新日時（UTC、'YYYY-MM-DD HH:MM:SS'）
    pub updated_at: String,
}

/// 作成・編集フォームの入力を検査したもの（タイトルと本文のセット）
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct NewNote {
    pub title: NoteTitle,
    pub body: NoteBody,
}

/// 作成・編集フォームの検査で見つかった問題（項目ごと）
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct NoteInputErrors {
    pub title: Option<TitleError>,
    pub body: Option<BodyError>,
}

impl NewNote {
    /// タイトルと本文をまとめて検査する。
    /// 両方の問題を一度に画面へ出せるよう、最初の失敗で止めずに全部集める。
    pub fn parse(title: &str, body: &str) -> Result<NewNote, NoteInputErrors> {
        match (NoteTitle::parse(title), NoteBody::parse(body)) {
            (Ok(title), Ok(body)) => Ok(NewNote { title, body }),
            (title, body) => Err(NoteInputErrors {
                title: title.err(),
                body: body.err(),
            }),
        }
    }
}

// ---------------------------------------------------------------------------
// 一覧の検索条件とページ送り
// ---------------------------------------------------------------------------

/// 一覧の検索条件（キーワードとページ番号）
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct NoteQuery {
    /// 検索キーワード。空なら None（すべて表示）
```

```
    pub keyword: Option<String>,
    /// 1 から始まるページ番号
    pub page: u32,
}

impl NoteQuery {
    /// URL の ?q= と ?page= から作る。空白だけのキーワードは無いのと同じに扱う
    pub fn new(keyword: Option<&str>, page: Option<u32>) -> NoteQuery {
        let keyword = keyword
            .map(str::trim)
            .filter(|k| !k.is_empty())
            .map(|k| k.chars().take(MAX_KEYWORD_CHARS).collect());
        NoteQuery {
            keyword,
            page: page.unwrap_or(1).max(1),
        }
    }

    /// SQL の OFFSET（何件読み飛ばすか）
    pub fn offset(&self) -> u64 {
        u64::from(self.page - 1) * u64::from(PER_PAGE)
    }
}

/// 一覧の1ページ分の結果
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct NotePage {
    pub notes: Vec<Note>,
    /// 今のページ番号（1 から）
    pub page: u32,
    /// 条件に合うノートの総数
    pub total_count: u64,
}

impl NotePage {
    /// 全部で何ページあるか（0 件でも 1 ページと数える）
    pub fn total_pages(&self) -> u32 {
        let pages = self.total_count.div_ceil(u64::from(PER_PAGE)).max(1);
        u32::try_from(pages).unwrap_or(u32::MAX)
    }

    pub fn has_prev(&self) -> bool {
        self.page > 1
    }

    pub fn has_next(&self) -> bool {
        self.page < self.total_pages()
    }
}
```

```rust
// ----------------------------------------------------------------------
// ユニットテスト（cargo test で実行される）
// ----------------------------------------------------------------------

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn note_title_is_trimmed() {
        let title = NoteTitle::parse("  買い物  ").unwrap();
        assert_eq!(title.as_str(), "買い物");
    }

    #[test]
    fn note_title_rejects_empty_and_spaces() {
        assert_eq!(NoteTitle::parse(""), Err(TitleError::Empty));
        assert_eq!(NoteTitle::parse("   "), Err(TitleError::Empty));
    }

    #[test]
    fn note_title_counts_characters_not_bytes() {
        // 日本語 80 文字は 240 バイトだが、文字数では 80 なので合格
        let eighty = "あ".repeat(80);
        assert!(NoteTitle::parse(&eighty).is_ok());

        let eighty_one = "あ".repeat(81);
        assert_eq!(
            NoteTitle::parse(&eighty_one),
            Err(TitleError::TooLong { len: 81, max: 80 })
        );
    }

    #[test]
    fn note_body_normalizes_newlines_and_limits_length() {
        let body = NoteBody::parse("1行目\r\n2行目").unwrap();
        assert_eq!(body.as_str(), "1行目\n2行目");

        let too_long = "a".repeat(MAX_BODY_CHARS + 1);
        assert!(matches!(
            NoteBody::parse(&too_long),
            Err(BodyError::TooLong { .. })
        ));
    }

    #[test]
    fn new_note_collects_all_errors() {
        let too_long_body = "a".repeat(MAX_BODY_CHARS + 1);
        let errors = NewNote::parse(" ", &too_long_body).unwrap_err();
        assert_eq!(errors.title, Some(TitleError::Empty));
```

```
    assert!(errors.body.is_some());
}

#[test]
fn email_is_normalized() {
    let email = Email::parse("  Alice@Example.COM ").unwrap();
    assert_eq!(email.as_str(), "alice@example.com");
}

#[test]
fn email_rejects_bad_forms() {
    assert_eq!(Email::parse("alice"), Err(EmailError::MissingAt));
    assert_eq!(Email::parse("@example.com"), Err(EmailError::Malformed));
    assert_eq!(Email::parse("alice@example"), Err(EmailError::Malformed));
    assert_eq!(Email::parse("a@b@c.com"), Err(EmailError::Malformed));
    assert_eq!(
        Email::parse("al ice@example.com"),
        Err(EmailError::Malformed)
    );
}

#[test]
fn password_length_is_checked_and_hidden_in_debug() {
    assert_eq!(
        Password::parse("short"),
        Err(PasswordError::TooShort { min: 8 })
    );
    let password = Password::parse("correct horse").unwrap();
    assert_eq!(format!("{password:?}"), "Password(***)");
}

#[test]
fn note_id_parses_only_positive_numbers() {
    assert_eq!("42".parse::<NoteId>().map(NoteId::get), Ok(42));
    assert_eq!("0".parse::<NoteId>(), Err(IdError::NotPositive));
    assert_eq!("-3".parse::<NoteId>(), Err(IdError::NotPositive));
    assert_eq!("abc".parse::<NoteId>(), Err(IdError::NotANumber));
}

#[test]
fn note_query_defaults_and_offset() {
    let query = NoteQuery::new(Some("  "), None);
    assert_eq!(query.keyword, None);
    assert_eq!(query.page, 1);
    assert_eq!(query.offset(), 0);

    let query = NoteQuery::new(Some(" rust "), Some(3));
    assert_eq!(query.keyword.as_deref(), Some("rust"));
    assert_eq!(query.offset(), 40);
```

```
        // page=0 は 1 として扱う
        assert_eq!(NoteQuery::new(None, Some(0)).page, 1);
    }

    #[test]
    fn note_page_counts_pages() {
        let page = |total_count, page| NotePage {
            notes: Vec::new(),
            page,
            total_count,
        };
        assert_eq!(page(0, 1).total_pages(), 1);
        assert_eq!(page(20, 1).total_pages(), 1);
        assert_eq!(page(21, 1).total_pages(), 2);
        assert!(page(21, 1).has_next());
        assert!(!page(21, 2).has_next());
        assert!(page(21, 2).has_prev());
    }
}
```

▼ コードを1つずつ分解して解説

解説1: 中身が private な「検査済みの型」

- `pub struct NoteTitle(String);` の `String` には `pub` が付いていません。型の名前は公開されていますが、**中身は `domain` の中からしか触れません**。だから、ほかのファイルでは `NoteTitle(…)` と書いて作ることも、`.0` で中身を書き換えることもできず、`NoteTitle::parse` で作り、`as_str()` で読むしかありません（エラー1）。
- こうすると、「`NoteTitle` 型の値がある」なら「1〜80 文字の検査に合格した」が、必ず成り立ちます。ルートやリポジトリは、`NoteTitle` を受け取るだけで、測り直しません（第9巻 UNIT 03 の「不変条件を型の検査で守る」）。
- `UserId` と `NoteId` は、中身はどちらも `i64` ですが、別の型です。ユーザーの ID を、ノートの ID の代わりに渡すと、コンパイルエラーになります（エラー2）。
- `NoteId` は `FromStr` を実装しているので、`"42".parse::<NoteId>()` と書けます。Topcoat の `module_param!` も、この `FromStr` で、URL の `/notes/{id}` の `{id}` を `NoteId` に変えます（UNIT 08）。

解説2: 数え方と、そろえ方

型	そろえ方	数え方と上限	Debug と Display
`NoteTitle`	前後の空白を除く（`trim`）	文字数（`chars().count()`）で1〜80	どちらもあり
`NoteBody`	`\r\n` を `\n` にそろえる。trim しない	文字数で 0〜10,000（空でよい）	Debug だけ
`Email`	trim して、小文字にそろえる	バイト数（`len()`）で 254 まで。`@` が1つ・ドメインに `.` など	どちらもあり
`Password`	そろえない（空白も、利用者が決めたパスワードの一部）	文字数で 8〜128	Debug は `Password(***)`。Display は無い
`NoteId`・`UserId`	なし	1 以上の `i64`	どちらもあり

- タイトルを `len()`（バイト数）で数えると、日本語は1文字が3バイトなので、27 文字で 80 を超えてしまいます。だから `chars().count()` で数えます（テスト `note_title_counts_characters_not_bytes`）。
- ブラウザのフォームは、改行を `\r\n` で送ってきます。本文は `\n` にそろえてから数えるので、改行を1文字と数えます。
- メールは、同じ人を同じ値にするために、小文字にそろえます。`Alice@Example.COM` と `alice@example.com` は、同じユーザーです。
- `Password` の `Debug` は、手で書いて `Password(***)` とだけ表示します。`#[derive(Debug)]` のままだと、`{:?}` でログに出したときに、パスワードがそのまま書かれてしまうからです（UNIT 03 の `Secret` と同じ考え方）。

解説3: NewNote::parse は、問題を全部集める

- `match (NoteTitle::parse(title), NoteBody::parse(body))` で、2つを**両方**検査してから、結果の組み合わせを見ます。両方 `Ok` なら `NewNote`、どちらかが `Err` なら、`NoteInputErrors` に、それぞれの問題（無ければ `None`）を入れて返します。
- こうすると、「タイトルが空」と「本文が長すぎる」を、一度に画面に出せます（問1で、`?` で書いた版と比べます）。

解説4: 一覧の検索の条件とページ送り

- `NoteQuery::new` は、URL の `?q=` と `?page=` から作ります。キーワードは trim して、空なら `None`（全部表示）、長ければ先頭の 100 文字だけにします。ページは、無ければ 1、0 なら 1 にそろえます（`max(1)`）。
- `offset` は、SQL の `OFFSET`（何件読み飛ばすか）で、`(page - 1) × 20` です。掛け算は `u64` でするので、大きなページの番号でも、あふれません。
- `NotePage::total_pages` は、`div_ceil`（割って切り上げ）で、全部で何ページあるかを数えます。0 件でも 1 ページです。`has_prev` と `has_next` は、「前へ」「次へ」のリンクを出すかどうかに使います（UNIT 07 の `pagination`）。

- **注意:** `NoteQuery` の項目（ `keyword` ・ `page` ）は `pub` です。ファイルの先頭のコメントには「検査を通らずに作る方法はない」とありますが、`NoteQuery` だけは、`new` を通らずに作れます（問2・第11巻 UNIT 04）。

例4-2: src/error.rs の全文

> **▼ このファイルがやること（先に日本語で）:** 失敗を2つの層に分けます。`RepoError` は DB の層の失敗（一意制約の違反・version の食い違い・見つからない・値を読めない・DB の故障）です。`AppError` は、利用者への返事を決めるための分類で、それぞれが HTTP の状態コードに対応します。`RepoError` を `AppError` に変える決まり（ `From` ）と、状態コード（ `status_code` ）、画面に出してよい文（ `public_message` ）も、ここにあります。

```rust
//! エラーの分類。
//!
//! - `RepoError` … DB の層で起きる失敗（一意制約違反・version の食い違い・接続の失敗など）
//! - `AppError`  … 利用者への返事を決めるための分類。HTTP のステータスコードに対応する
//!
//! | AppError | ステータス | 起きる場面 |
//! |---|---|---|
//! | `InvalidTitle` など入力の検証エラー | 400 | 空のタイトル、形式の違うメール |
//! | `Unauthorized` | 401 | ログインしていない（通常はログイン画面へのリダイレクトで扱う） |
//! | `NotFound` | 404 | 存在しないノート、**他人のノート**（あること自体を隠す） |
//! | `EmailTaken` / `Conflict` | 409 | 登録済みのメール、ほかの画面で先に更新された |
//! | `Internal` | 500 | DB の故障など。画面には詳細を出さず、ログに原因を書く |

use thiserror::Error;
use topcoat::router::StatusCode;

use crate::domain::{BodyError, EmailError, PasswordError, TitleError};

/// いろいろな種類のエラーをまとめて入れる箱（Internal の中身に使う）
pub type BoxError = Box<dyn std::error::Error + Send + Sync>;

// ---------------------------------------------------------------------------
// DB の層のエラー
// ---------------------------------------------------------------------------

#[derive(Debug, Error)]
pub enum RepoError {
    #[error("一意制約に違反しました: {0}")]
    UniqueViolation(String),
    #[error("version が一致しません（ほかの更新が先に保存されました）")]
    VersionMismatch,
    #[error("対象のデータが見つかりません")]
    NotFound,
    #[error("DB の値を読み取れません: {0}")]
    InvalidData(String),
    #[error("DB の操作に失敗しました")]
    Database(#[source] libsql::Error),
}

// libsql のエラーを RepoError に変換する。
// 「UNIQUE constraint failed」はローカル（SQLite）でも Turso でも同じ文言なので、文言で見分ける。
impl From<libsql::Error> for RepoError {
    fn from(error: libsql::Error) -> Self {
        let message = error.to_string();
        if message.contains("UNIQUE constraint failed") {
            RepoError::UniqueViolation(message)
        } else {
            RepoError::Database(error)
```

```
        }
    }
}

// ---------------------------------------------------------------------------
// アプリの層のエラー
// ---------------------------------------------------------------------------

#[derive(Debug, Error)]
pub enum AppError {
    #[error(transparent)]
    InvalidTitle(#[from] TitleError),
    #[error(transparent)]
    InvalidBody(#[from] BodyError),
    #[error(transparent)]
    InvalidEmail(#[from] EmailError),
    #[error(transparent)]
    InvalidPassword(#[from] PasswordError),
    #[error("ログインしてください")]
    Unauthorized,
    #[error("ページが見つかりません")]
    NotFound,
    #[error("そのメールアドレスは既に登録されています")]
    EmailTaken,
    #[error("ほかの画面で先に更新されました。読み込み直してください")]
    Conflict,
    #[error("サーバーの中でエラーが起きました")]
    Internal(#[source] BoxError),
}

// DB の失敗を、利用者への返事の分類に変換する
impl From<RepoError> for AppError {
    fn from(error: RepoError) -> Self {
        match error {
            RepoError::VersionMismatch => AppError::Conflict,
            RepoError::NotFound => AppError::NotFound,
            other => AppError::Internal(Box::new(other)),
        }
    }
}

impl AppError {
    /// 想定外の失敗を Internal に包む
    pub fn internal(error: impl Into<BoxError>) -> AppError {
        AppError::Internal(error.into())
    }

    /// HTTP のステータスコード
    pub fn status_code(&self) -> StatusCode {
        match self {
```

```
            AppError::InvalidTitle(_)
            | AppError::InvalidBody(_)
            | AppError::InvalidEmail(_)
            | AppError::InvalidPassword(_) => StatusCode::BAD_REQUEST,
            AppError::Unauthorized => StatusCode::UNAUTHORIZED,
            AppError::NotFound => StatusCode::NOT_FOUND,
            AppError::EmailTaken | AppError::Conflict => StatusCode::CONFLICT,
            AppError::Internal(_) => StatusCode::INTERNAL_SERVER_ERROR,
        }
    }

    /// 画面に出してよい文章（500 のときは中身を隠す）
    pub fn public_message(&self) -> String {
        match self {
            AppError::Internal(_) => "時間をおいてもう一度お試しください".to_string(),
            other => other.to_string(),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn repo_errors_map_to_app_errors() {
        assert!(matches!(
            AppError::from(RepoError::VersionMismatch),
            AppError::Conflict
        ));
        assert!(matches!(
            AppError::from(RepoError::NotFound),
            AppError::NotFound
        ));
        assert!(matches!(
            AppError::from(RepoError::InvalidData("x".to_string())),
            AppError::Internal(_)
        ));
    }

    #[test]
    fn status_codes_and_public_messages() {
        assert_eq!(
            AppError::from(TitleError::Empty).status_code(),
            StatusCode::BAD_REQUEST
        );
        assert_eq!(AppError::NotFound.status_code(), StatusCode::NOT_FOUND);
        assert_eq!(AppError::Conflict.status_code(), StatusCode::CONFLICT);
        let internal = AppError::internal("disk full");
        assert_eq!(internal.status_code(), StatusCode::INTERNAL_SERVER_ERROR);
```

```
        // 500 のときは原因（disk full）を利用者に見せない
        assert!(!internal.public_message().contains("disk full"));
    }
}
```

解説5: 2つの層と、From による変換

- `RepoError`（**DB の層**）: libsql のエラーは、`From<libsql::Error>` で `RepoError` に変わります。文に `UNIQUE constraint failed` があれば `UniqueViolation`、それ以外は `Database` です。この文は、手元の SQLite でも Turso でも同じなので、文で見分けています。`Database(#[source] libsql::Error)` の `#[source]` は、元のエラーを「原因」として持っておく印で、ログに原因まで書けます。
- `AppError`（**返事の分類**）: `InvalidTitle(#[from] TitleError)` の `#[from]` は、`From<TitleError> for AppError` を作ります。だから、`AppError` を返す関数の中では、`NoteTitle::parse(…)?` と `?` を付けるだけで、`TitleError` が `AppError::InvalidTitle` に変わります（`#[from]` が無い型に `?` を付けると、エラー3）。`#[error(transparent)]` は、「文は、中の `TitleError` の文をそのまま使う」という意味です。
- `From<RepoError> for AppError`: `VersionMismatch` は `Conflict`（409）、`NotFound` は `NotFound`（404）、それ以外は全部 `Internal`（500）です。`UniqueViolation` も、ここでは `Internal` になります。登録済みのメールを 409 にするのは、`src/routes/signup.rs` が、変換する**前**に `UniqueViolation` を見分けているからです（UNIT 07）。

```
// src/routes/signup.rs の register から（抜き出し）
let user = match app_state(cx).users.create(&email, &password_hash).await {
    Ok(user) => user,
    // users.email の UNIQUE に引っかかった ＝ 登録済みのメール
    Err(RepoError::UniqueViolation(_)) => {
        form.email_error = Some(AppError::EmailTaken.to_string());
        return Ok(SignupOutcome::Rejected(StatusCode::CONFLICT, form));
    }
    Err(other) => return Err(other.into()),
};
```

解説6: status_code と public_message

- `status_code` の `match` には、`_`（そのほか全部）が**ありません**。`AppError` に種類を足すと、ここがコンパイルエラーになり、「新しい種類は、何番の状態コードか」を決めるまで、先へ進めません（問3）。
- `public_message` は、`Internal` だけ「時間をおいてもう一度お試しください」にして、ほかは `#[error("…")]` の文をそのまま返します。500 の原因（DB の故障の文など）は、画面には出さず、ログにだけ書きます（第11巻 UNIT 07）。
- `AppError::internal(…)` は、`Into<BoxError>` にできる物なら何でも `Internal` に包みます。`&str`（`"disk full"`）も、`std::error::Error` を実装した型も渡せます。

もう一歩: 実務に近い形

例4-3: いろいろな入力で確かめる

> ▼ **このコードがやること（先に日本語で）**: 検査済みの型の `parse` に、ふつうの入力と、まぎらわしい入力（NUL・絵文字・全角の文字・前後の空白・とても大きな数）を渡して、結果を表示します。最後に、いろいろな失敗を `AppError` に変えて、状態コードと、画面に出す文を表示します。

```rust
//! 第13巻 UNIT 04: 検査済みの型の parse と、エラーの分類を、いろいろな入力で確かめる。
//! 実行: cargo run -q --example u04_types

use rustnote::domain::{Email, NewNote, NoteBody, NoteId, NoteQuery, NoteTitle,
Password};
use rustnote::error::{AppError, RepoError};

fn main() {
    println!("--- NoteTitle::parse");
    for input in ["  買い物  ", "", "   ", "\t買い物\t", "\0"] {
        println!("{input:?} → {:?}", NoteTitle::parse(input));
    }
    for (name, input) in [
        ("a×80", "a".repeat(80)),
        ("a×81", "a".repeat(81)),
        ("😀×80", "😀".repeat(80)),
    ] {
        println!(
            "{name} → {:?}",
            NoteTitle::parse(&input).map(|t| t.as_str().chars().count())
        );
    }

    println!("--- NoteBody::parse");
    println!("{:?}", NoteBody::parse("1行目\r\n2行目"));
    println!(
        "10001 文字 → {:?}",
        NoteBody::parse(&"x".repeat(10_001)).map(|_| ())
    );

    println!("--- NewNote::parse（両方まちがい）");
    println!("{:?}", NewNote::parse("", &"x".repeat(10_001)));

    println!("--- Email::parse");
    for input in [
        "  Alice@Example.COM ",
        "alice",
        "@example.com",
        "alice@example",
        "a@b@c.com",
        "ａｌｉｃｅ@example.com",
    ] {
        println!("{input:?} → {:?}", Email::parse(input));
    }

    println!("--- Password::parse");
    for input in [
        "short",
        "correct horse battery",
```

```
        " 空白も 文字 ",
        "あいうえおかきく",
    ] {
        println!("{input:?} → {:?}", Password::parse(input));
    }

    println!("--- NoteId（URL の {{id}}）");
    for input in ["42", "0", "-3", "abc", "01", " 1", "9223372036854775808"] {
        println!("{input:?} → {:?}", input.parse::<NoteId>());
    }

    println!("--- NoteQuery::new");
    let long = "あ".repeat(150);
    for (keyword, page) in [
        (Some("  rust "), Some(3)),
        (Some("   "), Some(0)),
        (Some(long.as_str()), None),
    ] {
        let query = NoteQuery::new(keyword, page);
        let shown = query.keyword.as_ref().map(|k| k.chars().count());
        println!(
            "キーワードの文字数 {shown:?}・page {}・offset {}",
            query.page,
            query.offset()
        );
    }

    println!("--- AppError の分類（状態コード・画面に出す文）");
    let errors = [
        AppError::from(NoteTitle::parse("").unwrap_err()),
        AppError::from(RepoError::NotFound),
        AppError::from(RepoError::VersionMismatch),
        AppError::from(RepoError::UniqueViolation(
            "UNIQUE constraint failed: users.email".to_string(),
        )),
        AppError::EmailTaken,
        AppError::internal("disk full"),
    ];
    for error in &errors {
        println!(
            "{:?} → {}・「{}」",
            error_name(error),
            error.status_code(),
            error.public_message()
        );
    }
}

/// 種類の名前だけを取り出す（中身は表示しない）
fn error_name(error: &AppError) -> &'static str {
```

```
    match error {
        AppError::InvalidTitle(_) => "InvalidTitle",
        AppError::InvalidBody(_) => "InvalidBody",
        AppError::InvalidEmail(_) => "InvalidEmail",
        AppError::InvalidPassword(_) => "InvalidPassword",
        AppError::Unauthorized => "Unauthorized",
        AppError::NotFound => "NotFound",
        AppError::EmailTaken => "EmailTaken",
        AppError::Conflict => "Conflict",
        AppError::Internal(_) => "Internal",
    }
}
```

```
cargo run -q --example u04_types
```

```
--- NoteTitle::parse
"  買い物  " → Ok(NoteTitle("買い物"))
"" → Err(Empty)
"   " → Err(Empty)
"\t買い物\t" → Ok(NoteTitle("買い物"))
"\0" → Ok(NoteTitle("\0"))
a×80 → Ok(80)
a×81 → Err(TooLong { len: 81, max: 80 })
😜×80 → Ok(80)
--- NoteBody::parse
Ok(NoteBody("1行目\n2行目"))
10001 文字 → Err(TooLong { len: 10001, max: 10000 })
--- NewNote::parse（両方まちがい）
Err(NoteInputErrors { title: Some(Empty), body: Some(TooLong { len: 10001, max: 10000
}) })
--- Email::parse
"  Alice@Example.COM " → Ok(Email("alice@example.com"))
"alice" → Err(MissingAt)
"@example.com" → Err(Malformed)
"alice@example" → Err(Malformed)
"a@b@c.com" → Err(Malformed)
"ａｌｉｃｅ@example.com" → Ok(Email("ａｌｉｃｅ@example.com"))
--- Password::parse
"short" → Err(TooShort { min: 8 })
"correct horse battery" → Ok(Password(***))
" 空白も 文字 " → Ok(Password(***))
"あいうえおかきく" → Ok(Password(***))
--- NoteId（URL の {id}）
"42" → Ok(NoteId(42))
"0" → Err(NotPositive)
"-3" → Err(NotPositive)
"abc" → Err(NotANumber)
"01" → Ok(NoteId(1))
" 1" → Err(NotANumber)
"9223372036854775808" → Err(NotANumber)
--- NoteQuery::new
キーワードの文字数 Some(4)・page 3・offset 40
キーワードの文字数 None・page 1・offset 0
キーワードの文字数 Some(100)・page 1・offset 0
--- AppError の分類（状態コード・画面に出す文）
"InvalidTitle" → 400 Bad Request・「タイトルを入力してください」
"NotFound" → 404 Not Found・「ページが見つかりません」
"Conflict" → 409 Conflict・「ほかの画面で先に更新されました。読み込み直してください」
"Internal" → 500 Internal Server Error・「時間をおいてもう一度お試しください」
"EmailTaken" → 409 Conflict・「そのメールアドレスは既に登録されています」
"Internal" → 500 Internal Server Error・「時間をおいてもう一度お試しください」
```

- **NUL の題（`"\0"`）が合格しました。** `NoteTitle::parse` は、NUL（番号 0 の文字）も1文字と数えます。ところが、DB の `CHECK (length(title) BETWEEN 1 AND 80)` の SQLite の `length()` は、NUL より前しか数えないので、この題は DB で断られて 500 になります（UNIT 02 の注意・第11巻 UNIT 04）。`MAX_TITLE_CHARS` のコメントの「SQLite の `length()` と同じ」は、NUL では成り立ちません（改善点）。
- **絵文字 80 個（`😀×80`）も合格しました。** `chars().count()` は、絵文字1つを1文字と数えます。ブラウザの入力欄の `maxlength="80"` は UTF-16 の単位で数えるので、ブラウザでは 40 個までしか入りません（第11巻 UNIT 05）。
- **全角の `ａｌｉｃｅ@example.com` も合格しました。** `to_lowercase` は大文字と小文字をそろえるだけで、全角を半角にはしません。`alice@example.com` とは別のユーザーになります。
- パスワードは、前後の空白もそのまま残ります（`Password(***)` と隠れて表示）。日本語の 8 文字も合格です。
- `NoteId` は、`"01"` を 1 と読み、前に空白がある `" 1"` と、`i64` に入らない `9223372036854775808` は `NotANumber` でした。
- `NoteQuery::new` は、150 文字のキーワードを 100 文字に切り、空白だけのキーワードを `None` に、`page` の 0 を 1 にそろえました。
- 失敗の分類では、`UniqueViolation` が、`From` では `Internal`（500）になりました（解説5）。`Internal` は、`disk full` のような原因を、画面の文に出しません。

よくあるエラーと読み方

エラー1: 検査を通らずに、`NoteId` を直接作ろうとした（E0423）

```
//! 第13巻 UNIT 04 エラー1: 検査を通らずに、NoteId を直接作ろうとした。
//! 実行: cargo run -q --example e04_private

use rustnote::domain::NoteId;

fn main() {
    let id = NoteId(0);
    println!("{id}");
}
```

```
error[E0423]: cannot initialize a tuple struct which contains private fields
  --> examples\e04_private.rs:7:14
   |
 7 |     let id = NoteId(0);
   |              ^^^^^^
   |
   = note: a struct named `NoteId` exists in another namespace
note: constructor is not visible here due to private fields
  --> src\domain.rs:69:19
   |
69 | pub struct NoteId(i64);
   |                   ^^^ private field

For more information about this error, try `rustc --explain E0423`.
error: could not compile `rustnote` (example "e04_private") due to 1 previous error
```

- **何が起きたか:** 「private な項目を含むタプル構造体は、作れません」というエラーです。`note:` が、`src/domain.rs` の `pub struct NoteId(i64);` の `i64` を指して、`private field` と教えています。
- **読み方:** 中身が private なのは、**わざと**です。`NoteId(0)` のような、検査に通らない値を作らせないためです。
- **直し方:** `NoteId::new(5)` か `"5".parse::<NoteId>()` で作り、`Result` を受け取ります。

エラー2: ユーザーの ID を、ノートの ID の代わりに渡した（E0308）

```rust
//! 第13巻 UNIT 04 エラー2: ユーザーの ID を、ノートの ID の代わりに渡した。
//! 実行: cargo run -q --example e04_mixid

use rustnote::domain::{NoteId, UserId};

fn show_note(id: NoteId) {
    println!("ノート {id} を表示します");
}

fn main() {
    let user_id = UserId::new(7).unwrap();
    show_note(user_id);
}
```

```
error[E0308]: mismatched types
  --> examples\e04_mixid.rs:12:15
   |
12 |     show_note(user_id);
   |     --------- ^^^^^^^ expected `NoteId`, found `UserId`
   |     |
   |     arguments to this function are incorrect
   |
note: function defined here
  --> examples\e04_mixid.rs:6:4
   |
 6 | fn show_note(id: NoteId) {
   |    ^^^^^^^^^ ----------

For more information about this error, try `rustc --explain E0308`.
error: could not compile `rustnote` (example "e04_mixid") due to 1 previous error
```

- **何が起きたか**: 「型が合いません。`NoteId` のはずが、`UserId` でした」というエラーです。
- **読み方**: 中身がどちらも `i64` でも、型が違えば渡せません。ID を全部 `i64` のままにしていたら、このまちがいはコンパイルを通り、「ユーザー 7 番の ID で、ノート 7 番を表示する」という、見つけにくいバグになります。
- **直し方**: 渡す値を、正しい型（ノートの ID）にします。

エラー3: `AppError` を返す関数の中で、数の変換の失敗に `?` を付けた（E0277）

```rust
//! 第13巻 UNIT 04 エラー3: AppError を返す関数の中で、数の変換の失敗に ? を付けた。
//! 実行: cargo run -q --example e04_question

use rustnote::error::AppError;

fn parse_page(input: &str) -> Result<u32, AppError> {
    let page: u32 = input.parse()?;
    Ok(page)
}

fn main() {
    println!("{:?}", parse_page("3").map_err(|error| error.to_string()));
}
```

```
error[E0277]: `?` couldn't convert the error to `AppError`
 --> examples\e04_question.rs:7:34
  |
6 | fn parse_page(input: &str) -> Result<u32, AppError> {
  |                              --------------------- expected `AppError` because of
this
7 |     let page: u32 = input.parse()?;
  |                           -------^ the trait `From<ParseIntError>` is not
implemented for `AppError`
  |                           |
  |                           this can't be annotated with `?` because it has type
`Result<_, ParseIntError>`
  |
  = note: the question mark operation (`?`) implicitly performs a conversion on the
error value using the `From` trait
  = help: `AppError` implements trait `From<T>`:
            From<BodyError>
            From<EmailError>
            From<PasswordError>
            From<RepoError>
            From<TitleError>

For more information about this error, try `rustc --explain E0277`.
error: could not compile `rustnote` (example "e04_question") due to 1 previous error
```

- **何が起きたか:** 「`?` が、エラーを `AppError` に変換できません」というエラーです。`?` は、エラーを `From` で、関数の返す型のエラーに変えます。`AppError` には `From<ParseIntError>` がありません。
- **読み方:** `help:` に、`AppError` が実装している `From` の一覧（`BodyError`・`EmailError`・`PasswordError`・`RepoError`・`TitleError`）が出ています。`#[from]` を付けた4つと、手で書いた `From<RepoError>` です。
- **直し方:** どの分類にするかを決めて、`map_err` で変えます。たとえば、利用者の入力のまちがいなら、入力のエラーの型を作って `#[from]` を足し、想定外の失敗なら `.map_err(AppError::internal)?` とします。

RustNote ではここで使う

- **入力の検査:** 登録のフォームは `Email::parse`・`Password::parse`（UNIT 07）、ログインの照合は `Email::parse`（UNIT 06 の `src/auth.rs`）、ノートの作成と編集のフォームは `NewNote::parse`（UNIT 08）、URL の `{id}` は `NoteId` の `FromStr`（UNIT 08）を通ります。
- **DB の失敗:** リポジトリは `RepoError` を返します（UNIT 05）。ルートは、それを `?` や `map_err(AppError::from)` で `AppError` に変えます。
- **エラー画面:** `src/views.rs` の `error_page` が、`status_code` と `public_message` で、状態コード付きの画面を作ります。500 のときは、原因をログにだけ書きます（UNIT 07）。

- **テスト:** 2つのファイルのユニットテストは、13 個です（`cargo test --lib` の 19 個のうち、`domain::` と `error::` の分）。

```
cargo test --lib -- domain:: error::
```

```
running 13 tests
test domain::tests::email_is_normalized ... ok
test domain::tests::email_rejects_bad_forms ... ok
test domain::tests::new_note_collects_all_errors ... ok
test domain::tests::note_body_normalizes_newlines_and_limits_length ... ok
test domain::tests::note_id_parses_only_positive_numbers ... ok
test domain::tests::note_page_counts_pages ... ok
test domain::tests::note_title_rejects_empty_and_spaces ... ok
test domain::tests::note_title_counts_characters_not_bytes ... ok
test domain::tests::password_length_is_checked_and_hidden_in_debug ... ok
test domain::tests::note_title_is_trimmed ... ok
test error::tests::repo_errors_map_to_app_errors ... ok
test domain::tests::note_query_defaults_and_offset ... ok
test error::tests::status_codes_and_public_messages ... ok
test result: ok. 13 passed; 0 failed; 0 ignored; 0 measured; 6 filtered out; finished
in 0.00s
```

- **改善点（この単元で見つかったこと）:**
- NUL の題が `NoteTitle::parse` を通り、DB で断られて 500 になる（例4-3）。
- `NoteQuery` の項目が `pub` なので、`page: 0` を作れる（問2）。
- `src/views.rs` の `error_page` は、`RepoError` を、`From<RepoError> for AppError` と**同じ表を、もう一度書いて**分類しています（`NotFound` は 404、`VersionMismatch` は 409、ほかは 500）。表を変えるときは、2か所を直す必要があります。

練習問題

問1: `NewNote::parse` は、なぜ `?` を使わずに、`match` で2つの結果を組み合わせているのでしょうか。`?` で書いた版と、実際に比べてください。

問2: 例4-3では、`NoteQuery::new(…, Some(0))` の `page` は 1 にそろいました。では、`NoteQuery { keyword: None, page: 0 }` と項目を直接書いて作り、`offset()` を呼ぶと、どうなるでしょうか。`cargo run` と、`cargo run --release` の両方で確かめ、どう直せばよいかを考えてください。

問3: `AppError` に、新しい種類 `TooManyNotes`（ノートの数が上限に達した）を足すと、どこでコンパイルエラーになるでしょうか。`public_message` は、どうなるでしょうか。

解答と解説

問1の解答:

```rust
//! 第13巻 UNIT 04 問1: NewNote::parse を ? で書いたら、どう変わるか。
//! 実行: cargo run -q --example u04_q1

use rustnote::domain::{NewNote, NoteBody, NoteTitle};

/// ? で書いた版（最初の失敗で止まる）
fn parse_with_question(title: &str, body: &str) -> Result<NewNote, String> {
    let title = NoteTitle::parse(title).map_err(|error| error.to_string())?;
    let body = NoteBody::parse(body).map_err(|error| error.to_string())?;
    Ok(NewNote { title, body })
}

fn main() {
    let long_body = "x".repeat(10_001);
    println!(
        "? で書いた版: {:?}",
        parse_with_question("", &long_body).map(|_| ())
    );

    let errors = NewNote::parse("", &long_body).unwrap_err();
    println!(
        "教材の版（タイトル）: {:?}",
        errors.title.map(|error| error.to_string())
    );
    println!(
        "教材の版（本文）: {:?}",
        errors.body.map(|error| error.to_string())
    );
}
```

```
? で書いた版: Err("タイトルを入力してください")
教材の版（タイトル）: Some("タイトルを入力してください")
教材の版（本文）: Some("本文は10000文字以内にしてください（現在10001文字）")
```

- `?` で書いた版は、タイトルの問題で止まるので、本文の問題（10,001 文字）は伝わりませんでした。利用者は、タイトルを直して送り直してから、初めて本文の問題を知ります。
- 教材の版は、両方の問題を集めて返しました。作成のフォームは、この2つを、それぞれの入力欄の下に、一度に表示します（UNIT 08）。

問2の解答:

```rust
//! 第13巻 UNIT 04 問2: NoteQuery の項目は pub なので、new を通らずに page: 0 を作れる。
//! 実行: cargo run -q --example u04_q2（--release を付けた場合とも比べる）

use rustnote::domain::NoteQuery;

fn main() {
    let made_by_new = NoteQuery::new(None, Some(0));
    println!(
        "new で作った: page {}・offset {}",
        made_by_new.page,
        made_by_new.offset()
    );

    let made_directly = NoteQuery {
        keyword: None,
        page: 0,
    };
    println!("直接作った: page {}", made_directly.page);
    println!("直接作った: offset {}", made_directly.offset());
}
```

```
new で作った: page 1・offset 0
直接作った: page 0

thread 'main' (225180) panicked at src\domain.rs:370:19:
attempt to subtract with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
new で作った: page 1・offset 0
直接作った: page 0
直接作った: offset 85899345900
```

- `cargo run`（debug）では、`offset` の `self.page - 1`（`u32` の 0 から 1 を引く）が、`attempt to subtract with overflow`（引き算であふれた）でパニックしました。
- `--release` では、あふれの検査が無いので、0 − 1 が `u32` のいちばん大きな数（4,294,967,295）に回り込み、`offset` は 85,899,345,900（その 20 倍）になりました。パニックはしませんが、SQL はそれだけ読み飛ばすので、**エラーも無く、空のページ**になります。
- 今の RustNote は、`NoteQuery` を `NoteQuery::new` でしか作っていないので、どちらも起きません。ただし、**型が、約束（`page` は 1 以上）を守っていません。**
- **直し方**: 項目を `pub` でなくし、`keyword()`・`page()` のような読むためのメソッドを足して、`NoteQuery::new` だけで作れるようにします。項目を直接読んでいる所（`src/repository/notes.rs`・`src/routes/notes.rs`・`src/views.rs`）は、メソッドを呼ぶように直します。教材のコードは、まだ直していません（巻末の総合演習で扱います）。

問3の解答:

`src/error.rs` の `Conflict,` の次に、`#[error("ノートの数が上限に達しました")]` と `TooManyNotes,` を足して、`cargo build` しました。

```
error[E0004]: non-exhaustive patterns: `&AppError::TooManyNotes` not covered
   --> src\error.rs:100:15
    |
100 |         match self {
    |               ^^^^ pattern `&AppError::TooManyNotes` not covered
    |
note: `AppError` defined here
   --> src\error.rs:58:10
    |
 58 | pub enum AppError {
    |          ^^^^^^^^
...
 76 |     TooManyNotes,
    |     ------------ not covered
    = note: the matched value is of type `&AppError`
help: ensure that all possible cases are being handled by adding a match arm with a
wildcard pattern or an explicit pattern as shown
    |
108 ~             AppError::Internal(_) => StatusCode::INTERNAL_SERVER_ERROR,
109 ~             &AppError::TooManyNotes => todo!(),
    |

For more information about this error, try `rustc --explain E0004`.
error: could not compile `rustnote` (lib) due to 1 previous error
```

- コンパイルエラーは、`status_code` の `match`（E0004、「`&AppError::TooManyNotes` が扱われていません」）の1か所でした。`_` が無いので、新しい種類の状態コードを決めるまで、先へ進めません。`help:` は `todo!()` の腕を足す案を出していますが、実際には、409 や 422 など、意味に合う状態コードを書きます。
- `public_message` は、`other => other.to_string()` の腕があるので、エラーになりません。`#[error("…")]` の文（「ノートの数が上限に達しました」）が、そのまま画面に出ます。画面に出してよい文かどうかは、`#[error]` を書くときに決めておきます。
- `_` や `other` を使うと、書く量は減りますが、種類を足したときに、コンパイラが教えてくれなくなります。返事の分類のように、「種類ごとに必ず決めてほしい」所では、`_` を使わずに全部の種類を書きます。

まとめ

- `src/domain.rs` の検査済みの型（`NoteTitle`・`NoteBody`・`Email`・`Password`・`NoteId`・`UserId`）は、中身が private なので、`parse`・`new` を通らずには作れない（E0423）。`UserId` と `NoteId` は取り違えられない（E0308）。

- タイトルは trim して文字数で 1〜80、本文は改行をそろえて 10,000 まで、メールは trim と小文字、パスワードはそろえずに 8〜128 文字で `Debug` を隠す。`NewNote::parse` は問題を全部集める。
- `src/error.rs` は、DB の `RepoError` と、返事の `AppError` の2つの層。`#[from]` と `From` で `?` が変換し（無いと E0277）、`status_code` は `_` 無しの `match`（足し忘れは E0004）、`public_message` は 500 の中身を隠す。
- 改善点: NUL の題が型の検査を通って 500 になる。`NoteQuery` の `pub` な項目で `page: 0` を作ると、debug ではパニック、release では空のページ。`error_page` が分類の表を二重に持つ。

UNIT 05: リポジトリ

この単元のゴール

- `src/repository.rs` が「約束（trait）」と「libSQL の実装」を組にして置き、`AppState` が `Arc<dyn … + Send + Sync>` で持つ理由と、`#[async_trait]` が要る理由を説明できる
- `src/repository/notes.rs` のどの SQL にも、持ち主の条件（`user_id = ?`）が入っていることと、それを消すと何が起きるか（どのテストが止めるか）を、本物の実行で確かめられる
- 楽観的ロック（`version`）の `update` が、0 行のときに `find` で「無い」と「先に更新された」を見分ける流れを説明できる
- LIKE の `%` と `_` を escape する理由を、escape しない検索との比較で示せる
- `users.rs`・`sessions.rs` が、パスワードのハッシュとセッションのトークンのハッシュを、どう保存し、どう探すかを言える

たとえ話でつかむ

図書館の、本の出し入れの窓口を考えます。

- 利用者（ルート）は、**窓口**（trait）に「この本を出してください」と頼むだけで、書庫の中の棚の場所（SQL）は知りません。窓口の係が、だれ（libSQL の実装・テスト用の実装）でも、頼み方は同じです。
- 係は、本を出すとき、必ず**貸出カードの名前**（`user_id`）を確かめます。他人が借りている本を頼まれたら、「ありません」と答えます。「ありますが、他人のです」とは言いません。
- 本には、**版の番号**（`version`）が書いてあります。書き直しを頼むときは、「私が読んだのは第2版です」と伝えます。もう第3版になっていたら、係は書き直さず、「先に別の人が直しました」と答えます（楽観的ロック）。
- 書き直せなかったとき、係は、**もう一度、棚を見て**、「本が無い」のか「版が違う」のかを確かめてから答えます。
- 「`100%` という言葉を含む本」を探すとき、`%` を「何でもよい」の印と取り違えないよう、**ただの文字だと印を付けて**から探します。

図書館	RustNote（`src/repository.rs`・`src/repository/`）
窓口（頼み方）	trait（`NoteRepository`・`UserRepository`・`SessionRepository`）
窓口の係	`LibsqlNoteRepository` など（libSQL で実装したもの）
係を選んで置いておく	`AppState` の `Arc<dyn NoteRepository + Send + Sync>`
貸出カードの名前を確かめる	どの SQL にも `user_id = ?`（持ち主の条件）
版の番号	`notes.version`（更新のたびに 1 増える）
版が違えば書き直さない	`UPDATE … WHERE … AND version = ?`
もう一度、棚を見る	0 行のときの `self.find(owner, id)`
ただの文字だと印を付ける	`escape_like` と `ESCAPE '\'`

> **用語:** リポジトリ（repository：データのしまい方・取り出し方を引き受ける部品。ほかの部品は、SQL を知らずに「保存して」「探して」と頼む）

> **用語:** 楽観的ロック（らっかんてきロック：「同じデータを同時に直す人は、めったにいない」と考えて、先に鍵をかけず、保存するときに版の番号を比べて、食い違いを見つける方法）

図で理解する

ルートから DB までの層と、エラーが戻る道です。

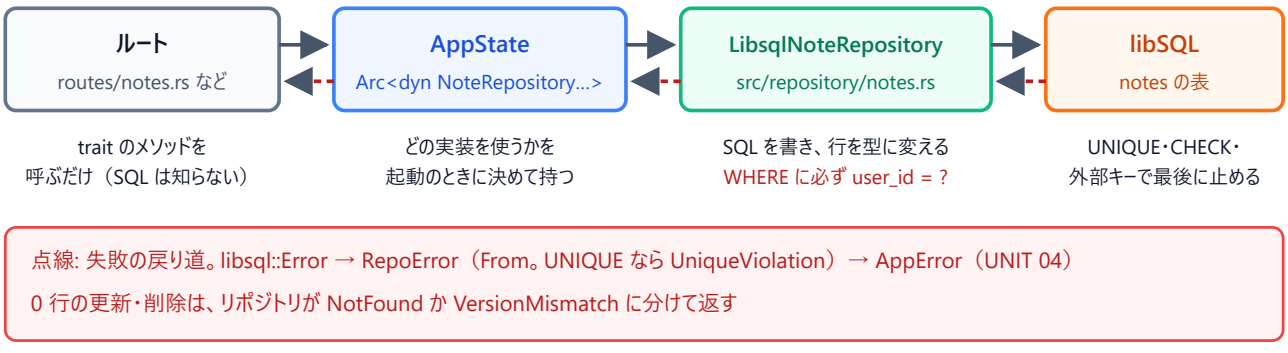


図13-12: リポジトリの層。ルートは trait だけを知り、AppState が実装を持つ。持ち主の条件は、いちばん DB に近い SQL で必ず付ける

`update`（楽観的ロック）が、0 行のときに理由を見分ける流れです（例5-4の結果）。

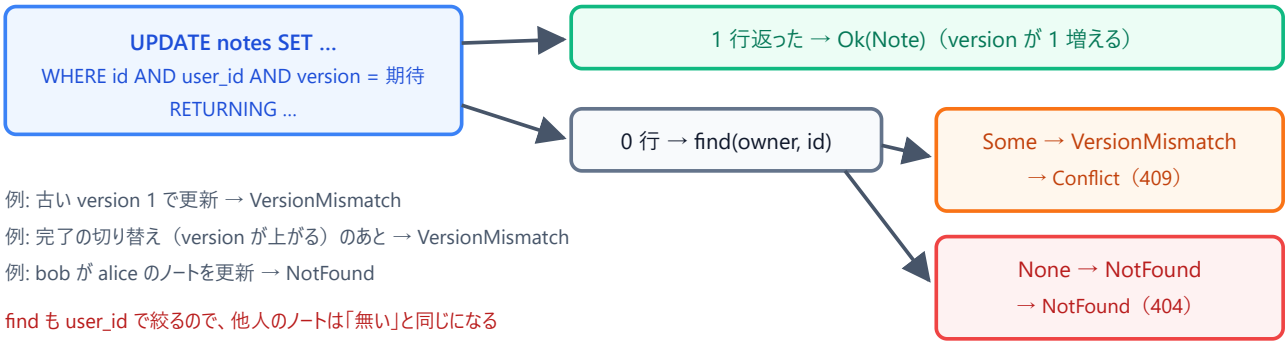


図13-13: 楽観的ロックの update。0 行のときだけ、もう一度 find して、「先に更新された」と「無い（他人のも含む）」を分ける

まず動かす最小コード

例5-1: src/repository.rs の全文

> **▼ このファイルがやること（先に日本語で）**：3つのリポジトリ（ `notes.rs`・`users.rs`・`sessions.rs` ）を部品として読み込み、その trait と実装を `pub use` で外に見せます。時刻を「1970 年からの秒数」に変える `unix_seconds` と、DB から読んだ値の変換の失敗を `InvalidData` にする `invalid_data` もあります。

```rust
//! リポジトリ（データのしまい方・取り出し方を引き受ける部品）。
//!
//! それぞれ「約束（trait）」と「libSQL で実装したもの」をセットで置いている。
//! - notes.rs    … `NoteRepository` / `LibsqlNoteRepository`
//! - users.rs    … `UserRepository` / `LibsqlUserRepository`
//! - sessions.rs … `SessionRepository` / `LibsqlSessionRepository`
//!
//! ページの処理（routes）は trait だけを知っていて、AppState に入った
//! `Arc<dyn NoteRepository + Send + Sync>` を通して使う。
//!
//! DB の操作は待ち時間があるので、メソッドは async fn にしている。
//! 普通の async fn を持つ trait は dyn（実行時に実装を選ぶ形）で使えないため、
//! `#[async_trait]` を付けて dyn で使える形に変換している。

mod notes;
mod sessions;
mod users;

use std::time::{SystemTime, UNIX_EPOCH};

pub use notes::{LibsqlNoteRepository, NoteRepository};
pub use sessions::{LibsqlSessionRepository, SessionRepository};
pub use users::{LibsqlUserRepository, UserRecord, UserRepository};

use crate::error::RepoError;

/// SystemTime を「1970-01-01 からの秒数」（sessions.expires_at の形）に変える
pub fn unix_seconds(time: SystemTime) -> i64 {
    let seconds = time
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_secs())
        .unwrap_or(0);
    i64::try_from(seconds).unwrap_or(i64::MAX)
}

/// DB から読んだ値の変換に失敗したら InvalidData にする
fn invalid_data(what: &str, error: impl std::fmt::Display) -> RepoError {
    RepoError::InvalidData(format!("{what}: {error}"))
}
```

▼ コードを1つずつ分解して解説

解説1: 約束（trait）と実装を組にする

- 3つのファイルは `mod notes;` と、`pub` を付けずに読み込みます。外に見せるのは、`pub use` で選んだ trait・実装・`UserRecord` だけです。外からは `rustnote::repository::NoteRepository` と、1段浅い名前で使えます。

- ルートは trait（`NoteRepository`）のメソッドだけを呼びます。どの実装を使うかは、`src/state.rs` の `AppState::new` が決め、`Arc<dyn NoteRepository + Send + Sync>` で持ちます（UNIT 06）。`Send + Sync` は「ほかのスレッドへ渡しても、同時に使っても安全」という印です。Topcoat の `app_context` は、リクエストを複数のスレッドで同時に処理するので、登録する値に `Send + Sync` を求めます。
- trait のメソッドは、DB を待つので `async fn` です。ところが、`async fn` を持つ trait は、そのままでは `dyn` で使えません（エラー1）。`#[async_trait]` は、`async fn` を「`Box` に入れた Future を返す普通の関数」に書き換えて、`dyn` で使える形にします（第9巻 UNIT 04）。

解説2: 小さな2つの関数

- `unix_seconds` は、`SystemTime` を「1970-01-01 からの秒数」の `i64` にします。`sessions.expires_at` の列の形です。1970 年より前なら 0、`i64` に入らないほど先なら `i64::MAX` にして、パニックしません。
- `invalid_data` は、DB から読んだ値が型の検査に通らなかったとき（たとえば `users.email` が `Email::parse` に通らない）に、どの列かと理由を付けて `RepoError::InvalidData` にします。DB に変な値が入っていても、パニックせずに 500 で返し、ログに理由を残せます。

例5-2: src/repository/notes.rs の全文

> **▼ このファイルがやること（先に日本語で）**：ノートの追加・1件の検索・一覧の検索・更新・完了の切り替え・削除の6つの約束（`NoteRepository`）と、それを libSQL で実装したもの（`LibsqlNoteRepository`）です。どの SQL にも、持ち主の条件（`user_id = ?`）を入れます。更新は、`version` を比べる楽観的ロックです。最後に、LIKE の特別な文字を escape する関数と、そのテストがあります。

```rust
//! ノートの保存・検索・更新・削除。
//!
//! どのメソッドも「持ち主（owner）」を受け取り、SQL の WHERE に必ず
//! `user_id = 持ち主` を入れる。だから他人のノートは「見つからない」扱いになる。

use async_trait::async_trait;
use libsql::Row;

use super::invalid_data;
use crate::db::Db;
use crate::domain::{NewNote, Note, NoteId, NotePage, NoteQuery, PER_PAGE, UserId};
use crate::error::RepoError;

/// ノートの保存先に求める約束
#[async_trait]
pub trait NoteRepository {
    /// ノートを追加し、保存された内容（id や日時つき）を返す
    async fn insert(&self, owner: UserId, note: &NewNote) -> Result<Note, RepoError>;

    /// 自分のノートを1件探す（他人のノート・存在しないノートは None）
    async fn find(&self, owner: UserId, id: NoteId) -> Result<Option<Note>, RepoError>;

    /// 自分のノートを、キーワードとページ番号で検索する（新しい順）
    async fn search(&self, owner: UserId, query: &NoteQuery) -> Result<NotePage,
RepoError>;

    /// タイトルと本文を更新する。
    /// `expected_version` が今の version と違えば RepoError::VersionMismatch（楽観的ロッ
ク）
    async fn update(
        &self,
        owner: UserId,
        id: NoteId,
        note: &NewNote,
        expected_version: i64,
    ) -> Result<Note, RepoError>;

    /// 完了（done）と未完了を切り替える
    async fn toggle_done(&self, owner: UserId, id: NoteId) -> Result<Note, RepoError>;

    /// ノートを削除する（無ければ RepoError::NotFound）
    async fn delete(&self, owner: UserId, id: NoteId) -> Result<(), RepoError>;
}

/// SELECT や RETURNING で読む列（row_to_note の順番と合わせる）
const NOTE_COLUMNS: &str = "id, user_id, title, body, done, version, created_at,
updated_at";

/// libSQL（Turso / ローカルファイル）に保存する実装
```

```rust
pub struct LibsqlNoteRepository {
    db: Db,
}

impl LibsqlNoteRepository {
    pub fn new(db: Db) -> Self {
        Self { db }
    }
}

#[async_trait]
impl NoteRepository for LibsqlNoteRepository {
    async fn insert(&self, owner: UserId, note: &NewNote) -> Result<Note, RepoError> {
        let conn = self.db.connect().await?;
        let sql = format!(
            "INSERT INTO notes (user_id, title, body) VALUES (?1, ?2, ?3) RETURNING
{NOTE_COLUMNS}"
        );
        let mut rows = conn
            .query(
                &sql,
                libsql::params![owner.get(), note.title.as_str(), note.body.as_str()],
            )
            .await?;
        let row = rows
            .next()
            .await?
            .ok_or_else(|| RepoError::InvalidData("INSERT が行を返しませんでし
た".to_string()))?;
        row_to_note(&row)
    }

    async fn find(&self, owner: UserId, id: NoteId) -> Result<Option<Note>, RepoError>
{
        let conn = self.db.connect().await?;
        let sql = format!("SELECT {NOTE_COLUMNS} FROM notes WHERE id = ?1 AND user_id =
?2");
        let mut rows = conn
            .query(&sql, libsql::params![id.get(), owner.get()])
            .await?;
        match rows.next().await? {
            Some(row) => Ok(Some(row_to_note(&row)?)),
            None => Ok(None),
        }
    }

    async fn search(&self, owner: UserId, query: &NoteQuery) -> Result<NotePage,
RepoError> {
        let conn = self.db.connect().await?;
        // キーワードが無いときは「%」（何にでも一致する）で検索する
```

```
        let pattern = match &query.keyword {
            Some(keyword) => format!("%{}%", escape_like(keyword)),
            None => "%".to_string(),
        };
        // LIKE の中の \ を「次の文字をそのまま扱う」という意味にする（ESCAPE '\'）
        let condition = "user_id = ?1 AND (title LIKE ?2 ESCAPE '\\' OR body LIKE ?2
ESCAPE '\\')";

        // 1) 条件に合う件数（ページ数の計算に使う）
        let count_sql = format!("SELECT COUNT(*) FROM notes WHERE {condition}");
        let mut rows = conn
            .query(&count_sql, libsql::params![owner.get(), pattern.as_str()])
            .await?;
        let total_count = match rows.next().await? {
            Some(row) => row.get::<i64>(0)?,
            None => 0,
        };

        // 2) このページに表示する分（新しく更新した順）
        let list_sql = format!(
            "SELECT {NOTE_COLUMNS} FROM notes WHERE {condition}
             ORDER BY updated_at DESC, id DESC
             LIMIT ?3 OFFSET ?4"
        );
        let offset = i64::try_from(query.offset()).unwrap_or(i64::MAX);
        let mut rows = conn
            .query(
                &list_sql,
                libsql::params![owner.get(), pattern.as_str(), i64::from(PER_PAGE),
offset],
            )
            .await?;
        let mut notes = Vec::new();
        while let Some(row) = rows.next().await? {
            notes.push(row_to_note(&row)?);
        }

        Ok(NotePage {
            notes,
            page: query.page,
            total_count: u64::try_from(total_count).unwrap_or(0),
        })
    }

    async fn update(
        &self,
        owner: UserId,
        id: NoteId,
        note: &NewNote,
        expected_version: i64,
```

```
    ) -> Result<Note, RepoError> {
        let conn = self.db.connect().await?;
        // version が一致したときだけ更新し、version を 1 増やす
        let sql = format!(
            "UPDATE notes
                SET title = ?1, body = ?2, version = version + 1, updated_at =
datetime('now')
              WHERE id = ?3 AND user_id = ?4 AND version = ?5
              RETURNING {NOTE_COLUMNS}"
        );
        let mut rows = conn
            .query(
                &sql,
                libsql::params![
                    note.title.as_str(),
                    note.body.as_str(),
                    id.get(),
                    owner.get(),
                    expected_version
                ],
            )
            .await?;
        if let Some(row) = rows.next().await? {
            return row_to_note(&row);
        }
        // 1行も更新されなかった: ノートが無いのか、version が違うのかを見分ける
        match self.find(owner, id).await? {
            Some(_) => Err(RepoError::VersionMismatch),
            None => Err(RepoError::NotFound),
        }
    }

    async fn toggle_done(&self, owner: UserId, id: NoteId) -> Result<Note, RepoError> {
        let conn = self.db.connect().await?;
        // done を 0 ⇔ 1 に反転する。中身が変わるので version も上げる
        let sql = format!(
            "UPDATE notes
                SET done = 1 - done, version = version + 1, updated_at =
datetime('now')
              WHERE id = ?1 AND user_id = ?2
              RETURNING {NOTE_COLUMNS}"
        );
        let mut rows = conn
            .query(&sql, libsql::params![id.get(), owner.get()])
            .await?;
        match rows.next().await? {
            Some(row) => row_to_note(&row),
            None => Err(RepoError::NotFound),
        }
    }
```

```
    async fn delete(&self, owner: UserId, id: NoteId) -> Result<(), RepoError> {
        let conn = self.db.connect().await?;
        let deleted = conn
            .execute(
                "DELETE FROM notes WHERE id = ?1 AND user_id = ?2",
                libsql::params![id.get(), owner.get()],
            )
            .await?;
        if deleted == 0 {
            return Err(RepoError::NotFound);
        }
        Ok(())
    }
}

/// 行を Note に変える（列の順番は NOTE_COLUMNS と同じ）
fn row_to_note(row: &Row) -> Result<Note, RepoError> {
    let id = NoteId::new(row.get::<i64>(0)?).map_err(|e| invalid_data("notes.id", e))?;
    let owner = UserId::new(row.get::<i64>(1)?).map_err(|e|
invalid_data("notes.user_id", e))?;
    Ok(Note {
        id,
        owner,
        title: row.get::<String>(2)?,
        body: row.get::<String>(3)?,
        done: row.get::<i64>(4)? != 0,
        version: row.get::<i64>(5)?,
        created_at: row.get::<String>(6)?,
        updated_at: row.get::<String>(7)?,
    })
}

/// LIKE で特別な意味を持つ文字（% と _ と \）を、ただの文字として扱わせる
fn escape_like(keyword: &str) -> String {
    let mut escaped = String::with_capacity(keyword.len());
    for c in keyword.chars() {
        if matches!(c, '\\' | '%' | '_') {
            escaped.push('\\');
        }
        escaped.push(c);
    }
    escaped
}

#[cfg(test)]
mod tests {
    use super::escape_like;

    #[test]
```

```
    fn like_special_characters_are_escaped() {
        assert_eq!(escape_like("100%"), "100\\%");
        assert_eq!(escape_like("a_b"), "a\\_b");
        assert_eq!(escape_like("c:\\tmp"), "c:\\\\tmp");
        assert_eq!(escape_like("買い物"), "買い物");
    }
}
```

解説3: どの SQL にも、持ち主の条件

- `find`・`update`・`toggle_done`・`delete` の `WHERE` には `user_id = ?`、`search` の条件には `user_id = ?1` が入っています。`insert` は、持ち主を `user_id` の列に書きます。
- 持ち主の条件を SQL に入れておくと、他人のノートは「行が無い」のと同じになります。ルートは、「無い」と「他人の」を区別できないので、どちらも 404 を返し、他人のノートがあること自体を知らせません（UNIT 04 の `NotFound`）。条件を1つ消すと何が起きるかは、問2で確かめます。

解説4: RETURNING と、列の順番

- `insert`・`update`・`toggle_done` は、SQL の終わりに `RETURNING {NOTE_COLUMNS}` を付けています。書き込んだ行（DB が決めた `id`・`created_at`・新しい `version` など）を、その場で受け取れるので、もう一度 `SELECT` しなくて済みます。
- 読む列の並びは、定数の `NOTE_COLUMNS` 1か所に書き、`row_to_note` は、その順番どおりに `row.get(0)`・`row.get(1)` …と読みます。`id` と `user_id` は、`NoteId::new`・`UserId::new` で検査し直し、通らなければ `InvalidData` にします。`title` と `body` は、DB の `CHECK` が守っているので、`String` のまま読みます。

解説5: 一覧の検索は2回の問い合わせ

- 1回目は `COUNT(*)` で、条件に合う総数を数えます（ページ数の計算に使う）。2回目は、`ORDER BY updated_at DESC, id DESC` で新しく更新した順に並べ、`LIMIT 20 OFFSET …` で、そのページの分だけを読みます。
- `updated_at` は、`datetime('now')` の秒までの時刻なので、同じ秒に更新したノートは、`id` の大きい順になります。
- キーワードが無いときは、パターンを `%`（何にでも一致）にして、同じ SQL を使います。キーワードがあれば `%キーワード%` にしますが、その前に `escape_like` で、`%`・`_`・`\` に `\` を付けます。SQL の `ESCAPE '\'` は「`\` の次の文字は、ただの文字」という意味です（問1）。

解説6: 楽観的ロックの update

- `UPDATE … WHERE id = ?3 AND user_id = ?4 AND version = ?5` は、版の番号が、編集の画面を開いたときと同じときだけ書き込み、`version = version + 1` で番号を上げます。
- 1 行も返らなかったときは、理由が2つあります。「ノートが無い（他人のノートも含む）」と「ほかの画面で先に更新された」です。`self.find(owner, id)` でもう一度探し、見つかれば `VersionMismatch`（409）、無ければ `NotFound`（404）にします（2つ目の図・問3）。
- `toggle_done` は、版の番号を比べずに切り替えますが、`version` は上げます。だから、編集の画面を開いたあとに、一覧で完了を切り替えると、編集の保存は `VersionMismatch` になります（例5-4）。

- `delete` は、`execute` が返す「消した行の数」が 0 なら `NotFound` にします。

例5-3: src/repository/users.rs と src/repository/sessions.rs の全文

> **▼ このファイルがやること（先に日本語で）：** `users.rs` は、ユーザーの登録と、メールでの検索です。同じメールの登録は、DB の `UNIQUE` で `UniqueViolation` になります。`sessions.rs` は、ログイン中のセッションの記録・検索・削除・期限切れの掃除です。トークンそのものではなく、トークンの SHA-256（32 バイト）を保存します。

```rust
//! ユーザーの保存と検索。

use async_trait::async_trait;
use libsql::Row;

use super::invalid_data;
use crate::db::Db;
use crate::domain::{Email, User, UserId};
use crate::error::RepoError;

/// ログインの確認に使う、パスワードのハッシュ付きのユーザー
#[derive(Debug, Clone)]
pub struct UserRecord {
    pub user: User,
    /// Argon2id の PHC 文字列（$argon2id$v=19$...）
    pub password_hash: String,
}

/// ユーザーの保存先に求める約束
#[async_trait]
pub trait UserRepository {
    /// ユーザーを登録する。同じメールが既にあれば RepoError::UniqueViolation
    async fn create(&self, email: &Email, password_hash: &str) -> Result<User,
RepoError>;

    /// メールアドレスでユーザーを探す（パスワードのハッシュも返す）
    async fn find_by_email(&self, email: &Email) -> Result<Option<UserRecord>,
RepoError>;
}

/// libSQL（Turso / ローカルファイル）に保存する実装
pub struct LibsqlUserRepository {
    db: Db,
}

impl LibsqlUserRepository {
    pub fn new(db: Db) -> Self {
        Self { db }
    }
}

#[async_trait]
impl UserRepository for LibsqlUserRepository {
    async fn create(&self, email: &Email, password_hash: &str) -> Result<User,
RepoError> {
        let conn = self.db.connect().await?;
        // RETURNING で、追加した行の id をその場で受け取る
        let mut rows = conn
            .query(
```

```
                "INSERT INTO users (email, password_hash) VALUES (?1, ?2) RETURNING id,
email",
                libsql::params![email.as_str(), password_hash],
            )
            .await?;
        let row = rows
            .next()
            .await?
            .ok_or_else(|| RepoError::InvalidData("INSERT が行を返しませんでし
た".to_string()))?;
        row_to_user(&row)
    }

    async fn find_by_email(&self, email: &Email) -> Result<Option<UserRecord>,
RepoError> {
        let conn = self.db.connect().await?;
        let mut rows = conn
            .query(
                "SELECT id, email, password_hash FROM users WHERE email = ?1",
                [email.as_str()],
            )
            .await?;
        match rows.next().await? {
            Some(row) => {
                let user = row_to_user(&row)?;
                let password_hash = row.get::<String>(2)?;
                Ok(Some(UserRecord {
                    user,
                    password_hash,
                }))
            }
            None => Ok(None),
        }
    }
}

/// 行（id, email, ...）を User に変える。sessions.rs からも使う
pub(super) fn row_to_user(row: &Row) -> Result<User, RepoError> {
    let id = UserId::new(row.get::<i64>(0)?).map_err(|e| invalid_data("users.id", e))?;
    let email = Email::parse(&row.get::<String>(1)?).map_err(|e|
invalid_data("users.email", e))?;
    Ok(User { id, email })
}
```

```rust
//! ログイン中のセッションの保存と検索。
//!
//! Topcoat はトークンを発行してクッキーで運ぶだけで、セッションの記録はアプリが持つ。
//! ここではトークンの SHA-256（TokenHash、32 バイト）と、ユーザー・有効期限を保存する。

use std::time::SystemTime;

use async_trait::async_trait;
use topcoat::session::TokenHash;

use super::unix_seconds;
use super::users::row_to_user;
use crate::db::Db;
use crate::domain::{User, UserId};
use crate::error::RepoError;

/// セッションの保存先に求める約束
#[async_trait]
pub trait SessionRepository {
    /// ログインしたときに記録する
    async fn create(
        &self,
        token_hash: &TokenHash,
        user_id: UserId,
        expires_at: SystemTime,
    ) -> Result<(), RepoError>;

    /// トークンのハッシュから、ログイン中のユーザーを探す（期限切れなら None）
    async fn find_user(
        &self,
        token_hash: &TokenHash,
        now: SystemTime,
    ) -> Result<Option<User>, RepoError>;

    /// ログアウトしたときに消す
    async fn delete(&self, token_hash: &TokenHash) -> Result<(), RepoError>;

    /// 期限切れのセッションをまとめて消し、消した数を返す
    async fn delete_expired(&self, now: SystemTime) -> Result<u64, RepoError>;
}

/// libSQL（Turso / ローカルファイル）に保存する実装
pub struct LibsqlSessionRepository {
    db: Db,
}

impl LibsqlSessionRepository {
    pub fn new(db: Db) -> Self {
        Self { db }
```

```
    }
}

#[async_trait]
impl SessionRepository for LibsqlSessionRepository {
    async fn create(
        &self,
        token_hash: &TokenHash,
        user_id: UserId,
        expires_at: SystemTime,
    ) -> Result<(), RepoError> {
        let conn = self.db.connect().await?;
        conn.execute(
            "INSERT INTO sessions (token_hash, user_id, expires_at) VALUES (?1, ?2, ?
3)",
            // TokenHash は [u8; 32] として読める。to_vec() で BLOB として渡す
            libsql::params![token_hash.to_vec(), user_id.get(),
unix_seconds(expires_at)],
        )
        .await?;
        Ok(())
    }

    async fn find_user(
        &self,
        token_hash: &TokenHash,
        now: SystemTime,
    ) -> Result<Option<User>, RepoError> {
        let conn = self.db.connect().await?;
        let mut rows = conn
            .query(
                "SELECT u.id, u.email
                   FROM sessions s
                   JOIN users u ON u.id = s.user_id
                  WHERE s.token_hash = ?1 AND s.expires_at > ?2",
                libsql::params![token_hash.to_vec(), unix_seconds(now)],
            )
            .await?;
        match rows.next().await? {
            Some(row) => Ok(Some(row_to_user(&row)?)),
            None => Ok(None),
        }
    }

    async fn delete(&self, token_hash: &TokenHash) -> Result<(), RepoError> {
        let conn = self.db.connect().await?;
        conn.execute(
            "DELETE FROM sessions WHERE token_hash = ?1",
            [token_hash.to_vec()],
        )
```

```
            .await?;
        Ok(())
    }

    async fn delete_expired(&self, now: SystemTime) -> Result<u64, RepoError> {
        let conn = self.db.connect().await?;
        let deleted = conn
            .execute(
                "DELETE FROM sessions WHERE expires_at <= ?1",
                [unix_seconds(now)],
            )
            .await?;
        Ok(deleted)
    }
}
```

解説7: パスワードのハッシュは、要る所でだけ

- `User`（`id` と `email`）には、パスワードのハッシュが入っていません。ハッシュ付きの `UserRecord` を返すのは、ログインの照合に使う `find_by_email` だけです。ふだん持ち回る `User` には、ハッシュが無いので、うっかり画面やログに出すこともありません。
- `row_to_user` は、`pub(super)`（親の `repository` の中だけで使える）で、`sessions.rs` の `find_user` からも使います。`users.email` も、`Email::parse` で検査し直します。
- `find_by_email` の `[email.as_str()]` のように、値が1つのときは、配列で SQL の値を渡せます。2つ以上なら `libsql::params![…]` を使います。

解説8: セッションは、トークンのハッシュで探す

- Topcoat は、ログインのトークンを発行して、クッキーで運ぶだけです。どのトークンがだれのものかの記録は、アプリが `sessions` の表に持ちます（第8巻）。
- 保存するのは、トークンの SHA-256 の `TokenHash`（32 バイト）を `to_vec()` で `BLOB` にしたものです。DB の中身が漏れても、トークンそのものは分かりません。
- `find_user` は、`sessions` と `users` を `JOIN` し、`expires_at > 今` の行だけを見るので、期限切れのトークンでは、ユーザーが見つかりません。期限切れの行は、`delete_expired` が、ログインのたびにまとめて消します（UNIT 06 の `src/auth.rs`）。

もう一歩: 実務に近い形

例5-4: 3つのリポジトリを、本物の DB で動かす

> **▼ このコードがやること（先に日本語で）**：一時フォルダに DB のファイルを作り、マイグレーションをかけ、本番と同じ `AppState::new` で3つのリポジトリを用意します。そして、ユーザーの登録（同じメールの2回目も）、ノートの追加と持ち主ごとの検索、楽観的ロックの更新、一覧の検索とページ送り、削除、セッションの期限を、順に試して表示します。

```rust
//! 第13巻 UNIT 05: 3つのリポジトリを、一時フォルダの本物の DB で、AppState を通して動かす。
//! 実行: cargo run -q --example u05_repo

use std::time::{Duration, SystemTime};

use rustnote::db::{self, Db};
use rustnote::domain::{Email, NewNote, NoteId, NoteQuery};
use rustnote::error::RepoError;
use rustnote::state::AppState;
use topcoat::session::TokenHash;

/// RepoError の種類の名前（中身は短くして表示する）
fn kind(error: &RepoError) -> String {
    match error {
        RepoError::UniqueViolation(message) => format!("UniqueViolation({message})"),
        RepoError::VersionMismatch => "VersionMismatch".to_string(),
        RepoError::NotFound => "NotFound".to_string(),
        RepoError::InvalidData(message) => format!("InvalidData({message})"),
        RepoError::Database(error) => format!("Database({error})"),
    }
}

/// 正しいと分かっている入力から NewNote を作る（NoteInputErrors は Error を実装していないので ? を使わない）
fn new_note(title: &str, body: &str) -> NewNote {
    NewNote::parse(title, body).expect("正しい入力")
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u05.db")).await?;
    db::migrate(&db).await?;
    // 本番と同じ AppState。中の3つは Arc<dyn …Repository + Send + Sync>
    let state = AppState::new(db);

    println!("--- users");
    let alice_email = Email::parse("alice@example.com")?;
    // 本物は Argon2id のハッシュ（UNIT 06）。ここでは仮の文字列を入れる
    let alice = state.users.create(&alice_email, "$argon2id$dummy").await?;
    println!("登録: {alice:?}");
    let again = state.users.create(&alice_email, "$argon2id$dummy").await;
    println!("同じメールで登録: {:?}", again.map_err(|e| kind(&e)));
    let bob = state
        .users
        .create(&Email::parse("bob@example.com")?, "$argon2id$dummy")
        .await?;
    let record = state.users.find_by_email(&alice_email).await?;
    println!(
```

```
        "メールで探す: {:?}",
        record.map(|r| (r.user.id, r.password_hash))
    );

    println!("--- notes（持ち主ごと） ");
    let note = state
        .notes
        .insert(alice.id, &new_note("買い物", "牛乳"))
        .await?;
    println!(
        "追加: id {}・version {}・done {}",
        note.id, note.version, note.done
    );
    println!(
        "alice が探す: {:?}",
        state.notes.find(alice.id, note.id).await?.map(|n| n.title)
    );
    println!(
        "bob が探す: {:?}",
        state.notes.find(bob.id, note.id).await?.map(|n| n.title)
    );
    let missing = NoteId::new(999)?;
    println!(
        "無い ID を探す: {:?}",
        state.notes.find(alice.id, missing).await?.map(|n| n.title)
    );

    println!("--- update（楽観的ロック） ");
    let edit = new_note("買い物（土曜） ", "牛乳と卵");
    let updated = state.notes.update(alice.id, note.id, &edit, 1).await?;
    println!(
        "version 1 で更新: title {:?}・version {}",
        updated.title, updated.version
    );
    let stale = state.notes.update(alice.id, note.id, &edit, 1).await;
    println!(
        "古い version 1 で更新: {:?}",
        stale.map(|n| n.version).map_err(|e| kind(&e))
    );
    let others = state.notes.update(bob.id, note.id, &edit, 2).await;
    println!(
        "bob が更新: {:?}",
        others.map(|n| n.version).map_err(|e| kind(&e))
    );
    let toggled = state.notes.toggle_done(alice.id, note.id).await?;
    println!(
        "完了に切り替え: done {}・version {}",
        toggled.done, toggled.version
    );
    // 編集の画面を version 2 のときに開いていた、という場面
```

```
    let after_toggle = state.notes.update(alice.id, note.id, &edit, 2).await;
    println!(
        "切り替えのあと、version 2 で更新: {:?}",
        after_toggle.map(|n| n.version).map_err(|e| kind(&e))
    );

    println!("--- search（キーワードとページ送り）");
    for i in 1..=23 {
        state
            .notes
            .insert(alice.id, &new_note(&format!("メモ {i}"), ""))
            .await?;
    }
    state
        .notes
        .insert(alice.id, &new_note("セール", "100%オフ"))
        .await?;
    state
        .notes
        .insert(alice.id, &new_note("値段", "1000円"))
        .await?;
    for (keyword, page) in [
        (None, None),
        (None, Some(2)),
        (Some("100%"), None),
        (Some("100"), None),
    ] {
        let found = state
            .notes
            .search(alice.id, &NoteQuery::new(keyword, page))
            .await?;
        let titles: Vec<&str> = found
            .notes
            .iter()
            .take(3)
            .map(|n| n.title.as_str())
            .collect();
        println!(
            "q={keyword:?} page={page:?} → 総数 {}・このページ {} 件・{}/{} ページ・先頭
{titles:?}",
            found.total_count,
            found.notes.len(),
            found.page,
            found.total_pages()
        );
    }
    let bob_page = state
        .notes
        .search(bob.id, &NoteQuery::new(None, None))
        .await?;
```

```
    println!("bob の一覧: 総数 {}", bob_page.total_count);

    println!("--- delete");
    println!(
        "1回目: {:?}",
        state
            .notes
            .delete(alice.id, note.id)
            .await
            .map_err(|e| kind(&e))
    );
    println!(
        "2回目: {:?}",
        state
            .notes
            .delete(alice.id, note.id)
            .await
            .map_err(|e| kind(&e))
    );

    println!("--- sessions");
    let now = SystemTime::now();
    let hour = Duration::from_secs(3600);
    let active = TokenHash::new([1; 32]);
    let expired = TokenHash::new([2; 32]);
    state.sessions.create(&active, alice.id, now + hour).await?;
    state.sessions.create(&expired, bob.id, now - hour).await?;
    let user = state.sessions.find_user(&active, now).await?;
    println!("期限内のトークン: {:?}", user.map(|u| u.email.to_string()));
    let user = state.sessions.find_user(&expired, now).await?;
    println!(
        "期限切れのトークン: {:?}",
        user.map(|u| u.email.to_string())
    );
    println!(
        "期限切れの掃除: {} 件",
        state.sessions.delete_expired(now).await?
    );
    state.sessions.delete(&active).await?;
    let user = state.sessions.find_user(&active, now).await?;
    println!("ログアウトのあと: {:?}", user.map(|u| u.email.to_string()));
    Ok(())
}
```

```
cargo run -q --example u05_repo
```

```
--- users
登録: User { id: UserId(1), email: Email("alice@example.com") }
同じメールで登録: Err("UniqueViolation(SQLite failure: `UNIQUE constraint failed:
users.email`)")
メールで探す: Some((UserId(1), "$argon2id$dummy"))
--- notes（持ち主ごと）
追加: id 1・version 1・done false
alice が探す: Some("買い物")
bob が探す: None
無い ID を探す: None
--- update（楽観的ロック）
version 1 で更新: title "買い物（土曜）"・version 2
古い version 1 で更新: Err("VersionMismatch")
bob が更新: Err("NotFound")
完了に切り替え: done true・version 3
切り替えのあと、version 2 で更新: Err("VersionMismatch")
--- search（キーワードとページ送り）
q=None page=None → 総数 26・このページ 20 件・1/2 ページ・先頭 ["値段", "セール", "メモ 23"]
q=None page=Some(2) → 総数 26・このページ 6 件・2/2 ページ・先頭 ["メモ 5", "メモ 4", "メモ
3"]
q=Some("100%") page=None → 総数 1・このページ 1 件・1/1 ページ・先頭 ["セール"]
q=Some("100") page=None → 総数 2・このページ 2 件・1/1 ページ・先頭 ["値段", "セール"]
bob の一覧: 総数 0
--- delete
1回目: Ok(())
2回目: Err("NotFound")
--- sessions
期限内のトークン: Some("alice@example.com")
期限切れのトークン: None
期限切れの掃除: 1 件
ログアウトのあと: None
```

- 同じメールの2回目は、`UniqueViolation` でした。中の文は、SQLite の `UNIQUE constraint failed: users.email` です（UNIT 04 の `From<libsql::Error>` が、この文で見分けます）。
- bob が alice のノートを探すと `None`、無い ID（999）も `None` で、**同じ答え**でした。更新も、bob は `NotFound` です。
- 古い `version 1` での更新と、完了の切り替え（version が 3 に上がる）のあとの `version 2` での更新は、どちらも `VersionMismatch` でした。
- 26 件のノートは、20 件と 6 件の2ページに分かれました。キーワード `100%` は「100%オフ」の1件だけ、`100` は「1000円」も含めて2件でした。bob の一覧は 0 件です。
- 期限切れのトークンではユーザーが見つからず、掃除で1件消え、ログアウト（`delete`）のあとは、期限内のトークンでも見つかりませんでした。

よくあるエラーと読み方

エラー1: `#[async_trait]` を付けずに、`async fn` を持つ trait を `dyn` で使った（E0038）

```
//! 第13巻 UNIT 05 エラー1: #[async_trait] を付けずに、async fn を持つ trait を dyn で使おうとした。
//! 実行: cargo run -q --example e05_dyn

use std::sync::Arc;

trait NoteCounter {
    async fn count(&self) -> u64;
}

struct Fixed;

impl NoteCounter for Fixed {
    async fn count(&self) -> u64 {
        3
    }
}

#[tokio::main]
async fn main() {
    let counter: Arc<dyn NoteCounter + Send + Sync> = Arc::new(Fixed);
    println!("{}", counter.count().await);
}
```

```
error[E0038]: the trait `NoteCounter` is not dyn compatible
  --> examples\e05_dyn.rs:20:26
   |
20 |     let counter: Arc<dyn NoteCounter + Send + Sync> = Arc::new(Fixed);
   |                          ^^^^^^^^^^^ `NoteCounter` is not dyn compatible
   |
note: for a trait to be dyn compatible it needs to allow building a vtable
      for more information, visit <https://doc.rust-
lang.org/reference/items/traits.html#dyn-compatibility>
  --> examples\e05_dyn.rs:7:14
   |
 6 | trait NoteCounter {
   |       ----------- this trait is not dyn compatible...
 7 |     async fn count(&self) -> u64;
   |              ^^^^^ ...because method `count` is `async`
   = help: consider moving `count` to another trait
   = help: only type `Fixed` implements `NoteCounter`; consider using it directly
instead.

For more information about this error, try `rustc --explain E0038`.
error: could not compile `rustnote` (example "e05_dyn") due to 1 previous error
```

- **何が起きたか**: 「trait `NoteCounter` は dyn で使えません」というエラーです。`...because method count is async`（`count` が `async` だから）と、理由が書いてあります。
- **読み方**: `dyn` で使うには、メソッドの表（vtable）を作れる必要があります。`async fn` は、実装ごとに大きさの違う Future を返すので、表に入れられません。
- **直し方**: RustNote のように、trait と実装の両方に `#[async_trait]` を付けます。実装が1つしか無く、差し替える予定も無いなら、`help:` のとおり、`dyn` を使わずに、その型を直接使う手もあります。

エラー2: `connect()` に `.await` を付け忘れた（E0277）

```rust
//! 第13巻 UNIT 05 エラー2: 接続を作る connect() に、.await を付け忘れた。
//! 実行: cargo run -q --example e05_await

use rustnote::db::Db;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("e05.db")).await?;
    let conn: libsql::Connection = db.connect()?;
    conn.execute("SELECT 1", ()).await?;
    Ok(())
}
```

```
error[E0277]: the `?` operator can only be applied to values that implement `Try`
  --> examples\e05_await.rs:10:36
   |
10 |     let conn: libsql::Connection = db.connect()?;
   |                                    ^^^^^^^^^^^^^ the `?` operator cannot be
applied to type `impl Future<Output = Result<Connection, libsql::Error>>`
   |
   = help: the nightly-only, unstable trait `Try` is not implemented for `impl
Future<Output = Result<Connection, libsql::Error>>`
help: consider `await`ing on the `Future`
   |
10 |     let conn: libsql::Connection = db.connect().await?;
   |                                                 ++++++

For more information about this error, try `rustc --explain E0277`.
error: could not compile `rustnote` (example "e05_await") due to 1 previous error
```

- **何が起きたか:** 「`?` は、`Try` を実装した値にしか使えません」というエラーです。`connect()` は `async fn` なので、呼んだだけでは、まだ実行されていない Future（`impl Future<Output = Result<…>>`）が返ります。
- **直し方:** `help:` のとおり、`?` の前に `.await` を付けます。リポジトリのメソッドも全部 `async fn` なので、呼ぶ所では `.await?` がひと組です。

エラー3: SQL の値に、`UserId` をそのまま渡した（E0599）

```
//! 第13巻 UNIT 05 エラー3: SQL の値に、UserId をそのまま渡した。
//! 実行: cargo run -q --example e05_params

use rustnote::db::{self, Db};
use rustnote::domain::UserId;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("e05.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;
    let owner = UserId::new(1)?;
    let mut rows: libsql::Rows = conn
        .query(
            "SELECT COUNT(*) FROM notes WHERE user_id = ?1",
            libsql::params![owner],
        )
        .await?;
    if let Some(row) = rows.next().await? {
        println!("{}", row.get::<i64>(0)?);
    }
    Ok(())
}
```

```
error[E0599]: the method `into_value` exists for struct `UserId`, but its trait bounds
were not satisfied
  --> examples\e05_params.rs:17:13
   |
17 |             libsql::params![owner],
   |             ^^^^^^^^^^^^^^^^^^^^^^ method cannot be called on `UserId` due to
unsatisfied trait bounds
   |
  ::: src\domain.rs:44:1
   |
44 | pub struct UserId(i64);
   | ---------------- doesn't satisfy `UserId: Into<Value>`, `UserId: IntoValue` or
`UserId: TryInto<Value>`
  …（満たされていない条件の一覧が続く）…
error: could not compile `rustnote` (example "e05_params") due to 1 previous error
```

- **何が起きたか:** `libsql::params!` は、中の値を `into_value` で libSQL の値（`Value`）に変えます。`UserId` には、その変換（`Into<Value>`）がありません。
- **読み方:** 長いエラーですが、`doesn't satisfy UserId: Into<Value>` の1行が要点です。

- **直し方**: RustNote のように `owner.get()` で中の `i64` を取り出して渡します。DB に渡す所でだけ、検査済みの型から中身を出します。

RustNote ではここで使う

- **共有の状態**: `src/state.rs` の `AppState::new` が、3つの libSQL の実装を作って、`Arc<dyn …>` で持ちます（UNIT 06）。
- **ルート**: ノートの一覧・作成・編集・切り替え・削除のルートが `state.notes` を（UNIT 08）、登録が `state.users.create` を、ログインの照合が `find_by_email` を、セッションの確認が `state.sessions` を使います（UNIT 06・07）。
- **テスト**: `escape_like` のユニットテストが1つと、`tests/repository.rs` の結合テストが8つあります（UNIT 10）。結合テストは、一時フォルダに本物の DB を作って試します。

```
running 1 test
test repository::notes::tests::like_special_characters_are_escaped ... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 18 filtered out; finished
in 0.00s
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test sessions_expire ... ok
test deleting_a_session_logs_out ... ok
test update_uses_optimistic_locking ... ok
test notes_are_isolated_per_owner ... ok
test toggle_and_delete ... ok
test search_filters_by_keyword_and_paginates ... ok
test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.20s
```

練習問題

問1: `escape_like` を使わずに、キーワードをそのまま `%…%` に入れて LIKE で探すと、`100%` と `a_b` の検索は、どうなるでしょうか。

問2: `notes.rs` の `find` の SQL から、持ち主の条件（`AND user_id = ?2`）を消すと、何が起きるでしょうか。どのテストが止めてくれるでしょうか。

問3: `update` で 0 行だったとき、`find` で見分けずに、いつも `VersionMismatch` を返すようにすると、何が困るでしょうか。

解答と解説

問1の解答:

同じ4つのタイトルに対して、escape しないパターンと、escape したパターン（ `ESCAPE '\'` 付き）で探しました。

```rust
//! 第13巻 UNIT 05 問1: LIKE の % と _ を、escape せずに検索したら、どうなるか。
//! 実行: cargo run -q --example u05_q1

use libsql::Connection;
use rustnote::db::{self, Db};

/// LIKE で title を探し、当たったタイトルを集める
async fn titles(conn: &Connection, sql: &str, pattern: &str) ->
anyhow::Result<Vec<String>> {
    let mut rows = conn.query(sql, [pattern]).await?;
    let mut found = Vec::new();
    while let Some(row) = rows.next().await? {
        found.push(row.get::<String>(0)?);
    }
    Ok(found)
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u05_q1.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;
    conn.execute(
        "INSERT INTO users (email, password_hash) VALUES ('alice@example.com', 'x')",
        (),
    )
    .await?;
    for title in ["100%オフ", "1000円", "a_b の計算", "acb の計算"] {
        conn.execute(
            "INSERT INTO notes (user_id, title, body) VALUES (1, ?1, '')",
            [title],
        )
        .await?;
    }

    let plain = "SELECT title FROM notes WHERE title LIKE ?1 ORDER BY id";
    let escaped = "SELECT title FROM notes WHERE title LIKE ?1 ESCAPE '\\' ORDER BY
id";
    for (keyword, raw, safe) in [("100%", "%100%%", "%100\\%%"), ("a_b", "%a_b%",
"%a\\_b%")] {
        println!("キーワード 「{keyword}」");
        println!(
            "  escape なし 「{raw}」 → {:?}",
            titles(&conn, plain, raw).await?
        );
        println!(
            "  escape あり 「{safe}」 → {:?}",
            titles(&conn, escaped, safe).await?
```

```
        );
    }
    Ok(())
}
```

```
キーワード 「100%」
  escape なし 「%100%%」 → ["100%オフ", "1000円"]
  escape あり 「%100\%%」 → ["100%オフ"]
キーワード 「a_b」
  escape なし 「%a_b%」 → ["a_b の計算", "acb の計算"]
  escape あり 「%a\_b%」 → ["a_b の計算"]
```

- escape しないと、`100%` の `%` が「何文字でもよい」になり、「1000円」まで当たりました。`a_b` の `_` は「どれか1文字」になり、「acb の計算」まで当たりました。
- 利用者は「`100%` という文字」を探したいので、escape した結果が正しい答えです。RustNote の `search` は、escape してから探すので、例5-4のとおり、`100%` は1件だけでした。

問2の解答:

教材のコードのコピーで、`find` の SQL を `WHERE id = ?1` だけにして、例5-4とリポジトリのテストを動かしました。

```
alice が探す: Some("買い物")
bob が探す: Some("買い物")
bob が更新: Err("VersionMismatch")
```

```
running 1 test
notes_are_isolated_per_owner --- FAILED
thread 'notes_are_isolated_per_owner' (224780) panicked at tests\repository.rs:78:5:
assertion failed: notes.find(bob.id, note.id).await.unwrap().is_none()
test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 7 filtered out;
finished in 0.04s
```

- bob が、alice のノートを**見つけられる**ようになりました。ノートの画面のルートは、`find` で自分のノートを探すので（UNIT 08 の `find_own_note`）、bob が `/notes/1` を開けば、alice のノートが表示されることになります。
- 更新の SQL には持ち主の条件が残っているので、bob の更新は書き込まれません。ただし、0 行のあとの `find` が見つけてしまうので、答えが `NotFound` から `VersionMismatch`（409）に変わりました。
- 結合テストの `notes_are_isolated_per_owner` が、78 行目（「bob からは見えない」）で止めました。持ち主の条件は、1つ消しただけでは、コンパイルも、ほかのテストも通ってしまいます。「**他人からは見えない」をテストに書いておく**ことが、この種のまちがいを止める、ほとんど唯一の方法です。

問3の解答:

教材のコードのコピーで、`update` の最後を `Err(RepoError::VersionMismatch)` だけにして、例5-4とリポジトリのテストを動かしました。

```
bob が探す: None
古い version 1 で更新: Err("VersionMismatch")
bob が更新: Err("VersionMismatch")
切り替えのあと、version 2 で更新: Err("VersionMismatch")
```

```
running 8 tests
notes_are_isolated_per_owner --- FAILED
thread 'notes_are_isolated_per_owner' (220380) panicked at tests\repository.rs:82:5:
assertion failed: matches!(notes.update(bob.id, note.id, &changed, 1).await,
    Err(RepoError::NotFound))
test result: FAILED. 7 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.26s
```

- bob が alice のノートを更新しようとすると、`NotFound`（404）ではなく `VersionMismatch`（409「ほかの画面で先に更新されました。読み込み直してください」）になりました。存在しない ID なら 404 なのに、alice のノートの ID なら 409 なので、bob は、「その番号のノートが、だれかのものとして**ある**」ことを知ってしまいます。
- `notes_are_isolated_per_owner` が、82 行目（「他人は更新できない（NotFound）」）で止めました。ほかの 7 つのテストは通りました。
- `find` をもう一度呼ぶのは、更新が失敗したときだけなので、ふだんの更新は、問い合わせ1回のままです。

まとめ

- `src/repository.rs` は、3つの trait と libSQL の実装を `pub use` で公開する。ルートは trait だけを知り、`AppState` が `Arc<dyn … + Send + Sync>` で実装を持つ。`async fn` の trait を dyn で使うには `#[async_trait]` が要る（無いと E0038）。
- `notes.rs` は、どの SQL にも `user_id = ?` を入れ、他人のノートを「無い」と同じにする。条件を消すと、コンパイルもほかのテストも通り、`notes_are_isolated_per_owner` だけが止めた。
- `update` は `version` を比べる楽観的ロック。0 行なら `find` で `VersionMismatch`（409）と `NotFound`（404）を分ける。分けないと、他人のノートの存在が 409 で漏れる。`toggle_done` も `version` を上げる。
- 一覧は `COUNT` と `LIMIT/OFFSET` の2回。LIKE は `escape_like` と `ESCAPE '\'` で `%`・`_` をただの文字にする（しないと `100` が「1000円」に当たる）。
- `users.rs` はハッシュ付きの `UserRecord` をログインの照合のときだけ返し、`sessions.rs` はトークンの SHA-256 を `BLOB` で保存して、期限内の行だけを `JOIN` で探す。

この巻のまとめ

この版に収録した UNIT 01〜05 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: 版を固定する設定ファイル

- ☐ `Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml`・`rustfmt.toml`・`.gitignore`・`.env.example` の6つについて、読む道具と、決めていることを言える
- ☐ `"=1.53.3"`（その版だけ）と `"1.53.3"`（1.53.3 以上 2.0.0 未満）の違いを、`cargo update -p tokio --dry-run` の結果で説明できる
- ☐ `rust-version`（最低の版）と `rust-toolchain.toml`（このフォルダで使う版）の違いと、`overridden by` の意味を説明できる
- ☐ `--locked` が `Cargo.lock` の書き換えを止めることと、`Dockerfile` と `ci.yml` が `--locked` を付けている理由を説明できる
- ☐ features は依存のつながり全体で足し算されることを、`cargo tree -f "{p} [{f}]"` の出力で確かめられる。自分が使う機能は、自分の `Cargo.toml` に書く
- ☐ `cargo tree -i クレート名` で、あるクレートを誰が使っているかをたどれる（`ring` ← rustls ← hyper-rustls ← libsql）
- ☐ 版が古い（`requires rustc 1.98`）・機能の名前の打ち間違い・`--locked` の食い違い・checksum の食い違い・`#[tokio::main]` の E0433 を読み、直すファイルを決められる
- ☐ `git check-ignore -v` で、ファイルが `.gitignore` のどの決まりで外されるかを確かめられる

UNIT 02: 表を作る SQL と DB の入口

- ☐ `migrations/` の3つの SQL ファイルが作る表・索引・制約と、記録簿の `schema_migrations` を言える
- ☐ `db::migrate` が、未適用の版だけを古い順に、1つずつトランザクションの中で適用し、SQL の実行と記録を同じトランザクションに入れる理由を説明できる
- ☐ `migrate` を2回かけても2回目は何もしない（冪等である）ことを確かめられる
- ☐ WAL（ファイルに記録される）と、`busy_timeout`・`foreign_keys`（接続ごと）の違いを、実際の値で説明できる
- ☐ マイグレーションが途中で失敗したとき、作りかけの表と索引が `rollback` で残らないことと、直したあとに失敗した版だけが流れることを確かめられる
- ☐ 適用済みの SQL ファイルを書き換えてはいけない理由を、古い DB と新しい DB の食い違いで説明し、変更を新しい版として足せる
- ☐ `EXPLAIN QUERY PLAN` で、一覧の SQL が索引を使っているかを確かめられる

UNIT 03: 設定と起動の順番

- ☐ src/config.rs が環境変数から Config を作る決まり（既定値・Turso ならトークン・空のトークン）を、実際の結果で説明できる
- ☐ Secret 型が {:?} でトークンを隠し、{} では表示させない仕組みと、expose() の役目を言える
- ☐ src/main.rs の起動の順番を言え、どの段で止まると DB のファイルが残るかを、本物の実行ファイルで確かめられる
- ☐ .env は、すでにある環境変数を上書きしないことを確かめられる
- ☐ src/lib.rs の pub mod を消したときの E0432 と、edition 2024 の set_var の E0133 を読める

UNIT 04: ドメインの型とエラーの分類

- ☐ 検査済みの型の中身が private で、parse を通らずには作れない（E0423）ことと、UserId と NoteId を取り違えられない（E0308）ことを説明できる
- ☐ タイトル・本文・メール・パスワード・ID の数え方とそろえ方を、例4-3の結果（NUL・絵文字・全角）で言える
- ☐ NewNote::parse が問題を全部集める理由を、? で書いた版と比べて説明できる
- ☐ RepoError と AppError の2つの層、#[from] と From による ? の変換（無いと E0277）、status_code と public_message の役目を言える
- ☐ NoteQuery の pub な項目で page: 0 を作ると、debug と release でどうなるかを確かめ、直し方を言える

UNIT 05: リポジトリ

- ☐ trait と libSQL の実装を組にし、AppState が Arc で持つ理由と、#[async_trait] が要る理由（無いと E0038）を説明できる
- ☐ notes.rs のどの SQL にも user_id = ? が入っていることを示し、消すと notes_are_isolated_per_owner だけが止めることを確かめられる
- ☐ 楽観的ロックの update が、0 行のときに find で VersionMismatch（409）と NotFound（404）を分ける理由を言える
- ☐ LIKE の % と _ を escape しないと、100 で「1000円」まで当たることを示せる
- ☐ UserRecord（ハッシュ付き）と User の使い分けと、セッションをトークンの SHA-256 で探す仕組みを言える

> **お知らせ:** この版には UNIT 01〜05 を収録しています。UNIT 06（ログイン・お知らせ・共有の状態）・UNIT 07（画面の枠・部品・登録とログインの画面）・UNIT 08（ノートの CRUD）・UNIT 09（見た目）・UNIT 10（テスト・Docker・Fly.io・CI）と、巻末の総合演習・解答・全巻の用語集・索引は準備中で、次の更新から順に追加します。それまでの間、RustNote のファイルは、教材のフォルダの `Rust/rustnote/` で直接読めます。どのファイルも、先頭のコメントに役割が書いてあります。

用語ミニ辞典

この巻（UNIT 01〜05）に出てきた用語です。

UNIT 01 の用語

用語	意味
版の固定（pin）	使う道具や部品の版を、1つに決めて書いておくこと
依存クレート	自分のプログラムが使う、ほかの人が作ったクレート。そのクレートが使うクレートも、間接的な依存になる
`Cargo.toml`	パッケージの名前と、使うクレートと版を、人が書くファイル
`Cargo.lock`	cargo が選んだ全クレートの版と checksum を記録するファイル。手で書き換えない
checksum	クレートの中身から計算した指紋（SHA-256）。取ってきた中身が記録と同じかを確かめる
`--locked`	`Cargo.lock` の書き換えが必要なら、ビルドせずにエラーで止める指定
`rust-version`	そのパッケージに必要な、Rust の最低の版
`rust-toolchain.toml`	そのフォルダで使う Rust の版を決めるファイル。rustup が読む
features	クレートの機能のスイッチ。依存のつながり全体で足し算される
`[dev-dependencies]`	テスト・examples でだけ使うクレート。本番の実行ファイルには入らない
`cargo tree`	依存のつながりを木の形で表示する。`-i` で逆向きにたどる
`.gitignore`	Git に入れないファイルの名前の決まり
`.env` / `.env.example`	環境変数を書くファイルと、その見本。`.env` は Git に入れない
E0433	名前（モジュールやクレートの中の物）が見つからないエラー
E0752	`main` 関数に `async` を付けたエラー

UNIT 02 の用語

用語	意味
マイグレーション	表の作り（スキーマ）を変える手順を、番号順のファイルに分けて書いたもの
`schema_migrations`	適用済みのマイグレーションの版の名前と時刻を記録する表
トランザクション	いくつかの変更を、全部やるか全部やめるかの1まとまりにする仕組み。`commit` で確定、`rollback` で取り消す
冪等	同じ操作を何回くり返しても、1回したときと同じ結果になること
`include_str!`	ファイルの中身を、コンパイルのときに文字列として実行ファイルに埋め込むマクロ
WAL（`journal_mode = wal`）	変更をまず別のファイル（`-wal`）に書く方式。読み込みと書き込みが重なっても待ちにくい。ファイルに記録される
`busy_timeout`	ほかの接続が書き込み中のとき、何ミリ秒まで待つか。接続ごとの設定
`PRAGMA foreign_keys`	外部キーの確かめを ON にする SQLite の設定。SQLite では接続ごとに入れる
`sqlite_sequence`	`AUTOINCREMENT` の次の番号を、SQLite が自動で記録する表
`EXPLAIN QUERY PLAN`	SQL を、DB がどう実行するつもりか（索引を使うか、全部読むか）を表示する
`SQLITE_CANTOPEN`（14）	DB のファイルを開けなかったときのエラーの番号。フォルダが無いときなど

UNIT 03 の用語

用語	意味
環境変数	プログラムの外から渡す名前と値の組。RustNote は DATABASE_URL などを読む
`.env`	開発のとき、環境変数をまとめて書いておくファイル。すでにある環境変数は上書きしない
`Config`	RustNote の設定（DB の URL と、Turso のトークン）
`Secret`	秘密の値を包む型。`{:?}` では `Secret(********)`、`{}` は使えない
`expose()`	`Secret` の中身を取り出す、わざと目立つ名前のメソッド
`anyhow::Context`	エラーに「何をしようとして失敗したか」を付ける。main の `Error:`・`Caused by:` の表示になる
`pub mod`	ファイルを、クレートの部品（モジュール）にする宣言

UNIT 04 の用語

用語	意味
検査済みの型	決まりの検査に合格した値だけが入る型。中身を private にして、parse を通らないと作れないようにする
`thiserror`	`#[derive(Error)]` と `#[error("…")]` で、エラーの型とその文を短く書けるようにするクレート
`#[from]`	thiserror の印。`From` を作るので、`?` でそのエラーを自動で変換できる
`#[source]`	thiserror の印。元のエラーを「原因」として持っておき、ログに原因まで書けるようにする
`RepoError`	RustNote の DB の層の失敗（一意制約の違反・version の食い違い・見つからない など）
`AppError`	RustNote の、利用者への返事を決める分類。HTTP の状態コードに対応する
`public_message`	画面に出してよい文。500 のときは原因を隠す
`FromStr`	`"42".parse::<NoteId>()` のように、文字列から型を作れるようにする trait

UNIT 05 の用語

用語	意味
リポジトリ	データのしまい方・取り出し方を引き受ける部品。ほかの部品は SQL を知らずに頼む
`#[async_trait]`	async fn を持つ trait を、dyn で使える形に書き換えるマクロ
`Arc<dyn Trait + Send + Sync>`	実装を実行のときに選び、複数のスレッドで共有して使うための形
楽観的ロック	先に鍵をかけず、保存するときに版の番号（version）を比べて、食い違いを見つける方法
`RETURNING`	INSERT・UPDATE の終わりに付けて、書き込んだ行をその場で受け取る SQL の書き方
`ESCAPE '\'`	LIKE の中で、`\` の次の文字（`%`・`_`）を、ただの文字として扱わせる指定
`UserRecord`	パスワードのハッシュ付きのユーザー。ログインの照合のときだけ使う
`TokenHash`	セッションのトークンの SHA-256（32 バイト）。DB にはトークンそのものではなく、これを保存する